8

PASTA USE CASE

PASTA USE CASE EXAMPLE WALK-THROUGH

> "Tell me and I forget. Teach me and I remember. Involve me and I learn."
>
> *Benjamin Franklin*

In this chapter, we show how to use the PASTA risk-centric threat modeling process to analyze the risks of specific threat agents targeting a web applications and specifically the web application assets that include customer's confidential data and business critical functionality that the web application provides. Among the web application assets in scope for the protection of threats, we will also consider information technology assets such as the application software components, applications, systems, and services where this software is installed and run. The goal of this risk-centric threat modeling exercise is to determine the technical and business impact of opportunistic and targeted threat actors against the web application assets and to recommend protective and detective security controls that can be designed, implemented, and deployed to protect the web application assets from these threats and reduce the risk to the organization/business, that is, responsibility to either own or manage the web application assets.

Throughout this chapter, we will use NIST National Institute for Standards and Technology terminology and standard definitions for threats, vulnerabilities, attacks, and risks as well as NIST standard definitions for risk management activities such

Risk Centric Threat Modeling: Process for Attack Simulation and Threat Analysis, First Edition.
Tony UcedaVélez and Marco M. Morana.
© 2015 John Wiley & Sons, Inc. Published 2015 by John Wiley & Sons, Inc.

as threat analysis and risk management. *Note:* Refer to the book glossary for specific risk management terminology used throughout this book including this chapter. Within this use of terminology and in the context of managing web application risks, the Process for Attack Simulation and Threat Analysis is defined as "*a risk-centric threat modeling process aimed at considering possible threats scenarios, attacks and vulnerabilities within a proposed or existing web application environment for the purpose of assessing the risks and managing technical and business impacts.*"

We already defined the PASTA risk-centric threat modeling process in Chapter 6. What follows after this introduction to PASTA is an example/use case for conducting a risk-based threat modeling of a web application by walking through the various stages and activities of the PASTA Process for Attack Simulation and Threat Analysis. For this walk-through, we choose a web application that can be considered at high risk of attacks from value-driven threat actors seeking to steal confidential data such as bank accounts and credit card data as well compromise business critical functionality that such web application could provide such as accessing and checking bank accounts online, making payments such as paying bills and transfer money. Today such a type of web application, because it stores and processes valued data as well as money, is highly sought target by money-driven threat agents such as fraudster and cyber-criminals. Since such web application can also be reached over the Internet and is highly visible website with globally recognized company, it is also highly sought target by politically motivated threat actors seeking to impact the company reputation with defacing the company website and by attacking it with denial-of-service attacks.

A good example of a web application that fit this profile is a financial type of web application such as online banking. With PASTA, we would like to analyze the threat and risk profile of such web application from both compliance and cyber-threat risk perspective. We would like to determine the technical scope of the web application and identify the assets at risk to determine how well these are protected from targeted and opportunistic attacks. To know how attacks from various threat agents affect the web application before gets attacked we would like to simulate these attacks and identify any security control weaknesses and vulnerabilities that these attacks can possibly exploit. After we have simulated the attacks, we would like to identify countermeasures that can be designed, implemented, and deployed to effectively detect and protect from these attacks and reduce the risk for the organization of possible confidential data compromise and online fraud. These general risk management objectives just stated can be achieved by following the seven stages of PASTA process depicted in Figure 8.1.

A brief summary of what each of these seven stages of PASTA consists of and specifically what will be the goal of each stage as related to the web application context and online banking use case is provided herein. Stage I of PASTA consists of the "Definition the Objectives (DO) for the Analysis of Risks" in stage I, we will demonstrate how to derive the security and compliance requirements, determine the business impacts and derive the risk profile that is the basis of a set of high-level objectives for securing the web application for security/compliance but also for the risks due to emerging cyber-threats affecting the business and technical environment in which the application is planned to operate.



Figure 8.1 PASTA Threat Modeling: Stages and Activities

Stage II of PASTA consists of the "Definition of the Technical Scope (DTS)" of the process. This stage is where we capture and enumerate the various technical details of users and functional accounts, software and architecture components including system-level services and third-party infrastructure components. After these various components of the application architecture and technology stack that support them are captured through enumeration in checklists and worksheets, they also need to be asserted in the current implementation and included in the technical documentation of the application in scope. This technical documentation that includes the architecture design documentation and technical functional documentation will also include the security requirements previously identified and asserted for completeness.

During stage III, that is during the "Application Decomposition and Analysis (ADA)" of the threat modeling process, we will decompose the application in basic elements of application functionality that includes the various type of users, and their roles, the various type of data that is accessed as data in storage (e.g. databases) or in transit (e.g. data flows), the boundaries of trust that are crossed by the data to access functionality and resources and the security controls that protect data and functions from external and internal threat agents.

Once we have decomposed the application and analyzed how the application components interact with the data in transit and storage and we have identified the data and functional assets inherent risk due to the sensitivity and exposure of these assets, we can start analyzing the various threats against these components and assets to determine if these are potentially at risk of attacks. This can be done during stage IV, "Threat Analysis (TA)." Specifically, we will show how we can conduct the analysis of specific threats targeting the application whose technical scope was defined in stage II and whose functional and data assets were identified in stage III.

Initially we will look at the threat landscape and analyze the threat scenario/environment for the application in scope. Specifically during the threat analysis, we look at analyzing the threat scenarios and we seek to gather threat intelligence from internal as well as external sources to understand the type of threats agents and threat targets sought by these threat agents including the type of weaknesses and vulnerabilities that these threat agents seek to exploit using different Tools, Techniques, and Processes (TTPs).

We will show how the categorization of the threat agents can be done according to their motives (e.g. political, ego-driven, and money-driven), the threat agent's capabilities (e.g. financial resources, attacking tools, and knowledge of attack techniques), and the threat agent caused security incidents (e.g. publicly reported data breaches). In this stage, we will use a model for threat agents that is instrumental to the creation of a threat library/knowledge base that associates threats with the who (i.e. the threat agents or actors) the how (i.e. the attack tools and techniques used and the vulnerabilities that are exploited) and the what (i.e. the targets in terms of data and functional assets sought).

After the threat intelligence information is collected from sources of internal and external threat agents, we will update the threat libraries and map the threats to the assets targeted so that we can model the threats against these assets later on during the attack modeling stage. In this stage, we will show how this threat library/knowledge base can be updated with threat sources from threat intelligence and internal logged events.

We will also show how to assign a probability to these threats so that these can be prioritized later on for risk assessments including vulnerability assessments specifically for the vulnerabilities and weaknesses that these threats seek to exploit and have high probability to be realized in attacks against the application assets (e.g. data and functions).

During stage V, that is, during the "Weakness and Vulnerability Analysis (WVA)," we will show how each threat of the threat library can be associated with a previously

assessed vulnerability or gaps in the security controls of the application (e.g. authentication, authorization, encryption, session management, data validation, audit, and logging). During this stage, we also look in depth on how the application is designed, which technical security controls are in place both at the architectural level and functional level to identify any weaknesses that might expose the application assets to the previously identified threats. These are weaknesses in security controls that might expose the assets (e.g. data or functions) to threat agents hence increasing the threat probability of such threat agents to be realized in attacks.

At the end of this stage, we will identify weaknesses and vulnerabilities that need to be prioritized for and validated by security tests in order to reduce the probability risk of each threat. These security assessments will also include security test cases to test the security requirements that were previously documented and included in the technical design in stage II. At the end of stage V, we will have a clearer picture of weaknesses and vulnerabilities affecting the various components of the application architecture and of the type of threat agents that might exploit them in the case of specific attack scenarios.

This will trigger threat-driven security assessments of these vulnerabilities and bring value added for previously executed security assessments that were done blind to threat and by considering weaknesses and vulnerabilities independently from the risk of specific threat actor exploiting them. The mapping of threats to vulnerabilities allows us to move from a vulnerability-centric assessment to a threat/risk-centric assessment. A vulnerability-centric assessment focuses on the identification and remediation of vulnerabilities such as web application vulnerabilities that are either introduced during design (e.g. design flaws) or source code (e.g. coding errors). The identification of the design flaws and coding errors should be the focus of specific assessment such as architectural risk analysis and source code analysis as well as manual assessment such as secure code review, manual security tests such as ethical hacking pen test (referred as black-box testing) and automatic security tests scanning for vulnerabilities such as dynamic and static source code analysis.

When we use the PASTA methodology to conduct a risk-centric threat modeling exercise, the focus of vulnerability assessments is threat/risk centric and PASTA allows us reassess these vulnerabilities with the lenses of threat agents trying to exploit them. By applying the PASTA to the application, we will leverage any vulnerability data of weaknesses and vulnerabilities that was identified in previous vulnerability assessment conducted on the web application in scope for the analysis. The focus of at this stage is to gather data such as vulnerability reports on any vulnerabilities and weaknesses such as design flaws and implementation type of vulnerabilities that are introduced in software and libraries as well as misconfigurations of systems and network infrastructure components and that the threats previously analyzed could target in their exploits and use for attacks against the web application.

Among the weaknesses in security controls any design flaws in the design of the online banking application such as weaknesses in authentication, managing of authenticated sessions, errors in the design of user authorization, missing input validation of data as well as encryption of highly sensitive data such as authentication data, encryption keys and confidential data of customers take a specific focus in the

analysis of the risks of the application architecture. At this stage, we should have a good picture of vulnerabilities and weaknesses that have been introduced because of errors in design, source code and implementation of security controls to protect data and functional assets. We can therefore prioritize specific vulnerability assessments such as ethical hacking pen tests and source code analysis and code reviews using both automated and manual techniques to validate and assert the severity of these weaknesses and vulnerabilities in the threat environment that we previously analyzed.

The other core of the PASTA assessment besides the modeling and the analysis of threats are the attack modeling and simulation. During stage VI "Attack Modeling and Simulation (AMS)," we will analyze the attack scenario for each specific threat to determine the likelihood of the threat occurring and the technical impacts in case of exploits of a condition such as either a control gap or vulnerabilities previously identified. The attack scenario will be analyzed by looking at the course of actions and the events that are triggered by the attacks so that these can be detected and alerted for actions (e.g. through a security incident response). Attack trees to learn about the attacker goals and methods can be used as methodology to determine the path of attacks sought by the attackers as path of least resistance and minimum cost hence bearing the highest probability to be executed to cause damage to the application.

Attack modeling seeks to model and simulate the attacks used against the web application to analyze how these attacks operate from the step of initial deployment of the agent of the attack (e.g. malware or bot) to the installation in the compromised systems to the conduction of the exploit of vulnerabilities and weaknesses in the web application's security controls to bypass them and to the final execution of the attack to cause the desired impact including the exfiltration of the data, outage of the online services provided and, last but not least, the steal of monetary assets from the victims (e.g. money from the bank accounts). The modeling of the attacks and the simulation of the attack scenario are critical for the derivation of attack-driven security tests whose goal is to validate how weaknesses and vulnerabilities are exploited as well as the effectiveness of exiting countermeasures in mitigating the impacts. Based on the attack simulations and tests, it is possible to determine which weaknesses and vulnerabilities should be prioritized for fixing including which countermeasures should be designed and deployed to detect and protect from the attacks. The identification of the countermeasures depends on different factors besides the effectiveness and the reduction of the initial risk of each threat to a residual that can be accepted also the costs for implementing them.

At the end of this stage, we will derive a list of attack simulation test cases that include the most probable attacks leading to exploits of control gaps and vulnerabilities and conduct simulated test cases of these attacks to determine whether the security measures that include both detective and preventive controls that are currently deployed are sufficient to mitigate the risk of a threat to be realized. The goal of the simulated tests is to consider additional security measures and controls to determine the security control effectiveness in the prevention and detection of possible attacks.

Finally in stage VII, "Risk Analysis and Management (RAM)" we will analyze the information in hand such as the threats and the simulated tests of attack scenarios to identify the technical impacts and to determine the business impacts on the assets and the type of security measures (e.g. preventive and detective controls) that can be implemented in order to mitigate the risks of the various threats considered including DDoS and malware banking hacking threats. The types of security measures that are recommended are the ones that are proved most effective in the mitigation of the risks of threats. During this stage, we will assess the technical and business impacts with the security measures in place and the reduction of these assuming that additional countermeasures will be deployed. The costs and effectiveness of these security measures will be evaluated and commiserated with the impacts to the business in the case these measures either will not be implemented. In some cases, the risk mitigation strategy consists of transferring the risks to a third-party. Another risk mitigation strategy might consist of eliminating the potential sources for risk such as by taking the critical business functionality off the Internet and made available only to internal users.

Stage I: Definition of The Objectives (DO)

Goals of This Stage The goal of the first stage of the application threat modeling methodology is the definition of the objectives for the analysis and management of the various types of cybersecurity risks affecting the application in scope. In order to derive these objectives, it is important to collect and analyze business requirements, information security policies and standards, privacy laws of organization/business and the application/product that are in scope for the assessment.

Guidance for the Execution of This Stage To define the objectives for the treatment of risk it is necessary to define information security requirements, compliance requirements, data privacy requirements and risk management requirements in the scope of the web application. Prerequisite for the execution of this stage is the reference to information security policies and standards, regulatory compliance, privacy laws, and business requirements.

Since information security policies and regulatory compliance apply also to web application assets that are owned and managed by the organization, it is important to review also application domain specific security standards if these are available. The outcomes of this stage are

1. The initial security and risk profile of the web application in scope for the analysis;
2. A set of documented high-level objectives for managing application risks including noncompliance with information security, unlawful regulatory noncompliance, business continuity risks, financial risks, and security incident risks of the web application.

Specific information security, compliance, and risk mitigation requirements can be documented during the different design phases of the SDLC for both new applications and for application changes. Business risks are derived from a preliminary risk analysis of potential business impacts based on the initial classification of the confidentiality of the data and the value given to the application as an asset for the organization.

In order to be ready to execute this stage, it is mandatory that an organization has adopted information security standards and policies for the protection of the organizational data assets that are processed by software and applications. From the compliance perspective, this includes applicable requirements derived from information security standards (e.g. ISO 27001) and industry specific requirements (e.g. PCI-DSS). For the security risk management perspective, the definition of risk mitigation objectives includes the definition of the inherent risks based upon the initial business functional requirements and exposure of the application assets and the inherent value of the asset.

At high level, a risk statement objective can be "the web application might expose high financial risk assets and functionality such as payment and money transfers to external threat agents and these risks are considered HIGH and need to be prioritized for a focused assessment of the application such as application threat modeling.

At high level, the main objectives include both information security, compliance, and risk management objectives and these constitute the initial set of high levels security and risk mitigation requirements for the application in scope. These are security requirements that can be followed both during design of the web application and to assess the various risks. At this stage, the severity level of cyber threats targeting the application that are received from threat advisories should also be taken into consideration (e.g. in the case of the financial sector, this will be FS ISAC).

Since threats will be later analyzed in stage IV of the PASTA™ threat modeling methodology the initial risk management objectives for these threats are mostly derived from what is known initially about the existing application/product such as the confidentiality of the data stored and processed, the exposure of the application to the Internet or intranet, the functionality at risk such as payments and money transfers as well as any new features and technologies that will be introduced in the application/product and will increase the overall risk profile of the application.

In regard to the risk profile of the application that is derived during this stage, the objective of this initial risk profile is mainly to identify at high level the inherent technical and business risks where the technical risks depend upon the risk of losing confidentiality, integrity and availability of the data assets while the business risks depend on the value given by the business to these assets whose exposure for the business in the case these assets are lost/compromised include a business impact.

Inputs for Conducting This Stage To conduct this stage, it is important to gather and assess information regarding the applicable information security policies and standard, applicable compliance regulations, and data privacy laws as well as to capture high-level business requirements for the application. This information could be already documented and if not documented it needs to be captured by conducting

interviews with the application stakeholders including application development units, application business owners, information security teams and continuity of business managers, risk managers and compliance officers.

A nonmandatory list of technical documentation that can be captured for conducting this stage includes the following:

1. High-level business objectives
2. IS policies and standards
3. Data classification policies
4. Compliance regulations and data privacy laws
5. Risk assessment processes and standards
6. Security incident response procedures
7. Asset IT inventory
8. Asset values

Artifacts Produced at the End of This Stage At the end of this stage, we will produce the following artifacts:

1. Functional business requirements
2. Functional security requirements
3. Analysis of business impacts
4. Risk profile of the application/IT asset in scope

Tools The execution of this stage can be facilitated by the availability of IT asset repository that allow the threat modeler user to update the IT asset with the risk profile information, the classification of the data and the security and risk analysis and management requirements that are specific for this asset. A repository of documentation such as web portal to access information security and application security standards and documentation as it applies to the application in scope is also useful. Ideally the threat analyst, application security architect, security software developer, or technical risk manager conducting the application threat modeling assessment should have access to all these resources from share repositories such as MS SharePoint. For conducting this stage, we rely on the use of the tool ThreatModeler™ developed by MyAppSecurity Inc.

Activities to Be Conducted In this Stage This stage consists of the following activities:

DO 1.1 – Document the business requirements for the new product/IT asset or application in scope for the threat modeling exercise. Application business requirements describe the application functionality from the business objectives perspective that is the service that is provided to customers/users of the application. These business requirements should already be documented otherwise a set of high-level business objectives can be used to derive security

functional requirements describing the business functionality of the application in scope.

DO 1.2 – Define the security requirements: at high level, these are security requirements for the protection of the data assets and the security parameters such as the confidentiality, integrity, availability, and accountability of the data accessed, processed, and stored by the application. The data protection requirements can be derived in compliance with data privacy laws with scope on the type of data that is considered private and need to be protected (e.g. personal identifiable information (PII)). The compliance driven security requirements depend on different factors such as the type of data that the application is meant to protect and the applicable technical security standards based upon the type of data that is processed: example, if the application handles credit card payment data, the application will be in scope for PCI-DSS technical standards that will drive security requirements for the protection of credit card data that can be documented for the design of the application that process credit card data such as encryption of credit card data during transmission and storage and for masking of the credit card data when is displayed by the application user interfaces.

DO 1.3 – Define the business impacts such as the potential impact to the organization/business in the case of the valued asset data such as confidential data and business critical functions are either lost or compromised such as in the case of a security incident. Examples of security incident whose business impacts need to be identified and estimated are the business impacts caused by the loss of confidential data, loss/degradation of services and system/application functionality as well as losses due to fraudulent transactions. Business impacts can be estimated as economic loss based upon the value of the data/functional asset for the organization that is lost in a security incident. Business impacts caused by the loss of the assets can be used to estimate the level of business risk by factoring the estimated frequency of the security incident with a data loss event and the expected monetary value that is lost in that event.

DO 1.4 – Determine the risk profile for the application/product in scope. The purpose of the risk profile is to capture the technical and business risks for the application/asset in scope based upon a set of high-level risk analysis and management objectives. From cyber-threat risk perspective, these are risk analysis objectives such as capabilities, motives, and attack vectors used and the determination of how these threats might be realized in attacks to cause a negative impact to the application assets. Technical risk management objectives are aimed at analyzing and reducing the impact of attacks against the application security controls. High-level objectives can be derived from applicable information security policies and standards and the analysis of the impact risk levels estimated in the risk profile assuming the type of threats and the inherent information security risks. Business risk management objectives aim to the management of the estimated impacts caused by cyber-threats when these cause

an impact to the business. The estimated business impact need to consider different factors such as legal and regulatory risks for noncompliance, loss of data privacy and the business impacts due to loss of data availability and service, and compromise of critical functionality at risk of security incidents caused by cyber-threats.

A risk profile can be defined after risk management objectives are identified. In the case of a security incident, risk management objectives ought to consider the risk of cyber-threats besides regulatory compliance and information security risks. The general notion of risks on which the risk-centric methodology is based is that cyber-threat risks is associated with a probability of a cyber-threat to be realized in a cyber-attack that can cause a tangible loss and negative impact due to the compromise of the assets. From quantitative risk management perspective, cyber-threats cause business impacts such as revenue-monetary loss, fraud loss, operational losses, fines and penalties from regulators, data privacy loss fines, license liabilities, and legal costs lawsuits.

The application risk profile is dependent on known information about the application operational environment and the inherent risks such as information security risks factors such as the data exposure, confidentiality and integrity. The noncompliance risks might include a possible loss of privacy and business impacts caused by potential security incidents with breaches of data. Besides the inherent risks that are driven by the asset values, the documented risk profile need to factor the exposure to threats such as the severity level of the cyber-threat as probability of the application product to be attacked. Estimate and assign risk impact levels for threat severity-probability of attacks, technical impacts and business impacts (High, Medium, and Low). The analysis of these risk factors includes also the analysis of the impact to the business based on previous estimates.

Web Application Use Case Example The first activity of this stage consists of the activity "*DO 1.1 – Document the business requirements.*" It is not the objective of stage I to document business requirements but to gather the business requirement for the application in scope. Typically any given product or service including a web application should have the business requirement documented. These are requirements that describe what the new product/IT system, software, and process in scope need to satisfy in order to provide value for the business/organization when satisfied.

Typically business requirements lead to description of WHAT should be implemented by the application and these requirements are documented in Business Requirement Documents (BRD). The definition of how to achieve these business requirements with the application software or product in scope is typically the scope of both functional and nonfunctional requirements that are captured in System Requirement Specifications (SRS) or System Requirement Documents (SRD). For the online banking web application in scope, we would like to capture the specific functional business requirements that describe the functionality of the online banking application as these are describing the various features that the application is meant to provide. The reason we focus on "business features" is because these

are "business assets" and therefore need to be protected from risk of threat agents seeking to compromise them for fraudulent transactions as well as for unauthorized access to confidential data. At high level, the business requirements for the online banking application in scope can be described from the perspective of the user bank customer being "the asset" for the organization as well as the confidential data that is stored and processed (e.g. bank accounts, credit card data) and the various business functions that are made available for the user/customer. From this perspective, the business requirements more closely resemble "business use cases" describing business functionality provided to the user of the application. For example, a nonexhaustive list of user business requirements for the online banking application includes the following Business Requirements (BR):

- *BR-1:* A bank customer can register to the online banking site using his debit card information and personal documentation.
- *BR-2:* A registered user can login into the online banking site.
- *BR-3:* A bank user can check his bank account statement, balance, and account transactions.
- *BR-4:* An authenticated user can apply for a new bank account.
- *BR-5:* An authenticated user can apply for a new loan.
- *BR-6:* An authenticated user can apply for a new credit card.
- *BR-7:* An authenticated user can pay bills online.
- BR-8: An authenticated user can transfer funds between accounts.
- *BR-9:* An authenticated user with multifactor authentication can transfer funds to a recipient of an external bank account through ACH.
- *BR-10:* An authenticated user can manage his or her account such as change his or her password, change the contact information (e.g. e-mail, postal address, and phone number).

In the description of these business requirements, we need to include a description of each of the type of users of the application (e.g. bank customer visitor a registered user and an authenticated user), the business function that is provided to each user and the type of data that is handled by the application (e.g. general public information or confidential information of the customer). This emphasis allows later to identify the type of user as well as the type of business functions that need to be provided and the sensitivity of the data that is processed.

As we start a new application threat modeling project, these business requirements can be captured using a threat modeling tool such as ThreatModeler™. In Figure 8.2, we show how the ThreatModeler™ tool wizard allows us to capture the business requirements associated with the online banking application in scope.

Once these business requirements are gathered and associated with the threat modeling project, the next step of the activity consists of conducting the activity "*DO 1.2 – Define the security/compliance requirements.*"

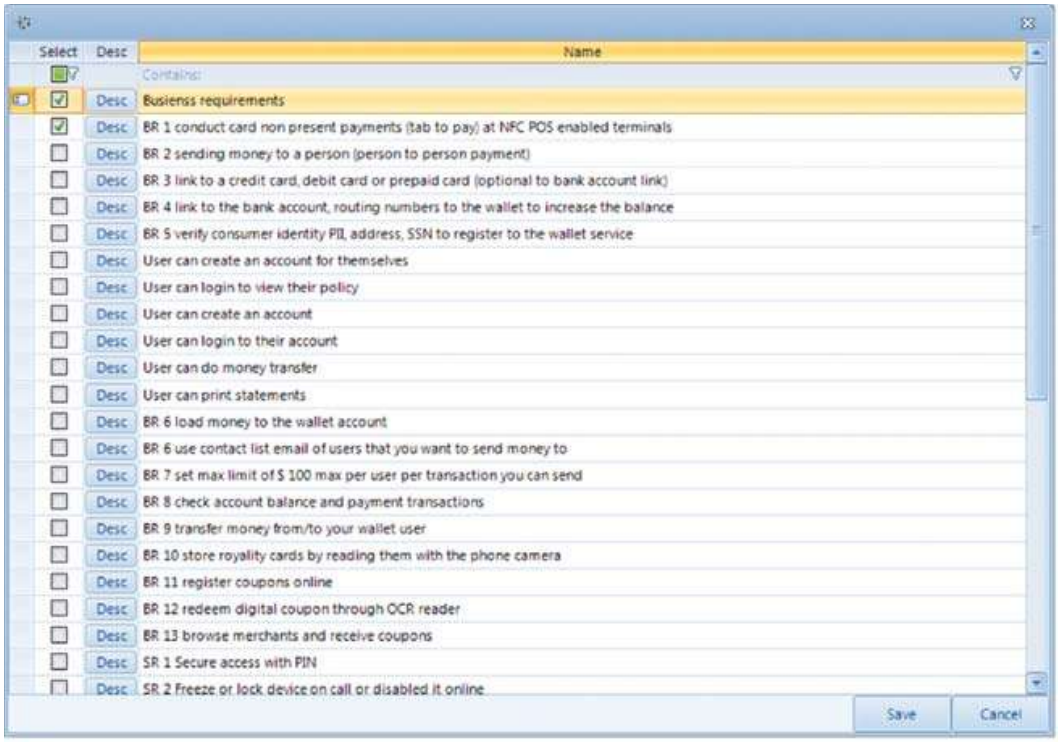


Figure 8.2 Entering Business Functional Requirements/Use Cases Using the ThreatModeler™ Threat Modeling Tool

Since the scope of security requirements is the protection of the confidentiality, integrity, and availability of the data and specifically the security requirements for protecting information that is considered confidential, it is important to identify the type of data that is processed and stored by the application. One possible way to identify which data is processed and stored is to look at each of the business requirements previously defined and identify the data that is stored or processed. The sensitivity of the data, that is, the data classification, depends on the organization's information security policy and the data classification policy. In order to classify the sensitivity of the data, it is necessary that the organization has adopted information security standards and policies (e.g. ISO 27001). These standards and policies need to be aligned with industry specific security requirements as it is mandated by the type of business vertical in which the organization operates and the type of data that is processed. For example, if the organization is developing a website that has in scope payment by processing credit card data, it will be in scope for PCI-DSS. PCI-DSS as well as other regulatory security standards might apply to the online banking application in scope as well. Since confidential data should be considered one of the organization assets that need to be protected, it is important to analyze the business requirements to determine if any confidential data is either processed or stored by the application. An example of this analysis is shown in Table 8.1.

After the type of data and the sensitivity has been identified using the organization data classification policy, it is possible to derive the security requirements for protecting these data based up the classification of the sensitivity of such data.

TABLE 8.1 Sensitive Data Analysis and Business Requirements of Online Banking Application

Business Functional Requirement	Data Classification of Data Processed by Each Business Function in Scope for the Business Functional Requirement
BR-1	Confidential Data (Bank Account, Card Number), Personal Identifiable Information (PII) (SSN, DOB)
BR-2	Username and authentication data (password)
BR-3	Confidential Data (Bank Account, Name, Surname, Address, Account Number)
BR-4	Confidential Data (Bank Account), Personal Identifiable Information (PII) (SSN, DOB)
BR-5	Confidential Data (Bank Account), Personal Identifiable Information (PII) (SSN, DOB)
BR-6	Confidential Data (Bank Account), Personal Identifiable Information (PII) (SSN, DOB)
BR-7	Confidential Data (Bank Account)
BR-8	Confidential Data (Bank Account)
BR-9	Authentication data (Secure Token/PIN), Confidential Data (Bank Account, Routing Number), Personal Identifiable Information (PII)
BR-10	Authentication data (password), Confidential data, Personal Identifiable Information (PII)

At high level, security requirements for the protection of the data that is processed and stored by the application in scope can be derived from the information security policies and standards that adopted by the organization/business. These information security policies and standards apply to all information data assets including the ones that are processed by the online banking application that include PII of customers and authentication data.

Typically information security policies include information security requirements for the security of digital assets (e.g. confidential data and business critical functions) that are owned and managed by the organization. At high level, information security requirements are not associated with the usage and function of the web application but rather are nonfunctional requirements that describe at high level the security constraints such as necessary security requirements that apply to protect the confidentiality integrity and availability of the confidential data and the business functions that handle such information.

From the perspective of the threat modeling exercise, we would like to focus on functional security requirements. These are requirements that need to be satisfied by the security controls such as authentication, authorization, data protection/encryption, data validation/filtering, session management, audit and logging and error management and exception handling.

An example of functional security requirements that can be derived for the online banking application based upon applicable standards and regulations are included

herein. *Note: these requirements are not comprehensive of all types of security requirements for an online banking application but provided as an example of high level requirements:*

SR-1: Passwords should be encrypted using irreversible encryption (e.g. hashes) using standard algorithms and minimum key lengths.

SR-2: Passwords should comply with the password management policy (e.g. minimum 8 characters, alphanumeric, masked on display and expired every 90 days).

SR-3: A password can be changed after authentication by providing old and new password. The new password should be typed twice.

SR-4: A password can be reset by validating the username and a set of challenge questions. Upon validation, the user will be redirected to a login page where he would log in using a one-time use only password. The one-time only use password will be sent to the e-mail address on file for that user.

SR-5: An account profile change such as change of mailing address and contact information such as e-mail, phone number requires additional verification of the customer identity (e.g. customer is called by customer support representative to validate his or her identity, customer needs to call the bank to confirm the change request).

SR-6: Access to PII stored by the application requires the use of multifactor authentication such as security token and PIN.

SR-7: Access to high-risk transactions such as transfer of funds to an external account requires use of multifactor authentication such as security token and PIN.

SR-8: Authentication data should always kept encrypted in all systems and environments.

SR-9: Confidential data need to be encrypted in storage and in transit.

These functional security requirements can be documented in the technical design documentation for the online banking application. These security functional requirements are used to assert if are supported with the necessary technical details that should be in scope for the threat modeling exercise during the stage II of PASTA "Definition of the technical scope."

Besides functional security requirements driven by information security policies and standards, it is also important to include security requirements that satisfy regulatory and compliance obligations. Also, these regulatory and compliance obligations need to be in scope for the security functional requirements of the online banking application in scope. For example, in the case of online banking applications, there are several US and international standards and regulatory compliance that might be applicable for the security of online banking application, some of these include:

1. Information Security Policies and Standards (e.g. ISO 27011)
2. Federal Financial Institutions Examination Council's (OCC-FFIEC)

3. Gramm-Leach-Bliley Act of 1999 (GLBA)
4. Sarbanes–Oxley Act of 2002 (SOX)
5. Payment Card Industry Data Security Standard (PCI DSS)
6. Security Breach Notification Laws (SB 1386)

Some of these regulatory bodies and standards also rule specific security requirements that should be in scope for the application and can be added to the list of security requirements previously identified such as

SR-10: Access to high-risk transactions such as transfer of funds to an external account requires use of multifactor authentication such as security token and PIN (mandated by OCC-FFIEC).

SR-11: Credit and debit card holder data such as credit/debit card number should be protected by masked (e.g. show only last four digits) on display and protected with either encryption (e.g. hash) or tokenized while in transit and in storage. The card PINs and the authentication card codes such as CAV2/CVC2/CVV2/CID should never be stored by the application after authorization not even if encrypted (mandated by PCI-DSS).

Additional functional security requirements that should be defined for the application are security functional requirements that satisfy data privacy requirements. These security requirements depend on the various privacy laws in scope that are different by country and jurisdiction and concern the protection of the citizen's data that is considered private by the different data privacy laws in scope for that country and jurisdiction. For example, for what concerns US online banking customers, these are requirements for the protection of PII. An example of the requirements for applications that process and store PII consists of restricting the access to these data only to authenticated and authorized users and to provide disclosure to the customers affected when their PII data is either lost or compromised. In the case of the online banking web application in scope, these privacy security requirements apply to customers' private data such as customer's PII data that is processed and stored by the web application. This also applies to confidential PII data used to identify a customer when applying for a new bank account, when applying for new credit card, when getting a credit report from a credit bureau as well as for identifying a customer to recover or reset passwords and online user accounts.

Depending on the country and state where the private information of the customers is stored, there are different types of privacy laws that should be considered in effect. Specifically what is considered private data varies among countries, for example, under EU directive 95/46/article 2a: "*personal data is any information relating to an identified or identifiable person, an identification number or one or more factors specific to the physical, physiological, mental, economical cultural or social identity.*" In the United States of America, what is defined as PII is defined in personal data breach notification laws such as SB 1386 that include the combination of a name and a tax ID such as Social Security Number (SSN), a Driver License Number, and an Account/Credit/Debit card number and a PIN.

Depending on the applicable privacy and data breach notification laws, it is important to derive a security requirement for the protection of personal identifiable data of the customers as well as to provide necessary disclosures, notifications, and options when private information of the customer can be collected and shared with other parties. Another important reason for protecting PII is to protect it from the identity thefts and fraud such as when a fraudster will use stolen identities to impersonate a legitimate user and commit fraudulent transactions. An example of security requirement that can be derived for the application to satisfy the requirement of protection of PII is the following:

SR-12: PII such as Name and Last Name and a Tax ID such as Social Security Number should be encrypted when stored and transmitted and masked when displayed.

The list of security functional requirements SR1 to SR12 shows an example on security functional requirements that can be derived for the online banking application to be compliant with information security policies enforced by the organization to protect the confidential data of the customers as well as by the various regulatory compliance and privacy laws in scope for the protection of the customer's private information.

After the functional security requirements have been defined the next step consists of performing the activity "*DO 1.3 Define the business impact.*" This activity consists of defining what the business impact will be in the case of the application data assets such as confidential data and functionality is being either lost or compromised in security incidents.

Examples of security incidents that impact data assets include loss of availability of data and service such as in the case of accidental outrage or denial-of-service attack and loss of confidentiality and integrity of the data caused by data breach incident.

These examples as relate to the possible loss of confidentiality and integrity and availability of the data consist of either information security impact such as the loss of Confidentiality Integrity and Availability of the data or the associated technical impact caused by loss or degradation of the security control (e.g. encryption, authentication/authorization, digital signing, quality of service) that satisfies protection against that impact. From the business impact perspective, these information security and technical impact translate to business impact when these are associated with the consequences to the business in the case these data and functions are either loss or compromise. At high level, these impacts to the business are associated with the monetary/economical loss associated with the security incident as this security incident might be caused by a threat agent exploiting weaknesses and vulnerabilities in the online banking application to produce a business impact. The definition of the business impacts is often done based upon estimates of the value of the data assets and functions and the assessment of the quantitative risks of security incident events. These business impacts are specific for each type of threat, vulnerability exploits and attacks and the impact on the assets. In stage VII risk analysis and management will provide example on how this business impacts can be calculated.

At this preliminary stage of the risk-based threat modeling exercise, we will focus on high-level estimates of business impacts that can be captured in the risk profile of the online bank application to determine the level of financial business risk for the online bank application.

Business impacts include both tangible and nontangible losses. Tangible losses can be estimated based upon the possible loss of the monetary value of the assets.

Nontangible losses are difficult to estimate since they have an impact on the company brand and reputation and customer perception of the company value. Examples of the possible tangible and nontangible business impacts include the following:

1. *Tangible loss* such as the costs incurred by the organization because of data breach security incident. For each of the banking customers, this cost is estimated to be $355/record (a financial organization Ponemon Institute 2010 data).
2. *Tangible impacts to the business* because of loss and degradation of service caused by a DDoS (Distributed Denial-of-Service attack. The average annual cost of DDoS attacks for a business is estimated to be $187,506/year (based on 2011 Second Annual Cost of Cyber Crime Study Benchmark Study By Ponemon Institute).
3. *Tangible impact caused by online fraud* for example online fraud due to malware banking Trojan compromising a customer PC and successfully conducting a fraudulent transfer up to the max value amount (e.g. $100,000) allow for the personal or business account of the customer/client.
4. *Tangible impacts due to violation of EU data privacy laws* (e.g. up to 1 million EUROs or 2% of worldwide annual turnover whichever is greater under new rules June 2013 proposed by the European Commission).
5. *Tangible impacts due to regulatory fines* imposed by the payment credit card industry (VISA, MasterCard, AMEX, Discover) due to loss of credit card data from their customers (PCI DSS with fines up to $500,000 when credit cardholder data is either lost or stolen).
6. *Nontangible reputational damage* caused by inability of customer to access the remote functions that the online banking application provides such as open bank accounts, pay bills, apply for loans, book resources and services, transfer funds, trade stocks, view account information, download bank statements and others.
7. *Nontangible reputation* loss such as, in the case of publicly traded company, a drop in stock price as consequence of announced security breach.

These estimates of the business impacts are business specific and financial related and are based upon vendor surveys and statistical data of the impact of security incident affecting financial organizations. It is important to notice that this list of business impacts is not all included list of business impact since it might include also lawsuits, legal fees, and additional measures and controls imposed after the security incidents by regulatory bodies and other factors. More accurate estimates can be derived based upon internal sources including cost incurred to recover and technical repairs after

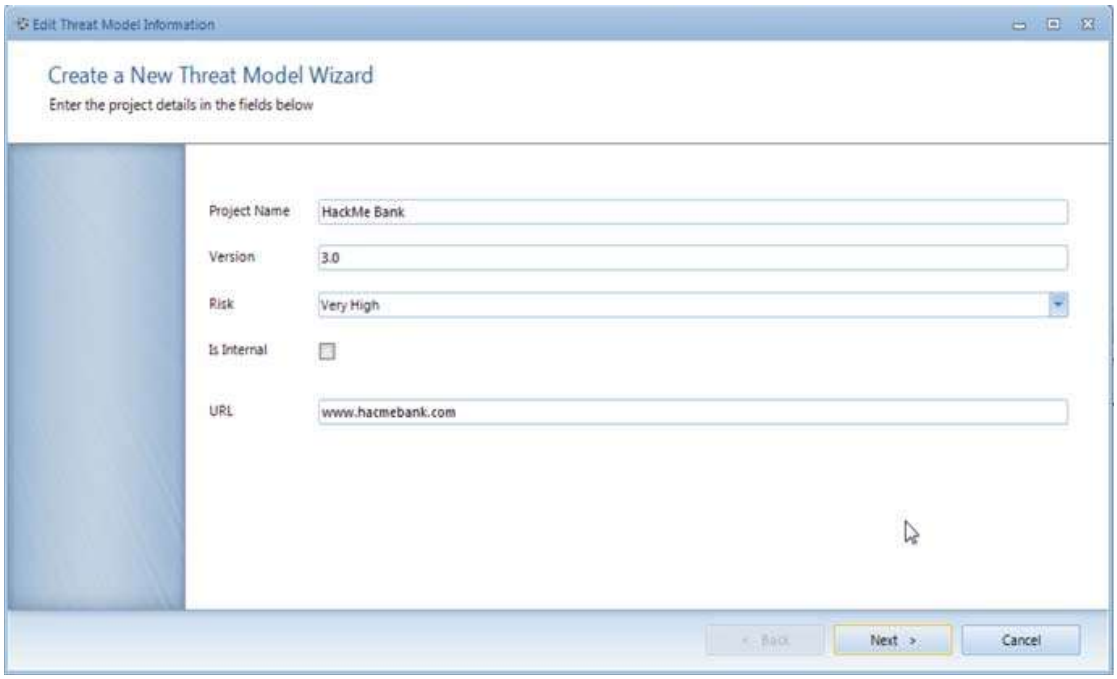


Figure 8.3 ThreatModeler™ Tool Wizard Capturing the Level of Risk for the Project HackMe Bank

incurring in data breach-related security incidents including incidents whose impact for the financial organization has been losses because of online fraud.

Because of the various business impacts calculated herein, the risk of the online banking application project "is considered very high and can be assigned to the new HackMe Bank" project using the ThreatModeler™ wizard screen shown in Figure 8.3.

After the business impacts have been defined, the next step of this stage consists of performing the activity "*DO 1.4 – Determine the risk profile for the application/product.*"

Before defining the risk profile for the online banking application, it is important to identify a set of high-level risk management objectives at organizational level. These are also risk management objectives that apply to the management of risks of the online application in scope and can be followed throughout the application threat modeling assessment process.

These are objectives for the identification and the analysis of threats against the online banking application and the determination of their severity, the analysis of how each of these threats might be realized to cause a negative impact on the application data and functional assets and the determination of the security measures that can be designed and deployed to reduce these risks to manageable levels.

From the perspective of technical risks and specifically for the risks caused by possible design flaws and vulnerabilities in the application, it is also important to articulate the possible impacts of security incidents whose risks need to be managed. This is the risk of security incidents targeting the online bank application and that might

expose sensitive data and functionality to different types of impacts such as impact of compromise of confidential data (e.g. information security risks), impact caused by noncompliance such as failing compliance audit with regulation (e.g. unlawful noncompliance, legal and regulatory risks) and business impact such as in the case of a security incident due to the realization of a threat in an attack that will cause a monetary loss of asset values (e.g. confidential data and business functionality).

The types of risk that an online banking application might be subjected to and is worth analyzing further might include the following:

1. *Information security/technical risks*
2. *Unlawful noncompliance* liabilities, legal and regulatory fines
3. *Violation of privacy* laws risks
4. *Business impacts* (e.g. business continuity risks)
5. *Security incident impacts* caused by specific threats

The management of these risks requires the organization to adopt information security policies and standards and have risk management processes in place. At high level, these are standards and processes that impact the organization/business as whole all web applications including the online banking application that is developed and managed by the organization.

This goal of the risk profile of the application is to provide a snapshot of the inherent risks that need to be managed in compliance with the organization risk management process. These risks depend on the inherent characteristics of the application/product in scope for the risk analysis. In the case of an online banking web application, for example, the risk profile can be derived from an IT asset repository that includes information to manage information security requirements, compliance, and risks of the online banking application. From information security perspective, the IT asset repository might include information about the application environment (e.g. Internet or intranet) the inherent risks based upon the classification of the data (e.g. confidential and private PII) and the volumes of such data (e.g. volume of records stored based upon number of users registered). An initial risk level for information security risks can be assigned based on an estimate of the possible levels of impacts due to the loss of confidentiality, integrity, availability of the data stored/processed by the online banking application. The level of impact can be estimated based upon the consideration of the classification of the data and the volumes of this data that can be possibly disclosed, tampered with, or made unavailable in the case of a security incident.

In the case of compliance risks, it is also possible to estimate the possible levels of impact in the case of a possible violation of security standards and regulations such as PCI-DSS in scope for the application whose impact can be estimated as additional costs to the business for fines and legal costs. In the case of privacy risks, the impact can be estimated as direct function of the number of registered users whose compromise of PII might have different levels of impacts and costs for the business.

From risk perspective of the business impacts, a possible estimate can be done as function of the economic levels of the impact of a security incident and the impact

of a loss or compromise of a business function such as data compromise due to malware/hacking and impact on the availability of the site when it is attacked by denial of service. Such level of impacts might be determined based upon the monetized amount of estimates of loss of data and service occurred because of the likelihood and impact of security incidents. Methodologies such as business impact analysis and quantitative risk analysis might help to quantify these estimates. These business impacts were previously estimated in the business impact analysis activity and included in the risk profile as estimate of the level of "Business Impact Risks."

Finally the initial risk profile of the application might also include the risks of cyber-threats as level of severity as probability of these cyber-threats to be realized in attacks such as malware/hacking and distributed denial-of-service attacks against the online banking application. The level of severity of these threats might be derived by the estimates provided by security vendors and cyber intelligence providers (Table 8.2).

Stage II Definition of the Technical Scope (DTS)

Goals of this Stage The goal of this stage is to enumerate the details of technical components including architectural and software components that scope for conducting the threat modeling exercise. The technical details should be included in the design and asserted against the security requirements that were previously identified. The technical details that are captured at this stage support the detailed analysis of the application architecture and components in stage III-Application Decomposition and provide the technical scope for assessing the exposure of the application assets (data and functionality) to threats in stage IV-Threat Analysis. The enumeration in the technical scope also seeks to identity all the components of the technical stack that includes the layers of software, processes, servers/systems, and network components that support the application functionality. The objective of this stage is to create a comprehensive list of technology assets that support the application in scope and made the various components of the application architecture.

Guidance for the Execution of This Stage The activities of stage II provide a list of the underlying technology stack of the application in scope such as the platform/system, relevant databases, application servers, network servers, infrastructure equipment, biometric hardware, and any other IT asset that will be used to fulfill the objectives defined under stage I. Essentially, the activities across this stage defines the various components (assets or sub-assets) that will be later dissected by the application threat model.

Any of these technologies could serve as either the immediate attack surface or target component within the threat model. The technology scope therefore helps form the environment in which future attacks will be taking place whose formal methodologies used for attack modeling such as attack kill-chain and attack trees will be representing, along with associated vulnerabilities, attacks, actors, and so on.

Stage II of PASTA (Technical Scope) attempts to answer the common problem or question "I didn't know that we were running/using that technology in our app."

TABLE 8.2 Online Banking Application Risk Profile

IT Asset Risk Profile: Online Banking Application	
General description of the application functionality	The online banking application allows customers to perform banking activities such as financial transactions over the internet. The type of transactions supported by the application includes bill payments, wires, funds transfers between customer's own accounts and other bank institutions, account balance-inquires, transaction inquires, bank statements, new bank accounts loan, and credit card applications. New online customers can register an online account using existing debit card, PIN, and account information. Customers authenticate to the application using username and password and different types of MultiFactor Authentication (MFA) and Risk-Based Authentication (RBA)
Operating environment	Internet (accessible over the Internet by banking customers an intranet only by administrators)
Compliance laws and regulations in scope	ISO 27001, ISO 22301, FFIEC, PCI-DSS, GLBA, SOX, SB 1386, EU Privacy-FTC Safe harbor rules
Sensitive data types	Confidential, Confidential PII, and Authentication Data
High risk transactions	Payments, wire transfers, online access to confidential information and PII
User roles	Visitor, registered user, administrator, customer support representative
Number of users	3 million registered customers
Information security risks	HIGH (High risk for potential loss of data confidentiality, integrity and availability)
Noncompliance risks	HIGH (High-risk impact in the case of violation of compliance regulations in scope)
Business impact risks	VERY HIGH (High risk of loss of revenue and online fraud in the case of denial of service and security incidents)
Privacy violations risks	HIGH (high risk of privacy violation for the volume of users registered)
Threat severity risk	ELEVATED (High risk of cyber-attacks as indicated by FS ISAC for the financial sector) (* this is dynamic value from monthly threat intelligence reports)

Stage II is essentially a large enumeration project. It focuses on itemizing software and hardware assets in order to later define a lean and relevant attack surface whose components could be exploited by various possible threats. Activities in this stage should produce a roster of technological components that are inventoried. This in itself is incredibly useful and can be leveraged over time as the technology scope of the application changes. The roster of application components needs to extend beyond traditional realms (i.e. – server, infrastructure appliance, endpoint, etc.).

The list should span to include layered pieces of technologies that may be embedded. The simple rule is to identify anything that has a direct exposure to a caller or

actor, whether it is from a human or a compiled library that only calls another component based upon an event or interpreted result. Essentially, the asset breakdown should extend beyond hardware and include software based technologies such as proxies, middleware, authentication servers, compiled libraries, open source libraries, third-party APIs, browser-based plug-ins, and so on. Today's feature-rich UIs, for example, need to be dissected properly in order to ensure that all of the client and presentation layer technologies are properly enumerated.

Later on, these assets will serve as the main "trunk" to the attack trees focusing on the various assets that can be attacked to be developed in PASTA's stage VI. The tree will ultimately show underlying use cases, data components, and actors interact with each other. From each one of these nodes, further branches can be split out to show unique threats for each underlying node stemming from the initial asset component.

Scoping is important for this exercise as too broad of a scope may confuse the subsequent steps. Too broad of a scope inhibits good analysis. For this reason, a tighter scope of assets is suggested in order that a simpler and more efficient threat model can be created. *T*his stage objective is to capture the technical details of the application/product so it possible to later identify the risks and the technical impacts. The technical details need to be captured and documented including the data assets and architectural components. For the definition of the technical scope, it is important to be able to collect these technical details in design documentation and captured in standard technical design documentation.

A comprehensive documentation should include the architecture design documents inclusive of the architecture components, servers and network infrastructure dependencies, the details of security controls such as authentication, authorizations, data protection in storage and in transit, data filtering, session management as well as details of the design patterns used and the architecture components, data interfaces and the trust boundaries of the application with other application software and software with server components, third-party software libraries and frameworks, servers O.S. and the supporting data center, network infrastructure in support of the application and product in scope.

Since threat modeling focuses on identifying how threats are realized in attacks against the various technical components of the application/product as well as the various components of the technology stack, it is important that this technical information is completed and documented with all the necessary details. These are the details necessary for conducting the threat modeling exercise.

At the project level, it is important to capture the technical details of the design and of the new features/changes introduced in the application/ products so that it is possible to assert the technical impact of these changes. These design changes are ideally documented in the technical documentation that is used for the design and implementation of the application. As the technical details are captured through interviews, questionnaires, and other methods (e.g. analysis of network topology, reverse engineering of code to determine libraries used etc.), they also need to be documented. The final activity consists of validating that the technical documentation is complete by asserting the security requirements and by validating if any technical details are missing and need to be documented in the technical design of the product/application.

Inputs for Conducting This Stage To conduct this stage, it is important to gather information about the technology stack used by the application at the various technology layers including software libraries, API used by the application software, software framework that support the execution environment (e.g. J2EE, dot NET), type of client technologies (e.g. Web 2.0 components, browsers, thick clients) servers (e.g. web servers, application servers, databases) and O.S. (Linux, MS Windows, UNIX) including the various network infrastructure components. The information of the technology stack should be made available based upon available technical documentation and assessable when stored in IT asset repository. In case this information is not readily available, it can be gathered by conducting a series of interviews with the application stakeholders such as application managers and technical SMEs. Some of the information of the technology stack can also be gathered using network scanning and process profiling tools to identify processes and systems running on the application in scope.

Example of documentation that can be reviewed to enumerate the various components of the application architecture and the technology stack includes the following:

(a) Security requirements from stage I
(b) Documentation of the technology stack used by the application components at the different tiers of the architecture
(c) Architecture design documents
(d) Architectural diagrams of the application
(e) Functional and technical specifications
(f) Network diagrams

Artifacts Produced at the End of This Stage At the end of this stage, we will produce details of the technical stack that compromises layers of components and services that are used by the application including the network infrastructure layer and the third-party infrastructure layers/components. The technology stack is also organized by the various architectural tiers of the web application in scope that includes the presentation tier, the business logic tier, and the data tier. The details of the technology stack need to be documented in the technical design documentation and specifications along to the functional security requirements defined in stage I.

Tools The execution of this stage can be facilitated by the use of IT asset repository with up-to-date information on technology stack used by the application as well as architecture design and project level design documentation such as technical specifications and functional level design. A project management workflow tool also helps in enforcing toll gates when this step is performed during the SDLC to ensure that technical documentation is complete with information and security requirements that can be asserted for architectural risks introduced in the design changes during stage III.

Activities This stage consists of the following activities:

DT2 2.1 – Enumerate software components: The goal of this activity is to enumerate software components used by the application. This information can be collected from the application stakeholders conducting targeted interviews and by using ad-hoc questionnaires. Example of the technical information that is relevant for the analysis and should be collected includes the various software components of the technology stack used in the application architecture such as the programming languages, custom libraries, and third-party software and services. Some organizations that have adopted a repository for manage technical assets including web applications might have this information already stored in the IT asset repository/catalog that include all software components, servers, and network components used by the application and in scope for design changes as well as operation/maintenance;

DT2 2.2 – Identify actors and data sinks/source: The goal of this activity is to identify the various actors (human or application based) that include users and user roles as well as process running under specific functional accounts. These actors can access resources such as files and data based upon permissions set for these resources and mapped to each user/agent or component/agent. The goal of this activity is to determine who the application actors are and the roles and the type of permissions of these actors to access resources including the various components of the application architecture at various tiers as well as layers of the technology stack. The identification of the actors such as users and roles can also be assessed by reviewing access control lists assigned to existing resources such as data and files and by using tools that allow retrieve information on roles and permissions for users to access data and files accessed by software components.

Configuration files can also be reviewed to identify users and roles. Other techniques to identify users/actors include the audit of databases that store user credentials and architectural components whose function is to manage users and roles such as LDAP-based directory services (e.g. Active Directory, SiteMinder Single Sign On LDAP Servers). The validation might go as far as collecting data from traffic to/from the authentication components and by analyzing how credentials are used by clients and hosts to identify any preshared keys, digital certificates used for authentication engine and by running authentication calls through a proxy to identify authentication calls have been missed in enumeration efforts. Similar to the other activities around enumerating software, systems, and third-party software, this activity is focused on identifying smaller components that actually run or operate within the previously listed exercises. Within these two exercises, we will identify all database, system, and application actors that are making/receiving requests on each asset or across the application environment.

Besides user actors, processes that run at the application level to perform various tasks such as connecting to databases opening files also need to be enumerated

when these are associated with system accounts and processes running on a system. These are processes that can be enumerated by process IDs (PIDs) and can be mapped to user accounts. This enumeration helps to identify some of the inherent actors on a given system or third-party product. Tools such as SysInternals by Microsoft can be used to conduct this assessment at system/platform level for either a physical or virtual MS Windows host. Beyond using tool-based techniques to extrapolate the actors and services of the product application environment, application architects and members of the development team should be able to create an initial list of actors and required services that sustain the application use contained within the environment. Developers, for example, can perform CRUD exercises to determine what actors will be used to Create, Read, Update, or Delete data from the data repository, whether it is a backend database or a flat file system.

DTS 2.3 – Enumerate system-level services: At system level, processes can run as services. The goal of this activity is to discover what system types or operating systems are being used at the various tiers of the application architecture such as presentation tier, business logical tier, and database tier. Examples of the system services that run in the presentation tier include various processes in support of the interaction of the user with the application and include processes for entering user data such as commands for the application, upload files as well as for visualize and use the data such as running data queries, search and download files, videos and other type of data. At the business logic layer, system-process might run on the application server and use middleware to communicate with other server processes that run the servers (e.g. backend servers) where the data is stored. System-level service enumeration is aimed at identifying what services are running on the various system assets on which much of the application software will be running. Both of these exercises will require the use of some system commands and/or simple tools that facilitate identifying both services and actors in association with various use cases of the product application. A port scanner, for example, would be able to do both the platform identification and the service enumeration for a given scope. If, for example, my test product application runs on a 192.168.52.0/28 network, I can scan that and see what the scope of platforms and services are. Note that system platforms encompass application servers and/or databases within each architectural tier. They also reveal possible third-party products or autonomous application environments (e.g. cloud services, SaaS services) that may support the overall application solution. Third-party products in general are difficult to include in the threat model since they are largely black boxes for which an underlying understanding of technology is absent. Third-party cloud environments are even more complex since the ability to scan or interface with any facet of the cloud infrastructure is not commonly permitted by the cloud or hosted operator. Vendor products and tools, although proprietary in nature, at least present the opportunity for security practitioners to scan and interface with the host in order to ascertain some level of security assurance. Both third-party services

and appliances should be listed under the correct container for activities under this step.

Enumeration of system-level services can be done by using different tools that are O.S. specific and therefore limited in their capabilities. If there is any diversity of operating systems on the network, then systems administrators will end up with a diversity of report types. Some security scanners (e.g. Tenable's Nessus or Rapid7's NeXpose) today have the system-level scanning capabilities and avoid unnecessary use of time and resources to conduct this assessment.

At the end of this activity, we will produce formalized descriptions of installed packages in a heterogeneous network with much less time and effort than using the aforementioned options. We will also provide functionality well beyond the scope of package management including the ability to identify services open on a given platform as well as the ability to fingerprint the platform OS and any embedded OS that may be present. All of this information helps the threat modeling process by identifying a blueprint of assets with possible inherent vulnerabilities. Such data will be further leveraged in stages IV allowing the mapping of threats to the system-level services enumerated and the various system assets that operate within the different tiers if the architecture such as presentation tier, business-layer tier, and data tier as well as the various trust boundaries of the network architecture such as client, DMZ layer, and internal network layers of traditional three tier web-based application architectures.

Typically architectural tiers represent an abstraction for the network tier, for example, for web applications, the presentation tier is mostly allocated on the client host and the services running on that host to support user interface functionality (e.g. thin-client browsers and thick clients) and the web server providing the web interface as a service (e.g. web service). The identification of client-side technologies that are developed by the product/software development team is also to the enumeration of the various host-level processes that run on the client as well as the web server and this includes also the system level services for presentation layer functionality that includes JavaScript, AJAX, and Silverlight development. At the presentation layer, some degree of input validation is done at the client/browser as well as at the web server (e.g. by filtering request and response using NSAPI filters as example).

A secure architecture typically involves the running of services that can enforce security functions such as authentication, authorization, data validation at the application layer. This includes input validations of requests and responses for data exchanges that are both downstream and upstream. Business logic is also provided by at the application servers that aggregate functionality from data sources. Lastly, a data layer represents where the greatest level of protection should be applied to the assets, given the presumed data centralization of key business and/or client data.

DTS 2.4 – Enumerate third-party infrastructure components: Many application environments use various external networks. Some may be in the form of cloud-based services such as Process as a Service (PaaS), Software as a

Service (SaaS), or Infrastructure as a Service (IaaS), while others will be along the lines of a more traditional third-party ASP or colocation model. Whatever the service, it is important to bring third-party infrastructures into the technical scope. Depending on the relationship and the legal terms around "right to audit," certain automated solutions could be used similar to the activities for a captive or self-hosted model. However, many third-party vendors do not like their environments to be scanned without proper permission, so it is important to ensure that any discovery scanning efforts against a multitenant or shared hosting environment is cleared beforehand. For this reason, realistic actions to perform under this activity include interviews with third-party technology SMEs in order to identify what software, data, platforms, and system services are to support the application, reviews with any existing network design documentation from the third- party and evidence in support of the reviews collected using tools that allow to enumerate software used, data repositories (flat files, relational DB), fingerprint platforms, and ports/services used (may depend on legal agreements with the third party). Generally speaking, it is important to rely on vendor-technical points of contacts to get the necessary information. Today, most account management teams themselves will have the information or know of the internal SME who can provide requested asset/component information pertaining to your hosted solution.

DTS 2.5 – Assert completeness of secure technical design: Once we have identified the various application actors, enumerated all components used at the different layers of the application architecture and the technology stack including software, we would like to review the technical documentation to validate that this information is documented in the technical design is complete and accurate and includes all the necessary details to conduct the application threat modeling exercise.

To make sure that document details are documented including all enumerated components of the technology stack, development teams can be given technical documentation templates for designing the application architecture that include all the design details necessary for the secure design of the application (e.g. security requirements) as well as the details for the application threat modeling exercise. The key items to document are security requirements for the design of security controls for the application/asset in scope. To determine which security controls should be documented and how as well as the security requirements that need to be followed, it is important to rely upon application security standards and guidelines and document templates that include the mandatory sections of the document that need to be documented for the initial technical scope assessment from the information security team. These document templates can also include pointed guidance of which architecture diagrams should be documented and how and include an information security section with mandatory documentation of security controls (e.g. authentication, authorization, data validation, encryption, session management, audit and logging). It should be responsibility of the application design architects to document all the mandatory sections of the architecture design and functional

design document. Each design document should include a set of Information Security (IS) requirements as mandated in the template document. The type of classification of the data and the risk levels assigned to this data in the risk profile should be used to define IS requirements for protecting these assets (e.g. confidential data should be encrypted). The technical documentation should provide enough details for identifying and analyzing the presence of gaps in security controls and architectural design flaws. The assertion of the completeness of design needs to include all software components, actors (human and not human), system-level services, third-party infrastructures that were enumerated and captured in the previous activities of this stage of the threat modeling process. The definition of what details should be in scope for technical design also depends on the business and security requirements previously defined and the type of assets such as data and functions that need to be protected for compliance and risk management perspectives. The technical scope assessment of a web application, for example, requires the documentation of logical and network architecture as well as of the application functionality that is exposed through the various channels, proprietary or third-party channels. The enumeration of the human actors drives the enforcement of the user's roles and permissions and the security requirement for the design of authorization controls such as role-based access controls.

Web Application Use Case Example The first activity of this stage consists of the activity *DTS 2.1 – "Enumerate software components."* Software components used by the application can be classified based upon different types of information such as the type of languages used by the application software such as J2EE, .NET, C/C++, PHP, JavaScript, Visual Basic, Python, Shell, Ruby, Objective-C, and C#, the customized and proprietary libraries developed for the various components of the application architecture tiers such as presentation layer (e.g. graphical APIs/libraries), business tier (e.g. business components API/libraries), and data tier (e.g. data access components libraries). These libraries can be developed using standard technologies and APIs such as Web HTTP messaging protocols, XML, and Web 2.0 APIs software components that use AJAX, JSON, and Web 2.0 service protocols such as SOAP and REST used in SOA Service Oriented Architectures. Since most of applications today are developed using third-party libraries and components that are deployed at the different layers and tiers of the application architecture. It is critical to enumerate them as well; these include libraries such as Web APIs that include Apache Struts, Spring, Log4j/NET.

An example of the software (S/W) technology stack of the online banking application in scope is included in Table 8.3.

The information of the application components s/w technology stack can be collected from the application stakeholders conducting targeted interviews and by using ad-hoc questionnaires. Some organizations that have adopted a (IT) Information Technology asset repository for managing technical assets such as IT systems deployed and maintained by the organization including the ones used by the web applications might have this data already stored in the IT asset repository/catalog

TABLE 8.3 Online Banking Application Components S/W Technology Stack

Online Banking Application Technology Stack: Software Components and Libraries	
Programming languages and software development environments	Java/J2EE, JavaScript
Standard software protocols	HTML / HTML5, SOAP, REST, XML, CSS / CSS3, WS Security, SAML
Third-party software frameworks	Java/JDK, JSP Struts, JavaEE (Servlets), JavaScript, Framework, RIA, AJAX, JSON Spring Sprint, Log4J
Custom software libraries and APIs*	Java Banking Platform (JBP)Secure Filtering APIs (SFA)Financial Web Services (FWS)
Third-party software libraries and APIs	MS Active X, Java Applet, Adobe FLASH, JDBC Oracle, TIBCO EMS APIs, SiteMinder SDK

*Note the custom S/W libraries developed for the online bank application are fictional and not used in any specific type of commercial banking application and provided as an example of proprietary custom developed libraries/software.

that include all software components, servers, and network components used by the application and in scope for design changes as well as operation/maintenance.

The next stage activity to follow is *DTS 2.2 – "Identity Actors and Data Interfaces."* The goal of this activity is to determine who the application actors are and the roles and the type of permissions of these actors to access resources including the various components of the application architecture at various tiers as well as layers of the technology stack. The identification of the actors such as users and roles can also be assessed by reviewing access control lists assigned to resources such as data and files and by reviewing how these data and files are accessed by software components.

The definition of the users and roles of each application is typically defined in the RBAC Role base Access Control matrix that defines who the users of the application are, their roles and their permissions to access the various resources (e.g. URLs, functions, data, files) that are stored and protected by the security control of the application. Often these user roles and permissions are defined in application configuration files, these files can also review to validate whether the user role-permission configuration implemented for the application 9e.g. stored in XML configuration policy file) corresponds to the roles and permissions for the users that are documented in technical design.

The definition of the user roles is critical for the application threat modeling exercise since from these roles depend the enforcement of authentication and authorization security controls.

An example of various users of the online banking application that can be defined using ThreatModeler™ and later one used to assign use cases is shown in Figure 8.4.

Besides user actors, processes that run at the application level to perform various tasks such as connecting to databases opening files and so on also need to be enumerated when these are associated with system accounts and processes running on

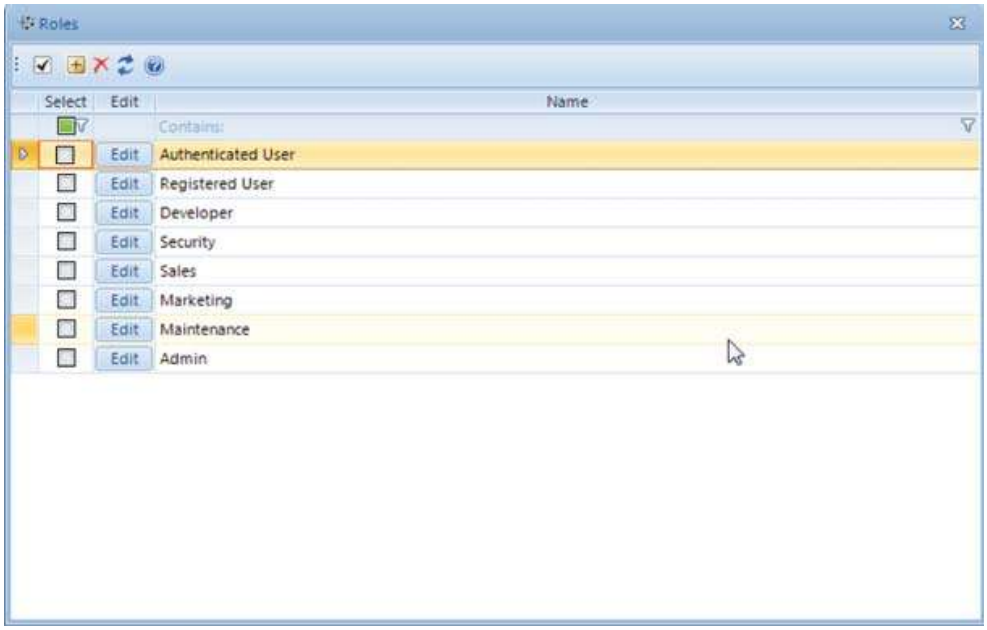


Figure 8.4 HackMe Bank Users

a system. These are processes that can be enumerated by PIDs and can be mapped either to user accounts or to functional/system accounts.

Once the various users have been enumerated defined the next step consists of capture the various data interfaces components that provide connections between the various components of the application architecture. At this stage of the enumeration exercise, it is also important to capture the type of authentication used to authenticate users as well as each connection and the protocol used. An example of this data interface enumeration is provided in Table 8.4.

TABLE 8.4 Online Banking Web Application: Data Interfaces

Data Interface ID	Data Interface Components	User and Server To Server Authentication	Protocol
1	Browser To/From Web Server	UID, Pwd, 2FA	HTTP/SSL
2	Web Server To/From Application Server	Mutual SSL Authentication	HTTP?SSL
3	Application Server To/From Database Server	AppID Pwd	JDBC/SSL
4	Application Sever To/From Messaging Bus Server	AppID Pwd	XML/JMS
5	Application Sever To/From Fraud Detection Web Service	Mutual SSL Authentication	XML/HTTP/SSL
6	Application Server To/From Authentication Storage-Database	AppID Pwd	JDBC/SSL
7	Messaging Bus To/From Mainframe	AppID Pwd	XML/JMS

After we have captured the data interfaces, the next step consists of performing the activity "*DTS 2.3 – Enumerate system-level service*." The goal of this activity is to discover what system types of servers, services, and operating systems are being used in the various tiers of the application architecture such as presentation tier, business logical tier, and database tier. These are tiers that have specific functions. An example of how the various application architectural tiers operate in an online banking application to run a query for bank account activity and present back the process data of account balance that is stored in the database is shown in Figure 8.5.

System-level service enumeration is aimed at identifying what services are running on the various system assets on which much of the application software will be running. Each tier of the application architecture is supported by different types of servers and services specifically:

1. *The presentation tier* is supported by PC clients such as different types of web browsers (Google Chrome, Mozilla Firefox, Microsoft Internet Explorer Safari) and stand-alone user clients-consoles and O.S., (such as Windows,

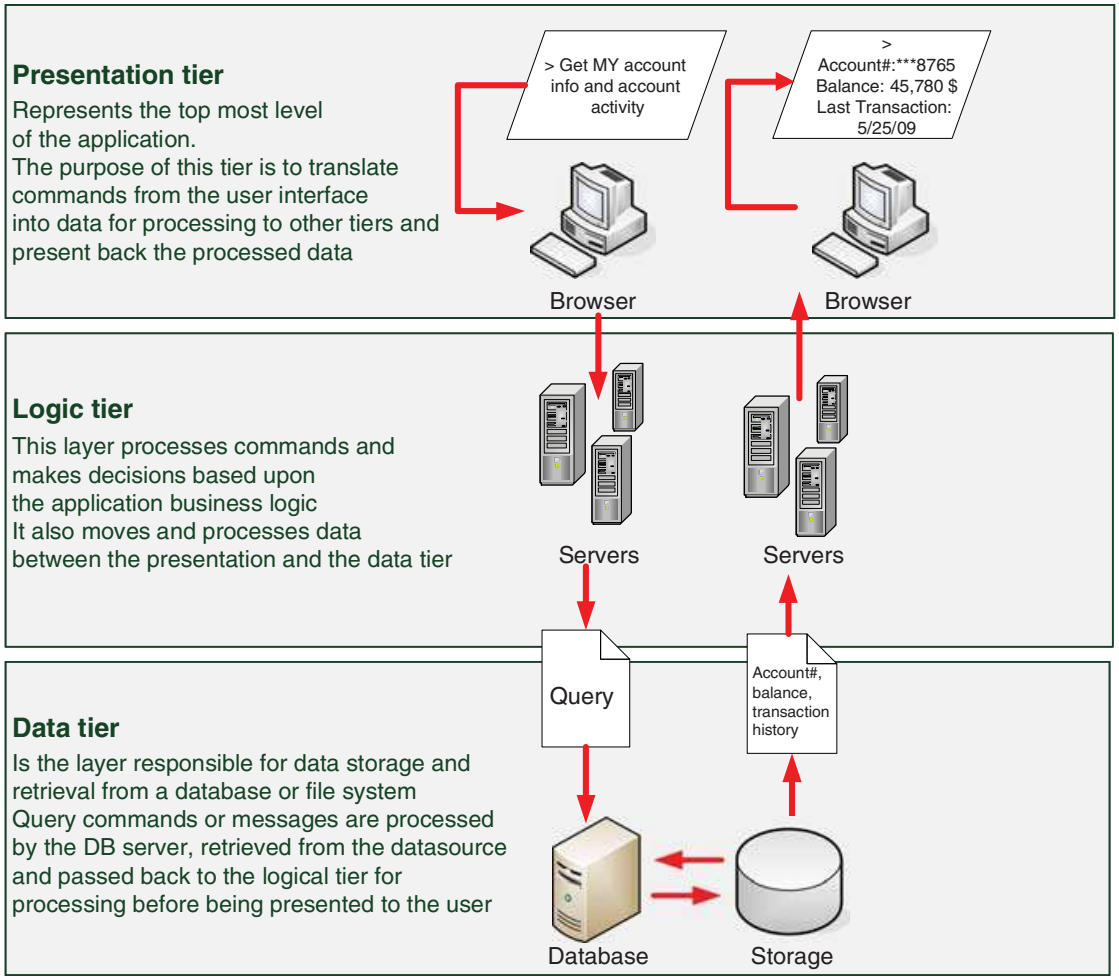


Figure 8.5 Representation of a Bank Account Query Transaction Through the Different Tiers of an Online Banking Application

UNIX, Apple O.S) running on desktops and laptops as well as O.S running on mobile devices (e.g. iPhone IOS, Android).

2. *The business logic tier* is supported by servers such as web servers (e.g. IIS, Apache tomcat) that can deliver web content that can be accessed online and the type of web application servers (e.g. Windows Server, Oracle Java Application Server, IBM WebSphere) that can execute the application business logic and access the various business services and APIs that provide these business services.
3. *The data tier* that is supported by various database and database servers (e.g. SQL Server, Oracle).

Web application architectures include both internal services (e.g. secure file transfers between servers via SFTP Secure File transfer Protocol and managed secure file transfers between mainframes and external services such as web services provided by another business partner that are not hosted in the internal network. The data collected and stored in the application services technology repository including the various types of services deployed with the application architectural components is shown in Figure 8.6.

It is important that each service component is associated with application components as containers of user functionality. An example of associating client site service components with user client presentation tier type of functionality or "widgets" is shown in Figure 8.7.

The next activity is "*DTS 2.4-Enurate Third-Party Infrastructure.*" Many application environments use various external networks. Some may be in the form of Cloud-based services such as PaaS, SaaS, or IaaS while others will be along the lines of a more traditional third-party ASP or colocation model. Whatever the service, it is important to bring third-party infrastructures into the technical scope. Generally speaking, it is important to rely on vendor-technical points of contacts to get the necessary information to enumerate the various architectural components. Often times, this information might be considered company proprietary and is not shared with the business partner or client. In this case, it is important to ask the third party to be in scope for a third-party security assessment that will assess the third-party infrastructure components including the security of type of services used by the application. In the case of integration with third-party cloud services, the enumeration of these cloud services can be done using cloud control matrix spreadsheets such as the one documented by the Cloud Security Alliance (CSA) as well as to provide evidence of compliance of industry standards such as SAS 70, SOC, FISMA, PCI DSS, ISO, FIPS-140, ISO/IEC 27001-2005, and others.

After the technology stack of all the application components that are in scope for the application have been enumerated and collected, the next step consists of documenting them in technical documentation. The goal of the activity "*DTS 2.5 – Assert completeness of secure technical design*" is to review the technical design of the application to validate that the details of the application technology stack previously captured/enumerated as well as the security requirements defined in STAGE I are documented in the technical design of the application. The documentation of these

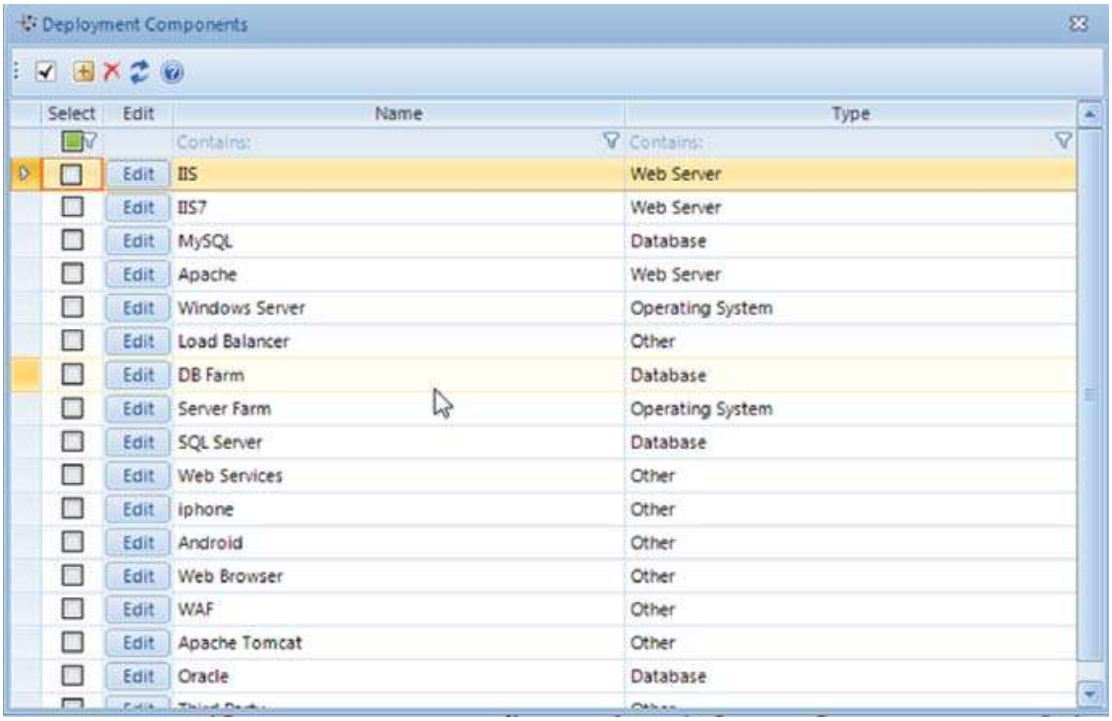


Figure 8.6 Internal Services Deployed with the Application Architectural Components

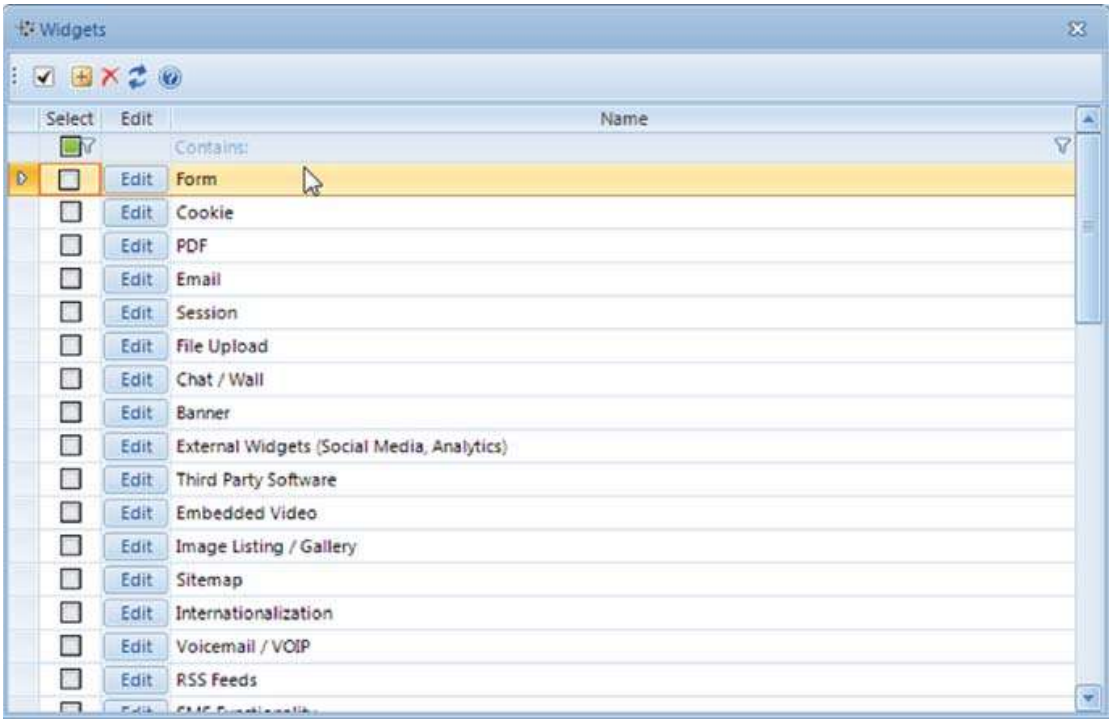


Figure 8.7 ThreatModeler™ Association of Widgets with Client Components

technical details is not only required to make sure that the application in scope will be implemented by following specific functional security requirements but also to make sure that all the technical details that need to be in scope for the application threat modeling exercise are documented and complete.

In the case of the online banking application, for example, the technical details of the application design can be documented in architecture design documents. These architecture design documents should conform to standard document templates and include all the details of the technical stack previously enumerated as well as technical details of the application architecture that can be later reviewed in the weaknesses and vulnerability stage to identify any design flaws in the design of security controls of the application (e.g. authentication, authorization, data protection in storage and transit, session management, data input and output filtering/sanitization, audit and logging, error and exception handling etc).

At high level, the technical design documentation should at a minimum include the following type of technical information:

1. Description of the users of the application and their roles and permissions as it can be documented in a role access control matrix.
2. Description of the application functionality from user perspective (e.g. use cases).
3. The functional security requirement section with description of the design of security controls.
4. Diagram of the logical architecture of the application showing the main architectural components, data interfaces, and communication protocols.
5. The technology stack (type of servers, O.S., software frameworks) from the IT asset repository.
6. The network topology of the application showing the physical location of the various network components including third-party components.

From the perspective of assessing the risk of targeted attacks, at the architectural design level, the documentation should provide enough details of the architecture to determine the level of exposure to external and internal threat agents. For example, all internal and external user and data interfaces to and from the web application in scope need to be properly documented, including the type of data flows (inbound, outbound), the type of authentication of these interfaces (e.g. server to server connections), and the type of protection of the data in transit to/from these interfaces.

For the online banking application, for example, this should include technical design information that shows the type of user interfaces (e.g. web and mobile), the type of servers and protocols and the different trust boundaries of the application architecture. This information can captured and depicted in an architecture design diagram of the online banking application as shown in Figure 8.8. The design documents of the application should include details about the user's roles-permissions, databases where confidential data is stored and the type of

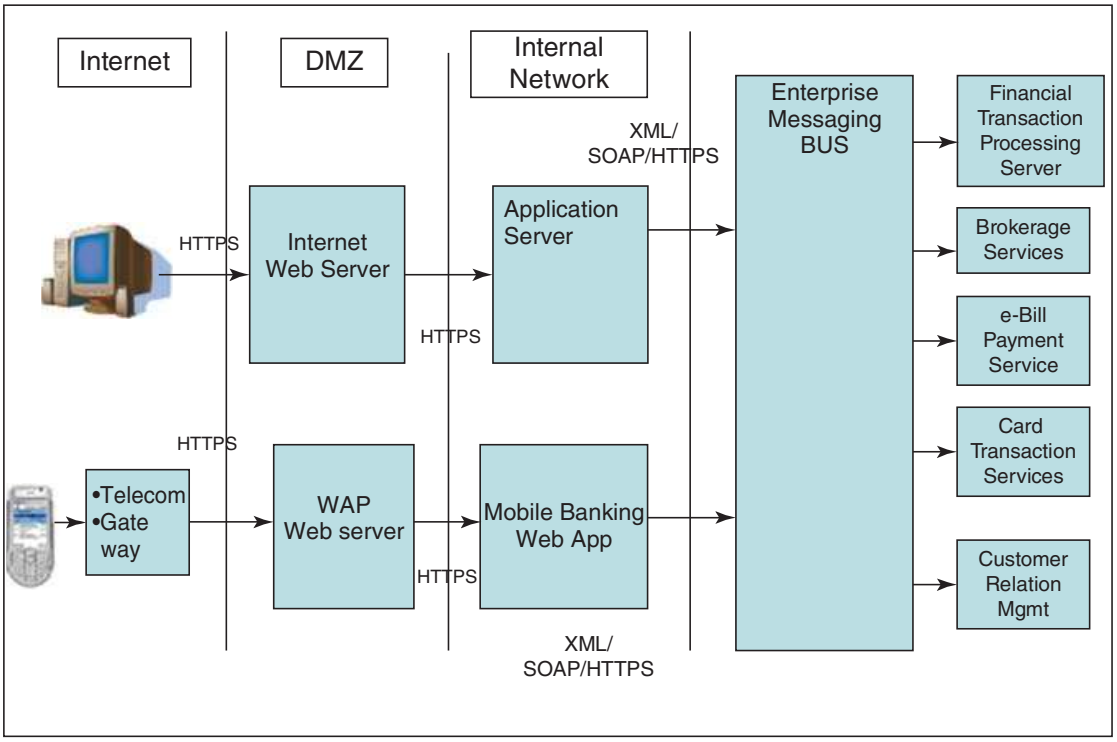


Figure 8.8 Architecture of Online Banking Application

protocols used for transmitting data as well information of the security controls such as user authentication, enforcement of authorizations to access data and functions, type of encryption used in storage and in transit, the type of session management used how is implemented, the type of data filtering APIs used, and the type of audit and logging controls that are implemented to monitor and detect suspicious activities.

At high level, the technical details of what should be documented need to validate that all the data assets at risk are documented including the confidential data and PII data in storage and transit and the security controls to secure access and ensure the confidentiality of such data such as simple and dual authentication factors used. The various types of user functions and user interactions with architectural components can be documented in use cases and sequence diagrams. The technical documentation should provide detail information that described how the various security controls of the application are designed and the security requirements and technology standards used.

After the technical design documentation has been reviewed to assert that the design documentation is complete, a preliminary security review can be conducted to validate compliance of the design with the security requirements that were previously documented in stage I.

A complete architecture design document for the application should include technical details describing how the various security controls should be designed in compliance with technical standards in the case of user authentication, for example, the design should conform to the standard approved design for user authentication

and multifactor authentication for online banking applications. The requirement to design multifactor authentication for the online banking application for access of confidential data and for access to high-risk transactions such as payments and transfer of funds is mandated by the previously defined security requirements, "*SR-6: Access to personal identifiable information stored by the application requires the use of multifactor authentication such as security token and PIN*" and "*SR-7: Access to high risk transactions such transfer of funds to an external account requires use of multifactor authentication such as security token and PIN (mandated by OCC-FFIEC).*"

The validation that the design includes a set of design requirements for implementation of multifactor authentication for specific functions can be done by asserting that these requirements are included in the design both at the architectural level and the functional level. The high-level architecture design documentation, for example, should include the technical details of which type of MFA is implemented (e.g. challenge questions, one time passwords/tokens, out of band authentication, certificate-based authentication).

The assertion that the design of the online banking application enforces encryption for data in storage and transit as defined in the security requirements can be done based upon the type of data that is processed and stored by the online banking application. For example, for authentication data, the design of encryption can be reviewed in the design to satisfy the security requirement *SR-1: Passwords should be encrypted using irreversible encryption (e.g. hashes) and using standard algorithms and minimum key lengths* and *SR-8: Authentication data should always be kept encrypted in all systems and environments.*

In the case of credit card data, the design should include specific design requirements for protection of credit card data in storage, transit, and when displayed such as *SR-10: Credit and debit card holder data such as credit/debit card number should be protected by masked (e.g. show only last four digits) on display and protected with either encryption (e.g. hash) or tokenized while in transit and in storage*.

The technical details of the type of controls used for protecting credit card (e.g. masking the card data, rendering the card data unreadable using either encryption or by replacing it with tokens) need to be in scope for the specific functional-technical design. Besides asserting that encryption is implemented by design to protect the confidentiality and integrity of the data, the type of encryption algorithms, key lengths, and their implementation including the protection for encryption keys can also be assessed by the review of the design to make sure that standard encryption algorithms, minimum key lengths, and standard secure key storages are also mandated in the security requirements section of the design documents.

Once the technical scope has been defined and the application security requirements have been asserted to be accurate and complete in the technical documentation of the application architecture and technical specifications, we are ready to conduct the architectural risk analysis and the next stages of the application threat modeling exercise.

Stage III – Application Decomposition and Analysis (ADA)

Goals of This Stage Decomposition of the application in basic components whose security controls and interaction with other components can be analyzed. Examples of components that can be used to describe the security of the application architecture include users, roles, data storages, data flows, functions, security controls, and trust boundaries.

Guidance for Performing This Stage The purpose of the application decomposition is to decompose the application in simple components so that each one can be analyzed for his exposure to threat agents/actors and for specific design flaws and vulnerabilities that these threat agent/actors might seek to exploit. These basic application components consists of the users of the application, their roles, the data assets in storage and transit, the application use cases and the type of functionality that can be assessed by these users, the security controls to protect sensitive data and functions, the analysis of the data flows including entry and the exit points for the data, the trust boundaries to access data and to process the data. Once the application is dissected in the various application components using architecture analysis techniques such as Data Flow Diagrams (DFDs), architectural design reviews, and whiteboard exercises, it will be possible to identify design flaws at component level and later analyze the impact caused by threats exploiting them.

By decomposing the application in architectural components, it is also possible to assert that security requirements for protecting the confidentiality, integrity, and availability of data are enforced by design for each application component. This analysis can be conducted by analyzing the functional design of each component of the application such as the user interfaces and user's functions, the web servers, the application servers, the databases and the backend services as well as the data interfaces, the data flows and the security controls in place to protect the data in transit and in storage in and between components.

Activities This stage consists of the execution of the following activities:

ADA 3.1 – Enumerate all application use cases (e.g. login, account update, delete users, etc) that describe an application business function and how the user interacts with the application to accomplish a specific functionality that the application provides. The use cases can be derived based upon the capture of the business requirements defined in stage I and users and roles of the application defined in stage II. The description/enumeration of the use cases need to include a description of the criticality of the business function being performed that includes the possible impact to the business caused by the impact of possible loss of accountability, availability, confidentiality, integrity of the data and the resulting impact such as economic loss resulting from loss of data including unlawful noncompliance, data privacy violations, and reputational damage.

ADA 3.2 – Document Data Flow Diagrams (DFDs) of the identified architectural components. By creating a high-level DFD of the application, it will be possible

to visualize the information flow end to end (ingress/ egress traffic at all layers of the application) and to describe how the data interacts end to end, from the external entities (client, user) to the data store through the processing elements of the application (components, servers). Depending on the detail used (e.g. DFD level 1 or DFD level 2), DFD can be used to describe data flow interactions among architectural components and among different functions of the application functionality. The goal of using high level 1 or 2 DFDs that allow to analyzing architectural level data flows by visually describing how actors (e.g. either users or processes) previously identified interact with the various architectural components enumerated in stage II at different tiers of the application architecture (e.g. presentation tier, business tier and data tier). These components and tiers can be represented within different trust boundaries (e.g. client trust, DMZ trust and internal trust zone), and the various server components that have been enumerated in stage II can be located within these trust boundaries. Example includes locating the external users and client in the external nontrusted zone, web server in the DMZ and application servers, middleware servers and backend servers/databases in the internal trust boundary zone.

ADA 3.3 – Security functional analysis and the use of trust boundaries. The goal of this activity is to analyze the security of the use cases previously identified and how the various data assets identified as components in the DFD are protected by the security controls from functional perspective. The goal is to decompose functionality of the application/system in users, data processed, and security controls that protect that data. Functionality is per se an asset as data that can be protected based upon the risk of the functionality/transaction. This activity is also referred as secure transactional analysis and consists of the analysis of application functions to assert the presence of security controls that protect the confidentiality, integrity, availability, and accountability of these functions. Examples of controls that protect functionality are authentication and authorization, encryption, input and output validation, session management, and audit and logging. At functional level, each one of the application asset and new functionality exposed by the application to a user need to be analyzed from the security perspective to determine whether protective and detective security controls are included in the design of the application. If a gap in a security control is identified, this will constitute a security finding and should be reported and considered for potential exposure. The remediation of this finding needs to consider the effectiveness of the control to mitigate threats and the additional countermeasures that should be put in place to reduce the risk-impact of such threat.

It is important to note that threat agents can exploit different type of vulnerabilities and weaknesses in the application. Examples of vulnerabilities include what constitute either a design flaw or vulnerability in the implementation and/or configuration of the application. Some of the threat agents might also abuse the business logic of which the application is built upon to cause a negative impact. From a threat analysis

and modeling perspective, a possible way to identify business logic flaws is to analyze all uses and abused of the application functionality using use and abuse cases.

Input for Conducting This Stage This includes different types of documentation such as

1. Business and security requirements (stage I)
2. Technical scope (stage II)
3. Technology stack (stage II)
4. Secure technical design details (stage III)
5. Architectural design documents and diagrams
6. Functional and technical specifications
7. Data interfaces
8. Users and user interfaces
9. Internal and external infrastructure components

Artifacts Produced at the End of This Stage At the end of this stage, we will produce the following artifacts: decomposition of application architecture and analysis of the data flows processed by each architectural component and of the security controls that protect each component. Specifically, the following information should be analyzed as part of the decomposition analysis of the application.

(a) *Use cases* (login, account updates, delete users, etc.) *interfaces* including the use cases, users roles-permissions (documented use cases and access control matrix)
(b) *Data interfaces and services* system level/process application to application and server to server including hosted by third parties in the cloud (e.g. SaaS, PaaS, and IaaS)
(c) *Decomposition and analysis of the application architecture* including architectural components (documented end-to-end data flows in DFDs, sequence diagrams, and top-down architecture level)
(d) *Functional security control transaction analysis* includes the identification of use cases, functions/transactions, and security controls to protect data and functions

Tools The execution of this stage can be facilitated by the use of visualization tools such as MS Visio that are also incorporated in threat modeling tool from Microsoft SDL Threat Modeling Tool™. Functional asset decomposition is a feature of MyAppSecurity Inc ThreatModeler™ Tool and is used herein as well as the ThreatModeler™ Architectural Analysis features

Web Application Use Case Example The first activity of this stage consists of the activity *ADA 3.1 – Enumerate all application use cases*. The main goal of this activity

is to enumerate the use cases. From the perspective of software and system engineering, use case describes a list of steps and specifically interactions between a user of the application that is defined also as actor and the application to achieve a desired goal. From the perspective of the application threat modeling exercise, a use case describes a business process flow that shows from the architectural stand point how a user interact with the application to perform a business function. These business functions were previously documented at high level as business requirements and should be either documented or captured at more detail level for the review during this stage. If these use cases are part of the design documentation such as UML, use cases can be easily enumerated and analyzed at this stage also by comparing with the business requirements defined in stage I and users and roles of the application defined in stage II.

The use cases that are enumerated in this stage need to describe an application business function and how the user interacts with the application function to accomplish a specific functionality that the application provides. These business functions might have different type of risk depending on how business critical is the process that this use case action allows to perform since might be targeted by a threat agent and cause a possible negative impact to the business such as loss of accountability, availability, confidentiality, integrity, of the function and the data.

In threat modeling, use cases can be represented as user interaction with the various architectural and functional processes provided by application. In the case of the online banking application, the use cases represent actions that are performed by the user to conduct business transactions with the online bank application in scope such as log-on into the online banking site to view the bank account statement and the transactions conduct money transfers and make payments. The visualization of use cases that can be entered using a palette of a tool such as ThreatModeler™ it is shown in Figure 8.9.

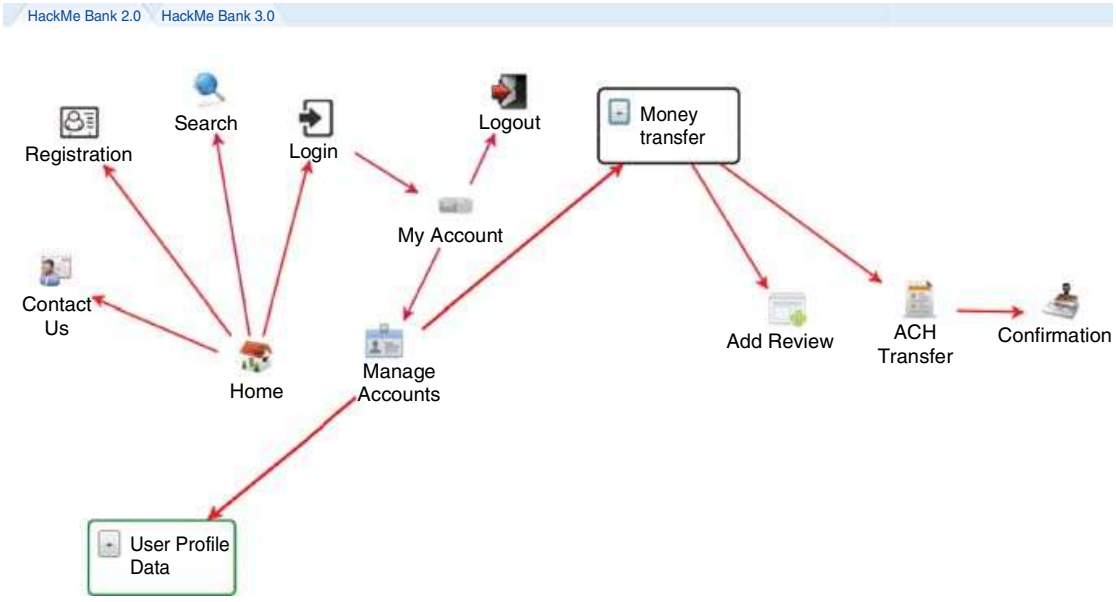


Figure 8.9 Component-Based Functional Use Cases of Online Web Application

Using the ThreatModeler™ tool palette, it is possible to enter each use case using the tool graphical palette to associate users with each component of the application with a functional component. Each functional component is associated with data that is processed by the component as well as with data that is either stored or processed by other functional components.

Similar to architectural level components where the data interactions with components can be expressed in terms of either the user or the data interface and the type of protocol in between as documented in the data interfaces at stage II, the functional interactions among components can also be expressed as interactions of data through communication protocols that are highlighted as arrows between components. In ThreatModeler™ functional components are also associated with the sensitivity of the data that these functional component process either for storage or for transit and this help to assert functional security requirements and security testing requirements for each of these functional components.

Each functional component can also be associated with the data elements such as user and roles with permissions to access or use the component and elements of functionality called widgets that represent how the component interacts with other components to support that functionality (e.g. login interact with the authentication data repository to validate passwords). In ThreatModeler™, each functional component can also be associated with the technology stack and the specific network components such as server components where the functionality of the functional component is installed and deployed.

The decomposition of functional components associated with data processed to and from the component and the associated functionality as defined in the business requirements lays down the foundation of the analysis of threats against data assets and functionality for each component and the evaluation of the exposure to the threat by considering the security measures implemented to protect the data and functionality associated with each component.

After we have captured the use cases, the next step consists of performing the activity *ADA 3.2 – Document Data Flow Diagrams (DFDs).* The goal of this activity is to visualize the information flow end to end (ingress/egress traffic at all layers of the application) and to describe how the data interacts end to end, from the external entities (client, user) to the data store through the processing elements of the application (components, servers). DFD seek to visualize the following elements of the web application architecture:

1. External entities (users, web browser, e-mail-client, external site/system)
2. Processes (Servers: Web, Application, DB server, functions: input validation, authentication, authorization, session management, encryption)
3. Data flows (inbound/outbound function calls, call types, type of data and protocol)
4. Data stores (file, database, any data at rest)
5. Trust levels and trust boundaries

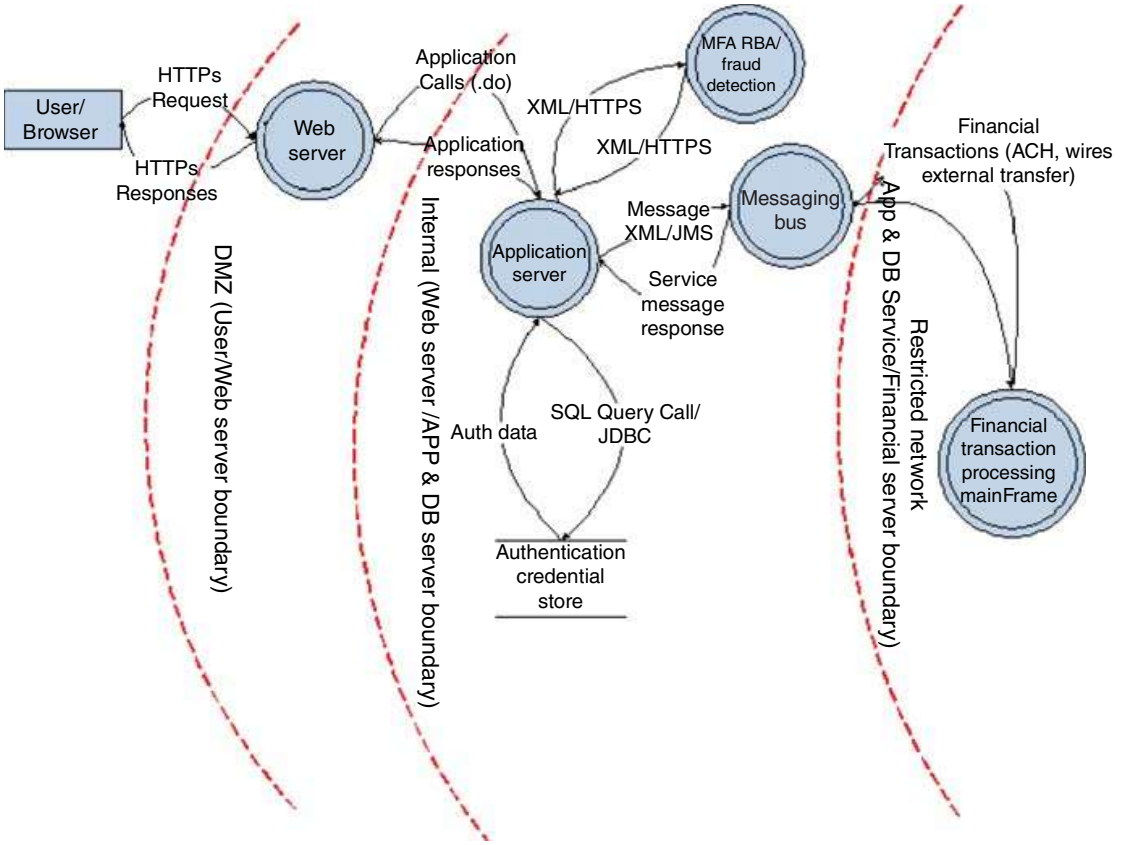


Figure 8.10 Data Flow Diagram for Online Banking Application

The analysis of the application data flows using DFD helps in the identification of the entry and the exit data points at the various trust boundaries and the access levels (anonymous, user authenticated, administrator, super-user) required to access the different architectural components (data, services) within these trust boundaries.

An example of high-level DFD for the online banking application is shown in Figure 8.10.

The DFD in Figure 8.10 can be created using a graphical tool such as MS Visio. The diagram shows the basic data processing components of the architecture of the online banking application that includes the client (e.g. user browser), the web server, the application server, the messaging bus to access the back end services for financial processing as well as web service components providing specific functionality for Risk-Based Authentication/MultiFactor Authentication and Fraud Detection. The data storages and the data flows depicting the type of data stored (user authentication data) as well as the data in transit through the various communication channels and protocols between components are also shown. The DFD also shows the various trust boundaries that are crossed by the data to access each of the components: the De-Militarized Zone, the Internal Only Accessed Network, Trust Zone, and the Restricted Only Accessed Network/Trust Zone.

Once the data components have been decomposed in a DFD diagram, it is possible to assert the presence of security controls for protecting the data such as encryption

in storage and in transit for confidential and authentication data accessed through these components as well as the presence of other controls such as authentication and authorization, input and output data validation, auditing and logging, and session management. By analyzing the flow of data, it is also possible to identify data entry and exit points to assert input validation and when the data crosses the trust boundaries to assert that access controls are enforced for data to cross that trust boundary.

The next activity to be performed at this stage is *ADA 3.3 – Security functional analysis and the use of trust boundaries*. The goal of this activity is to analyze the security of the use cases from the functional perspective by looking at the enforcement of the security controls such as authentication, authorization, input validation, and data protection in storage and transit for each user action and each type of data crossing the trust boundaries at the entry and exit points of the DFDs.

The visualization of the data flows with DFD allows a threat analysis to look at the data being processed for specific users and use cases from both defensive and offensive perspective to identify if the data coming from a specific user role with specific permissions can be trusted before being processed by processes that reside within trust boundary. A use case of authentication for defensive perspective, for example, needs to enforce protection of the confidentiality, availability, integrity of the authentication data in transit from the user client to the credential storage where the data is stored. From the defensive perspective, for example, each user data input needs to be validated before it can be processed by the application server and the authentication data need to be encrypted in storage at the database credential storage. From the attacker perspective, a threat agent can try different types of attacks against the authentication data flow by trying to spoof and tamper the authentication data in transit as well as by repudiate the authentication source by trying to authenticate from untrusted client. This analysis drives the consideration of security controls such as encrypting the authentication token in transit and in storage with digital hash signing the authentication tokens with digital signatures to provide nonrepudiation. At the entry point before the authentication token is issued after positive authentication, the user provided credentials such as userID and password are validated for malicious input such as SQL injection commands and XSS attack vectors. These validations are enforced within the web server trust boundary and within the applications server internal trust boundary. In essence, the visualization of the data flows with DFD helps to identify how likely attack vectors can impact the data from the entry points by exploiting weaknesses (vulnerabilities) of security controls such as authorization, authentication, and secure communication channels across the different layers of the architecture. These are security controls that need to be identified and included in the application decomposition along with their interaction with the various data and functional assets to assert that they can protect the data assets that are considered sensitive such as authentication and sensitive data as well as the business functionality at risk.

Trust boundaries can also be identified at the level of a process running on the specific component. For example, assuming that a user registers to create an online account to authenticate through Single Sign On (SSO) to the online banking

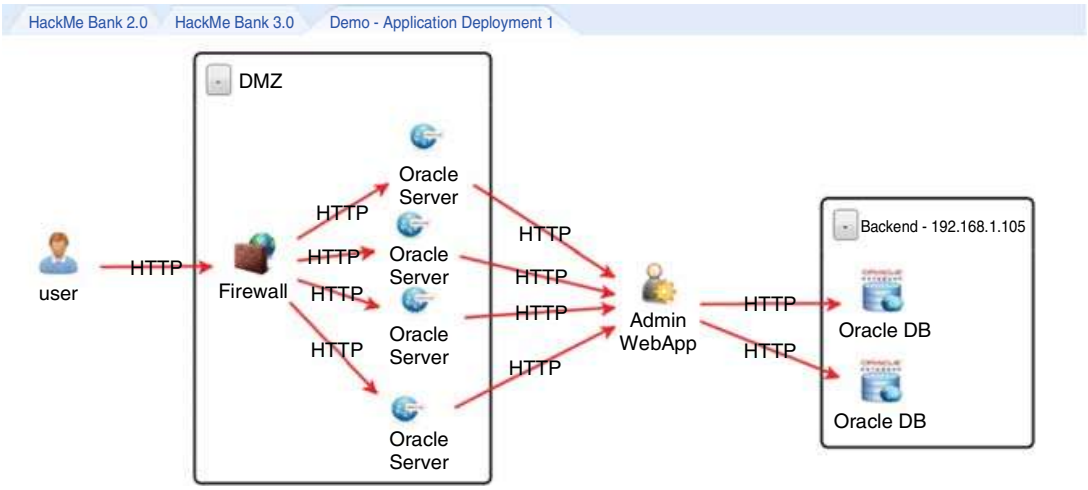


Figure 8.11 Functional Component Trust Boundaries Using ThreatModeler™

application, the user will be under the trust boundary of the registration process. When the user logs into and authenticate to the online banking application, he/she will be in the trust boundary of the web application and the external processes such as browser processes running on his/her desktop/PC. When the authenticated user selects a high-risk business critical transaction such as an ACH (Automated Clearing House) payment, the payment will execute in the trust boundary of that process.

Such visualization of trust boundaries for the user case of user registration, login to manage accounts and make a payment and logout from the online bank application is provided in Figure 8.11. When a user registers to create an account to authenticate through SSO to the web application is under the trust boundary of the registration process. When the user logs into and authenticate, he/she will be in the trust boundary of the web application and the external processes such as browser processes running on his desktop/PC. When the user selects a high-risk business critical transaction such as an ACH (Automated Clearing House) payment, the payment will execute in the trust boundary of that process.

By visualizing the trust boundaries at process/functional level, it is possible to assert the presence of security controls at that level of functionality such as for security controls that protect the confidentiality, integrity, availability, and accountability of the processes that support these functions and specifically high-risk functions such as money payments. Examples of controls that protect functionality are authentication and authorization, encryption, input and output validation, session management, and audit and logging.

The analysis of the application business functions and specifically business critical functions that can be targeted by fraudsters such as transfer of money to external entities, bill payments as well as the process for registration of new accounts, login authentication, and recovery of lost passwords and accounts can also be in scope for specific security functional analysis at the level of each business critical transaction. This activity consists of analyzing each of the business application functions at transactional level to identify the security controls that are in place for validating the data

before it is processed by the application, the enforcement of access controls such as authentication and authorization to access the function as well as session management controls, data encryption, error handling, and logging and auditing.

An example of transaction analysis for online banking application is provided in Table 8.5.

In the first row of the matrix herein included, we have listed a number of user functions that are implemented in the online banking application such as password reset, user ID recovery, account registration, log-on, fund transfers (e.g. wires), and payment functions (e.g. bill pay). For each transaction, we have assigned an inherent risk of the transaction that considers both technical and business impact, the classification of the data that is processed by each function/transaction, the type of data that need to be validated before processing and the various types of security controls that are implemented.

This type of analysis allows to assert the security requirements for the implementation of security controls in each transaction of the application, for example, a password recovery has a high inherent risk because of the possible impact on the confidentiality, integrity, and availability of the application in case passwords are compromised. The password recovery function is available to known registered users and requires a validation of the user debit card number, the debit card PIN, and the bank account number in order to be reset. A wire transfer is considered a critical risk transaction since it allows moving money to another bank account in another financial institution and therefore might have a high business impact if this transaction gets compromised. Because of the critical inherent risk of this transaction, strong authentication such as multifactor authentication is required and the function is only available to authenticated users.

Once the user is authenticated the session is managed by the server Role Base Access Controls (RBAC) and the user need to authorize the transaction by acknowledging the amount and the transaction through an Out Of Band (OOB) channel such as SMS message over the mobile phone. The confidential data in transit such as the bank account data to send the money to and the transfer amount can only be sent over encrypted channel. As detection security controls besides the standard audit and logging of security events, money transfers also require the logging of suspicious events that can be correlated with fraudulent transactions.

At functional level, each one of these critical business functions that can potentially be exposed to external threat agents will be later analyzed in stage V Weaknesses and Vulnerability analysis to identify possible design flaws and will be considered a vulnerability-security finding and reported as such and considered for the potential impact. During this stage, the goal is to architectural and functional level decomposition of the application in security controls and assets that include data, data flows, and application functions.

Stage IV – Threat Analysis (TA).

Goals of This Stage The goals of this stage are the analysis of the threat scenarios based on the various sources of internal and external threat intelligence and the

TABLE 8.5 Security Function Transactional Analysis

Application Function	Inherent Risk	Data Classification	User Role	Input Validation and Data validated	User Authentication	User Authorization	Data Encryption	Audit and Logging
Registration	LOW	Confidential and Confidential-PII	Site visitor	ESAPI, card data, PII, demographics	None	None	HTTPS	Server and App. Logs
Login	HIGH	Authentication data	Authenticated user	ESAPI, userID, password, MFA	Simple and multi factor	Server-side RBAC	HTTPS (transit) SHA1-256 (storage)	Server and App. Logs. Security Event Logs
Password recovery	HIGH	Authentication and Confidential-PII	Known registered user	ESAPI, debit card No, PIN, Bank account	Debit card PIN and card No.	Server-side RBAC	HTTPS (transit) SHA1-256 (storage)	Server and App. Logs, Security Event Logs
Access account balance	MEDIUM	Confidential	Authenticated user	ESAPI, Bank account	Simple	Server-side RBAC	HTTPS	Server and App. Logs
Bill payments	HIGH	Confidential	Authenticated user	Payee account, payment amount	Simple and multi factor	Server-side RBAC	HTTPS	Server and App. Logs, Security Event Logs
Fund transfer/ wires	CRITICAL	Authentication and Confidential	Authenticated user	ESAPI, bank account data, transfer amount	Multi Factor Only	Server-side RBAC, Out Of Band Auth.	HTTPS	Server and App. Logs. Fraud Event Logs

analysis of the threat agents that within this threat scenario are known to target application security controls weaknesses, flaws in design, and vulnerabilities. Based on the analysis of the threat agents and of the specific characteristics of the threat agents and threat targets, it will be possible to update the threat library and use the information of the threat library for the architectural design reviews and for estimating the risk of the potential impact of these threats based upon the knowledge of known weaknesses and vulnerabilities that these threat agents seek to target. Based on the information of threat intelligence and the analysis of the threat events, it will be also possible to provide an estimate of the probability of these threats to target the application data assets and application functionality in scope.

Guidance for Performing This Stage The objective of the Threat Analysis (TA) stage of PASTA(TM) is to conduct the threat analysis in the context of the application/product in scope. The goal of this threat analysis is to analyze the type of cyber-threat agents such as these being human (e.g. script kiddies, hacktivists, cyber-criminals, fraudsters, cyber-spies) and nonhuman (e.g. malware). These threat agents might intentionally or opportunistically target the application in scope. Initially it is important to capture the threat environment that depends on the business and technical environment in which the application operates. This initial analysis is necessary to characterize the threat landscape.

Every threat environment can be characterized by specific threat agent factors and events. The categorization of the threat agents might include threats that are either human or automated, their capabilities such as the type of skills, the attack techniques and tools at their disposal, their motivations (political, monetary, espionage), their opportunities such as the type of targets that can be explored and attacked and the application vulnerabilities that can be discovered and exploited.

The characterization of the threat landscape largely depends on the information at the disposal of the threat analyst such as threat reports and threat information gathered from different sources of threat intelligence such as the Information Sharing and Analysis Centers (ISACs) and sources of security incidents caused by cyber-attacks such as Web Hacking Incident Database (WHID). The purpose of the analysis of threat intelligence is to determine the level of severity of the threats targeting the application assets (data and functionality) in scope and the threat environment and decide the course of action such as issuing alerts and advisories for customers and clients, monitor specific events for detect incoming attacks and be prepared to respond (e.g. reduce the attack surface, monitor critical functions and set limits on functionality, including blocking access as last resort) in case the application will be attacked.

From the security incident response perspective, the monitoring of specific cyber threats events might rely on the logging of specific suspicious events such as abnormal access control violations and triggers (e.g. velocity checks on application functions, web page errors, etc.). For the correlation of these events, the threat analyst might utilize kill-chain techniques (e.g. chain of events) to determine if these events might indicate the potential course of action of a possible attack.

From the application threat modeling perspective, the analysis of threats and threat events is instrumental to the implementation of a threat analysis engine that correlates threat events with targets and security incidents as well as repository of threat information such as threat knowledge base of threat information and attack libraries that can be used to determine the probability of threat events, extraction of threat parameters and correlation of attack vectors with security controls and vulnerabilities that are affected by these attack vectors. The assignment of a probability value to each threat of the threat library can be based on the analysis and assignment of factors of threat probability such as threat agent capabilities (e.g. level of skills required to perform the attack), motive (e.g. level of possible gain/reward) and opportunity (e.g. the level of sophistication of the resources required to conduct the attack) and of threat events associated with the threat agent (e.g. refer to Structured Threat Information Expression (STIX) for threat actor correlation with threat techniques and procedures and correlation with threat observables and threat indicators and security incidents, course of actions and exploit targets.

Standards methods for scoring the severity of threats might be inherent of the threat agent risk factors to determine probability of threats such as the ones used in the OWASP Risk Rating Methodology and (MOSP) Motive, Opportunity, Skill Required, Population Size, and the risk of threat agents to be capable of exploiting vulnerabilities and produce an impact such as in the case of threat (DREAD) risk rating that stands for Damage Potential, Reproducibility, Exploitability, Affected Users, Discoverability. These factors can be assigned score levels and averaged to determine the overall threat agent likelihood factor (OWASP-MOSP) and threat risk severity (MS-DREAD) (Note in the case of MS-DREAD threats are classified as technical threats using STRIDE Spoofing, Tampering, Repudiation, Information Disclosure, Denial of Service, and Elevation of Privileges).

Once the threat library is updated with new threats, the level of risk as probability and impact for each threat should be estimated. One possible estimation of the relevance of each threat is to enumerate each of these threats to the application security controls to determine whether these controls are present to either reduce or increase the level of risk of the threat. The level of risk of the threat can be calculated as factor of probability of the threat to be realized in an attack as well as the technical and business impact of that threat.

Inputs for conducting this stage include the following:

1. Stage I risk profile
2. Stage II technical scope
3. Stage III assets and security controls
4. Threat intelligence reports and advisories
5. Security incidents and events from internal sources (SIRT, SIEM, WAF, NIDS)

Prerequisites for This Stage To conduct this stage, the following inputs are required.

Artifacts Produced at the End of This Stage At the end of this stage, we will produce the following artifacts:

1. *Threat scenarios for the application/systems in scope* based on the analysis of external cyber-threat advisories (e.g. threat severity level from threat advisory to know how close these threats are to be realized in attacks) as well as internal events such as security incidents that can be correlated with these threats.
2. *Updated threat library* that includes information of each of the threats that are part of the threat scenarios each characterized by threat agent factors (use normalized threat characterization) based upon the information learned from threat intelligence.
3. *Enumeration of threats to application data assets and components:* These are the components in scope for the risk analysis (use DTS for data assets and function asset in scope) that have been decomposed in stage III.
4. *Assignment of the initial risk severity of the threat* as this is the probability and impact for these specific threats targeting application security controls and application components (using threat agent probability factors of skill level, motivation, opportunity). (NOTE Possible threat probability assignment methods are OWASP and DREAD (but considered too subjective) to assign risk to each threat).

Tools The execution of this stage can be facilitated by the use of and standardized by relying on formal methods for the threat analysis such as:

(a) Sources of Open Source Threat intelligence (OSINT) such as threat alerts from various security vendors.
(b) Threat classification methods for categorizing threats (e.g. STRIDE, OWASP, MITRE-STIX, Events-Incidents, Agents, Motives, Damage Potentials, Tools, Exploits, Impacts (application-users).
(c) Risk control frameworks that map threats to assets and controls such as Microsoft STRIDE and Web Application Security Frame.
(d) Threat risk severity calculation formulas from OWASP, FFIEC, and Microsoft (i.e. DREAD) to calculate severity for each threat agent-source by considering the probability of the event.
(e) Threat libraries such as WASC WHID attack library and threat management dashboard integrated in threat modeling tools such as myAppSecurity Inc ThreatModeler™ tool.

Activities This stage consists of the following activities:

T4 4.1 – Analyze the overall threat scenario: Based on correlation of threat agents, events, campaigns, observables, indicators of compromises, techniques, tactics and strategies, and the targets. A visualization of the threat landscape with the

correlated threat information with graphics and diagrams allows the threat analyst to analyze the various sources of threat agents with indication of these threats being realized in attacks and correlated with possible security incidents occurring to the web application.

TA 4.2 – Gather threat information from internal threat sources: Examples of sources of threat intelligence might be collected as event triggers from different systems at network, system, and application layer. A security event might be considered a suspicious activity that can be monitored and detected. Examples of suspicious activities include repetitive failure of logging into an account as indication of brute force attacks against passwords, logging of traffic from suspicious domain or IP, detecting SQL injection attack vectors using a WAF as indication of a vulnerability scanning and so on. The association of internally logged information with a threat requires aggregation of events from different internal sources (e.g. firewalls, IDS/IPS, servers, network management routers and switches, and vulnerability scan engines), permanent events storage in a database and analysis using threat management systems and log aggregators and security management consoles with analytics and filtering of these events as possible threat event alerts that can be actioned for security incident response as well as for issuing internal threat alerts.

TA 4.3 – Gather threat information from external threat sources: Examples of sources of external threat intelligence are external vendors, law enforcement, secret services, (CERT) Computer Emergency Response Teams, and ISACs Information Security Assurance Centers. The type of cyber-threat intelligence information might include OSINT Open Source Intelligence such as publicly released advisories from response centers such as CERT as well as confidential and proprietary sources of threat intelligence. Prior to gather the threat intelligence is important to have established a protocol for threat information sharing and dissemination among parties and a standard to categorize the threat information. Examples of categorization of the threats including the identification of the threat agents (e.g. human, automated), their skills, the group capabilities, motivations, opportunities, the type of targets such as the vulnerabilities exploited and the threat severity as a factor of information of the threat targeting specific industry sectors or even specific assets within that industry sector/vertical using specific type of tools, techniques, and processes (e.g. account takeover with malware, DDoS). This activity includes the threat source/report data collection from source of threat intelligence and aggregation and categorization of threats. Standard threat dissemination protocols such as TAXII and standard threat representation such as STIX can be used for sharing information about threat events that are observable and the type of indication of compromise events of these observables.

TA 4.4 – Update the threat libraries: Update the threat libraries for emerging threats whose characteristics are aggregated from the sources of threat. These threat libraries should include information that is actionable for threat-risk mitigation such as the type of threat agents/actors, the threat campaigns, the threat

capabilities and tools used by the threat targets such as the company assets (e.g. data and functions), the course of actions, the incidents, and the security incidents observed including the security controls and measures impacted and the vulnerabilities/gaps that are exploited in attacks/incidents known to be attributed to these threat agents

TA 4.5 – Threat agents to assets mapping: Once threats have been visualized, they can be correlated with the application assets such as data and functionality as well as potential vulnerabilities of security controls that protect these assets and might expose these assets to the threats. The correlations of threats to assets support an initial analysis of the risks as factor of likelihood and impact of attacks being realized for these threats.

TA 4.6 – Assignment of the probabilistic values for identified threats: As this is the probability and impact for specific threats previously analyzed to target valued assets of the organization such as confidential and sensitive data stored and processed by an application system as well as business critical operations/functions of that organization/system. Threat probability can be associated with a threat agent probability as a factor of the threat agent skill level, motivation, and opportunity such as availability of cyber-crime tools, knowledge of attacks tools and techniques, use of known processes and procedures for successfully conducting attacks, attack strategies and organization in cyber threat groups (e.g. cyber-criminal groups, hacktivist group etc). At higher level, the probabilistic severity of a threat can be rated as how close is the threat to attack an asset and produce an impact. Attack kill-chain can be used to determine the threat probability such as how far the threat agent is in the execution of the attacks against a target starting from intention and survey initial discovery of the target, research of the target, reconnaissance, exploit of vulnerabilities to conduct the attacks and operation execution and control of the attack against the target. Note: the impact of the threats analyzed during this stage can be considered as unmitigated till a further validation of the presence of design flaws and vulnerabilities (stage V) in these controls as well as possible exploits of these controls by specific attack vectors identified during the attack modeling and simulation stage (stage VI).

Web Application Use Case Example The first activity of the threat analysis stage consists of "*TA 4.1 – Analyze the overall threat scenario.*" In the case of the online banking application in scope, there are several types of threat scenario.

A threat scenario for an online banking application includes the various types of threats such as malware and hacking threats seeking to compromise confidential customer information and to commit fraudulent financial transactions, Distributed Denial-of-Service DDoS threats seeking to flood the website with requests rendering it unserviceable for customers, threats of compromise of confidential data such as PII, bank account data, and credit card data of customers.

Prior to conducting a specific analysis of the various cyber-threat scenarios that are relevant to the online bank application, threats need to be prioritized for the threat

analysis. This prioritization might depend on different factors such as publicly available data of data breaches attributed and internal events such as security incidents that can attributed to specific threat actors and threat vectors used by these threat actors.

For example, if we consider the survey of data breach incidents from the 2013 Verizon data breach report, the top threat actions of malware include attack vectors that seek to capture data from user activity using key loggers, form-grabbers, and spyware involving sending data to external site and the installation of a backdoor to allow remote access control. The top hacking threat vectors include exploitation of guessable credentials, backdoors and command and control channels, brute force attacks, and use of stolen login credentials. Within the hacking threat, the most used attack vector used against large organizations are web applications, followed by the installation of backdoors or control channel and remote access by compromise of desktops. Based on the data of this survey, among the various sources of threat agents, external threat agents motivated by financial gain represent the largest threat accounting for 87% of attacks followed by only 5% of internal threat agents, 3% of partners, and another 5% as unknown.

Based on this information, external threat agents motivated by financial gain using malware and hacking techniques to target web applications represent a top priority threat scenario to be considered in the threat analysis. This is based on security incident data breach driven survey approach. In reality, internal threat agents cannot be neglected in the analysis as this might impact the financial organization with the largest impacts. Examples of internal threats against online banking applications include internal threat sources such as disgruntled employees and malware uploaded on internal servers for exfiltration of data from the inside network such as backdoors and sniffers.

Another driver to prioritize the type of threat scenarios is information from threat intelligence of specific threats targeting specific targets such as online banking applications for reputational damage and for cause disruptions of service to customers such as in the case of Distributed Denial-of-Service (DDoS) attacks. Since these types of attacks are politically motivated, they seek to gain attention from public. One method used is to announce the plan to conduct the attacks on social media sites.

An example of gathering of threat intelligence from public sources such as searches of attacks from threat agents is shown in Figure 8.12.

The diagram in Figure 8.12, for example, shows all the recorded events in the public media of the announced campaign of Distributed Denial-of-Service attacks by the Izz ad-Din al-Qassam Cyber Fighters AQCF threat agent group. For most of the financial organizations today, being either large or medium, the threat of DDoS attacks targeting the online banking sites cannot be neglected and need to be prioritized in the analysis of the threat scenarios.

The information of announced attacks of specific threats such as DDoS but also account takeover malware and other threats can be the driver for a more in-depth analysis of these threat scenarios to understand the various steps of the attacks, the types of tools, techniques, and processes used by the threat agents and the type of weaknesses and vulnerabilities that are exploited. This analysis allows to identify

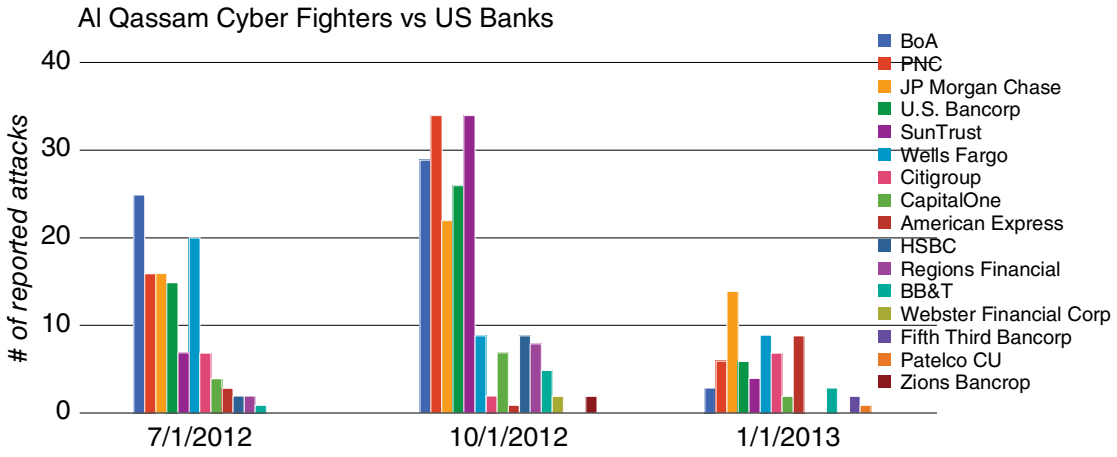


Figure 8.12 Campaign of DDoS Attacks Against Banking Sites Announced by AQCF Threat Agent Group

the effectiveness of current security measures and security controls and to propose improvements to better respond and reduce the impact of these attacks.

In order to analyze the overall threat scenario for the online bank application in scope, it is important to adopt a standard characterization of each type of threats. External threat agents, for example, can be classified as human and nonhuman attacking applications from outside the network perimeter. Depending on the type of external threat agents and the tools at their disposal, different type of malware and hacking techniques could be used. These techniques might not be very sophisticated such as in the case of exploit of server misconfigurations and exploit of vulnerabilities that can be easily identified with the use of free vulnerability scanning tools. These types of threats are also defined as opportunistic. Examples of opportunistic threats consist of threat agents seeking to exploit application vulnerabilities such as weak input validation, weak authentication and authorization, weak session management, gap and weaknesses in the protection of data in storage and transit such as encryption and gaps in auditing and logging of specific events to detect attacks as well as weaknesses in protection of the logs so that attackers can clean up their tracks.

Different from targeted threats, targeted threats might not exploit weaknesses in web applications but seek to compromise the client of the application by installing malware that is designed to attack the application and is designed to specifically bypass applications controls that are typically considered strong such as multifactor authentication and compromise servers by bypassing network layer defenses such as firewalls and intrusion detection systems. Examples of targeted threats against online banking applications include account takeover that consists of compromising the customer banking account by staging an attack that starts by compromising the customer's host computer by social engineering the victim to select malicious links that point to malware. This malware is often banking malware that when executes on the compromised host will hijack the authenticated session and take over the online banking transactions and later respond to a command and control center instructions under the control of the threat agent.

The analysis of the course of action of these threats is critical to determine which countermeasures might be deployed to proactively detect and protect from these threats. This is the value that threat analysis provides in terms of proactive risk management of these threats before the application will become a target by these threat agents.

From the perspective of proactively mitigating the risk of threats exploiting known vulnerabilities in web applications, today the focus is mainly around vulnerability assessments that identify and fix any vulnerability that can be exploited by the threat agents hence reducing the opportunity for the attacks to exploit them to conduct his attacks. This approach might be sufficient to mitigate the risk of threats opportunistically exploiting known vulnerabilities and weaknesses but might not be enough in the case of advanced and targeted threats including malware hacking and DDoS. For these types of threats, a focused threat analysis and attack modeling is required to identify additional security measures that can be designed and implemented in the web application as well as outside the web application. Such comprehensive analysis of malware banking and DDoS threats need to focus on the understanding of the characteristics of the threat agents and the attack techniques, tools, and strategies used.

To accurately capture the overall threat scenarios, it is important that we adopt a standard categorization for the types of threat agents (e.g. human, automated) and we define them according to their capabilities, motivations, opportunities, the type of vulnerabilities exploited, targets, and cyber-threat severity of incidents being reported. Threat categorization is also useful to create a schema for aggregate threat information that can be later analyzed to determine the threat targets and the type of weaknesses and vulnerabilities that these threats seek to exploit.

In order for the threat intelligence information to be useful for the threat analysis, it is important to categorize the threat sources with specific parameters so that these can be correlated to conduct specific analysis. The more detailed the categorization of the threats the more comprehensive and focused the threat analysis can be.

At high level, threats can be characterized based on the type of threat agents, the threat agent capabilities and motivations, the threat events such as security incidents associated with these threat agents, the threat agent capabilities, the various types of techniques and tools used and the threat targets specifically any type of value assets such as customer sensitive data and money-valued functionality that might also put the online banking application under possible threat of attacks from these threat agents. An example of categorization of threat agents and the type of overall threat scenarios and how they might affect web applications and IT systems of financial institutions is included in Table 8.6.

At high level, the analysis of the overall threat scenario consists of looking at the overall picture of the various types of threats. Based on this high-level view of threats and the correlation of the various parameters that characterize the threats, it is possible to decide which threats are more relevant and to conduct a more focused analysis such as by focusing on the threats that higher probability to attack the online banking application or the ones that might cause the larger impact (e.g. internal threats).

TABLE 8.6 Overall Cyber-Threat Scenarios Affecting Financial IT Systems and Applications

Cyber Threat	Cyber-Crime	Hacktivism
Threat actors	Cyber-gangs, fraudsters	Anonymous Groups, State Sponsored Groups, Script Kiddies
Motivation	Financial gain from stolen money from bank accounts, fraudulent transactions, purchase of good with stolen	Political agendas, cause disruptions, get noticed and damage the company reputation
Goals	Monetize stolen credit card data, online fraud, account takeover, movement of money to fraudulent accounts, counterfeit of credit and to purchase goods and debit card data to withdraw money from ATMs with counterfeit debit and prepaid cards.	Expose confidential data of adversaries to public for political reasons including company and government secrets. Defacing of websites to damage brand and reputation. Denial of service to web sites to gain attention and cause economical damage to organizations.
Events	Security incident reported on 5/9/2013: $ 45 million stolen at ATMs worldwide by breaking into computer systems of card payment processors in the United States and India.	Cyber-threat campaign announced on 5/1/2013: ACQF announced the start of the week nine of the third phase of operation Ababil, the distributed denial-of-service (DDOS) attack against major financial institutions in USA
Capabilities	Global cyber-crime and fraud operations, knowledge of configuration and use of cyber crime-tools such as banking malware and to conduct phishing campaigns to distribute malware and to run exploits of vulnerabilities in applications to install backdoors in systems for stealing sensitive data.	Recruiting of hacktivists threat actors by running campaigns on social media such as twitter, Facebook and YouTube. Capability of conduct coordinated attacks by renting and acquiring cyber-crime tools and botnets. Knowledge of DDoS attack tools as well as of exploit web application vulnerabilities.

(*continued*)

TABLE 8.6 (*Continued*)

Cyber Threat	Cyber-Crime	Hacktivism
Techniques	Social engineering users to select malicious links Drop malware with key logger/spyware/form grabbing capability to steal confidential data Use of man-in-the-middle attack techniques to bypass MFA and to hijack authenticated session and take over accounts and money movement transactions Use of man-in-the-browser attacks to inject HTML to social engineer users to provide additional ATM PINs and credit card data such as CVV Send stolen credentials to external site/entity Transfer money to money mule accounts Install backdoors on compromised servers allowing remote access to credit card and payment data Install network sniffers to capture payment data in transit from POS and credit card processing	Public recruiting of threat actors through social media outlets and public forums Use of free and off-the-shelf DDoS attacking tools for coordinated attacks against targets Renting of botnets to coordinate DDoS attacks from compromised hosts Exploit of layer 4 (network layer) and layer 7 (application layer) weaknesses and vulnerabilities Exploit of general web-application vulnerabilities, such as XSS, SQL injection, and Directory Traversal for stealing confidential data Used anonymity services to disguise sources of the attacks Post documents obtained from hacking attacks in anonymous web sites (e.g. pastebin)
Targets	Banks and bank customers, merchants, credit card processors, host computers such as desktops and server such as web and application servers and databases, web and mobile applications and POS software and hardware.	Online banking sites, high visibility government sites, sites or services hosted on high-profile web servers, credit card payment gateways, root name servers and services used at Internet service providers

A tool that aggregates threat information such as threat dashboard can be useful for a threat analyst to conduct a more focused analysis of the threats based on specific parameters and narrow down the threats that matter, for example, at the application and project level. Specifically, such threat dashboard could be embedded in an application threat modeling tool and allows the threat analyst/modeler to extract the threat information about the top type of threats and threat targets, the type of data and functional assets being targeted, the vulnerabilities and weaknesses exploited, and the assessed presence of countermeasures and controls to mitigate these threats.

Based upon the data provided in a threat dashboard, a threat analyst can prioritize the enumeration of threats by starting from the ones whose risk is higher. This is the approach of "analysis in-depth first" opposite of the approach of "breadth first" that in essence consists of enumerating all cyber-threats of the threat library that data and functional components are affected to. This type of threat analysis is also referred to as "threat enumeration" and "threat analysis per asset and component" and will be conducted later on during this stage by conducting the activity TA 4.5 threat agents to asset mappings.

After the overall threat scenarios have been analyzed, the next activity consists of *TA 4.1 analyzing threats by gathering threat information from internal sources.* The gathering of threat intelligence is essential activity for proactive cyber-risk management and cyber-threat prevention specifically for the analysis of the specific types of security events that can be monitored to detect the occurrence of attacks as well as for the type of vulnerabilities that can be targeted during these attacks.

Examples of sources of internal threat intelligence include threat events gathered by internal honeypots, threat events correlated by the analysis of application and system logs, information collected from Web Application Firewalls (WAF) set to monitor specific cyber-threat events such as exploit of common vulnerabilities targeting the organization website. SIEM (Security Internet Event Monitoring) can also log threat events when these are logged by different systems and can be correlated with suspicious activities of user actions indicating possible attacks based on specific rules such as velocity checks, authentication failure events, attempts to scan for common vulnerabilities for the sake of later exploit them in an attack. Other internal sources of threat events include events detected by fraud detection systems shared through internal fraud management teams and threat events that might have triggered a security incident by the Security Incident Response Teams SIRTs

Gathering threat intelligence from internal sources provides a lot value for threat prevention when this information is correlated with threat information coming from external sources. This is the focus of the activity *TA 4.1 – Analyzing threats by gathering threat information from internal sources.* Before we gather threat intelligence information from external sources, it is important to cover some basic standard threat categorizations for threat information that is shared across different external threat sources. This standardization of threat information is also important to represent threat information in structured manner that can be shared, understood, and analyzed by different threat analysts. An example of this standardized language for the sharing and analysis of cyber-threats is the STIXthat stands for Structured Threat Information expression.

STIX has been developed by the MITRE Corporation as an open standard for describing cyber threat information in a standardized and structured manner. According to MITRE "*STIX characterizes an extensive set of cyber threat information, to include indicators of adversary activity (e.g., IP addresses and file hashes) as well as additional contextual information regarding threats (e.g., adversary Tactics, Techniques and Procedures [TTPs]; exploitation targets; Campaigns; and Courses of Action [COA]) that together more completely characterize the cyber adversary's motivations, capabilities, and activities, and thus, how to best defend against them.*"

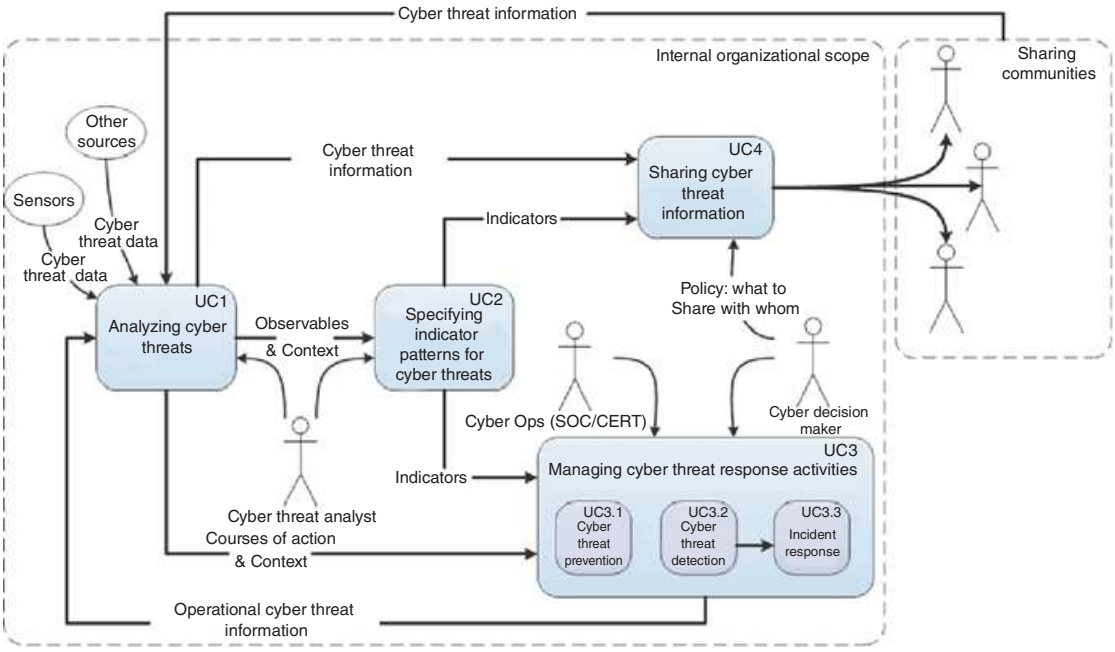


Figure 8.13 Ontology of (STIX) Structured Language for Cyber Threat Intelligence Information (Courtesy of MITRE Corp)[1]

The core use cases of sharing and analyzing cyber threat intelligence are shown in Figure 8.13 as well as specifying indicators patterns for cyber-threats.

From the cyber-threat risk management perspective, the most important aspect of STIX is to use the collected cyber-threat information for preventing or detecting cyber threat activities that includes mitigate vulnerabilities, weaknesses, or misconfigurations that may be targets of exploit. From the threat modeling perspective, STIX cyber-threat syntax includes the notion of threat TTP that stands for Tactics, Techniques, and Procedures that is a representation of the behavior or modus operandi of a cyber-threat agent including the use of particular attack patterns and exploits.

In the STIX cyber threat schema, the tagging of cyber-threat target consists of the "ExploitTarget" and represents a potential victim that may make them susceptible to a particular adversary TTP (e.g. a system vulnerability, application weakness, or configuration issue).

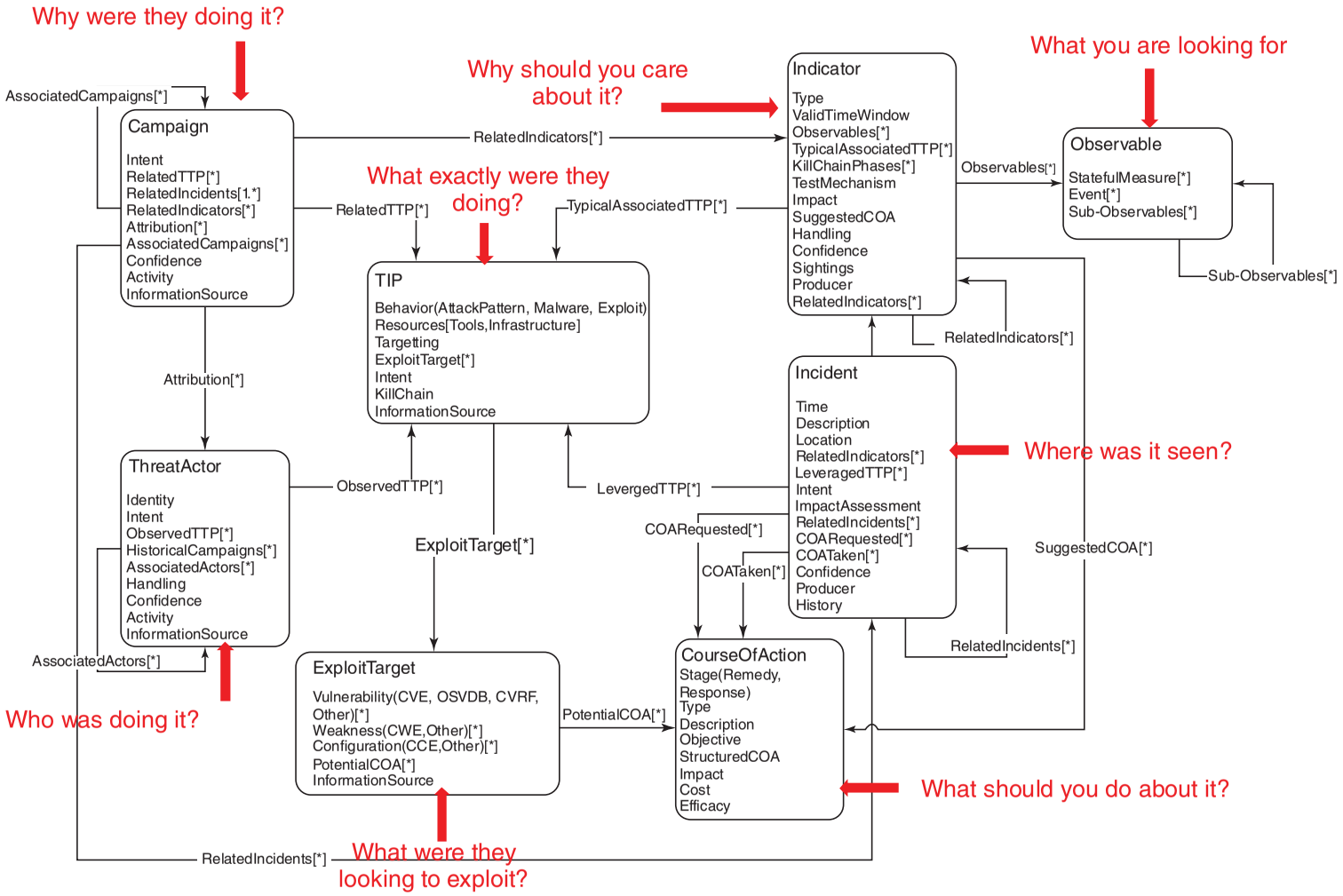
The STIX cyber-threat schema provides a very comprehensive set of attributes that can be used for the analysis of cyber-threats as shown in Table 8.7.

Each of the STIX threat intelligence information sharing attributes has a special meaning. The main cyber-threat attributes of STIX include the following:

1. *Observable* is an event that can be observed in the cyber crime domain. Examples include IP addresses, registry keys, HTTP methods as well as recorded events such as the deletion of a file.

[1]https://stix.mitre.org/images/STIX_Core_Use_Cases_large.jpg.

TABLE 8.7 Structured Threat Information eXpression (STIX) Architecture vs 3.0



2. *Indicator* is a parameter that provides the indication of the cyber-attack context for the observable; for example, if the observable is dropping of a file, the indicator is a malicious file including a zero-day vulnerability or a Trojan.
3. *Incident* describes the activity associated with the attack that includes information of the "who" is involved, the "when," that is, when the incident occurred, the impact, and the Course of Action (CoA) taken in response to the incident.
4. *TTPs* are the Tactics, Techniques, and Procedures that describe how the threat agent operates to conduct attacks.
5. *Exploit Target* is the threat agent target for the exploit. The target can be associated with a vulnerability and a weakness in the system, application, or network being targeted.
6. *Course of Action* is the action that can be taken to prevent, mitigate, and remediate the effects of a given cyber-threat.
7. *Campaign* is a set of adversary activities such as the tactics, techniques and procedures, indicators, exploit targets, and incidents.
8. *ThreatActor* is the cyber-adversary that is the entity perpetrating the attack.

According to STIX definition, an observable is defined as "An Observable is an event or stateful property that is observed or may be observed in the operational cyber domain, such as a registry key value, an IP address, deletion of a file, or the receipt of an http GET. STIX uses Cyber Observable eXpression (CybOX™) to represent Observables."

An example of using STIX language for sharing threat intelligence of an observable is shown in Table 8.8.

The cyber-threat observable has an "observableID," a cybox "ObjectID" and represents a URL (CybOX URI object), the "kill-chain" phase relevant for this observable and a phase ID of the kill-chain. The handling of the threat intelligence information is done according to a marking. The information marking allows a threat analyst to specify how the content within this observable should be handled based upon the color given to the information that maps to different levels of sensitivity of the information. In the example herein, the marking of this information is considered AMBER marking and the sensitivity if "For Official Use Only (FOUO)" a term that under DoD rules usually designates unclassified information.

This threat intelligence information is actionable toward the monitoring of the specific domain as possible source of delivery of malware through drive by download that has Silverlight browser plug-in enabled.

From the application threat modeling perspective, an interesting element of STIX is the "ExploitTarget" construct. This construct allows a threat analyst to map cyber-threats to the exploit targets such as, for example, vulnerabilities that can be exploited to compromise the target and conduct an attack. The "ExploitTarget" construct actually allows a specification of vulnerability that can be exploited such as a Common Vulnerability and Exposure (CVE) type of vulnerability.

TABLE 8.8 Example of Description of Browser Exploit Threat Using STIX

id	Repository-00ff06f0-996d0-44bb-a1263-449cf4a53444	
Title	Browser Plug-in Exploits	
Type	Domain Watchlist	
Alternative_ID	CISCP:indicator-5889988f2381-5887-451d-9b9b-685a15aea480	
Description	Browsers were redirected to download files from hacked.acme.com. No further information such as port number or protocol is available.	
Observable id	repository-9f6c1099-a630-4d64-bf6c-55997efd5ccf""	
	cybox:Object id:	repository-f999ebca-88c2-41f0-87f1-82e641432733
	cybox:Properties:	URIObj:URIObjectType
	URIObj:URIObjectType	Domain Name
	URIObj:Value	Equals hacked.acme.com
	Kill_Chain_Phases	stixCommon
	Kill_Chain_Phase	
	Ordinality	3
	Name	Delivery
	phase_id	stix:KillChainPhase-79a0e041-9d5f-49bb-ada4-832299b162d
	Handling	
	Marking	
	Controlled_Structure	//node()
	Marking_Structure	AMBER
	simpleMarking:Statement	FOUO

An example of a sample of threat intelligence sharing of an observable using STIX is shown in Table 8.8. This is the observable of an exploit of a browser plug-in to redirect the browser to download malicious files from a specific domain and whose kill-chain phase is "delivery."

The STIX cyber-threat schema also incorporates the notion of the "kill-chain." The kill-chain is a representation of the sequence of events that cyber-threat will undertake to realize a threat into an attack that is from the initial reconnaissance events to the execution and maintenance of the attack. A graphical example of the steps pre- and postexploit of vulnerabilities from a cyber-threat is shown in Figure 8.14.

The modeling of attacks with kill-chain is particularly useful to decide which cyber-threat events need to be detected and monitored to respond to cyber-attacks. In the case of cyber-threat against online banking sites, for example, the kill-chain information can be used to decide which threat events can be monitored and correlated as indication of preparation of an exploit (e.g. installing malware on a target machine) and after the exploit take place as indication of execution of the exploit (e.g. stealing of sensitive data).

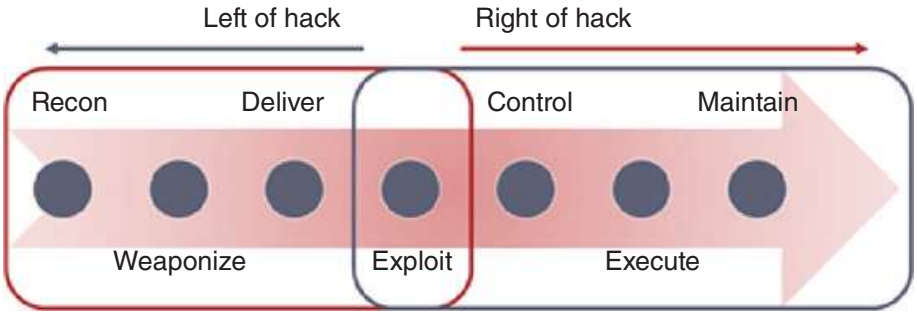


Figure 8.14 Example of Kill-Chain (Courtesy of MITRE corp)[2]

A kill-chain of a malware attack against an online banking site for example might include the following sequence of events:

1. *Reconnaissance* event triggered by the threat agent to learn about the online banking sites that can be targeted such as by crime exploit of common vulnerabilities using scanning tools.
2. *Weaponize* the attack tool trojan/malware by collaborating with other threat actors in the underground cyber-crime forums.
3. *Deliver* of the banking malware to the target (e.g. client machine) such as through e-mail phishing campaigns directly targeting the online banking customers and by injecting malware on servers (e.g. inject malicious links in vulnerable web server/websites) so that it can be later dropped to the victim visiting the site with a cyber-threat technique referred as "drive by download."
4. *Exploit* that is compromise of the client machine with banking malware or the server with backdoors for the sake of stealing sensitive data such as payment data, transactions, and credit card data.
5. *Control* that is the control of configuration of the online banking malware tool by the bot command and control server granting the cyber-threat agent the control to initiate the attack (e.g. when the online banking customer log in to conduct a transfer of money to external account or a payment).
6. *Execution* that is the execution of Man-in-the Browser (MiTB) to collect sensitive data from the victim and Man-in-the-Middle (MiTM) to conduct the authenticated web session hijacking for bank account takeover and movement of money from the victim account.
7. *Maintenance* that is to maintain persistence on the compromised server at the financial institution to exfiltrate bank account and credit card data from the backdoor installed on compromised server in the internal network.

[2]https://stix.mitre.org/about/.

From the perspective of aggregating the cyber-threat information from sources of threat intelligence such as the FS ISAC (Financial Sharing, Information Security Assurance Center), the classification and structuration of cyber-threat information according to STIX allow performing cyber-threat analysis on various types of cyber-threats attributes and to decide the course of action for responding to these cyber-threats.

Another source of cyber-threat intelligence includes the Web Hacking Incident Database (WHID) that is a project run by the Web Application Security Consortium (WASC) and is dedicated to maintain a list of web application-related security incidents. The data of WHID, for example, includes information about attack methods such as reported incidents caused by cyber-attacks against web applications that are reported in the public domain. These events can be correlated with the attacks observed in the public honeypot and then mapped to the most common vulnerabilities that these attacks seek to exploit.

The information about the attacks against web applications that is collected by the WHID can be later used to prioritize the remediation of vulnerabilities that are observed to be exploited by the attacks observed. WHID attack libraries can be incorporated into threat modeling tools such as ThreatModeler™, which is shown in Figure 8.15.

After the cyber-threat intelligence information has been aggregated and stored in the organization threat intelligence knowledge base, the next step consist of updating the threat libraries that can used to conduct a threat analysis.

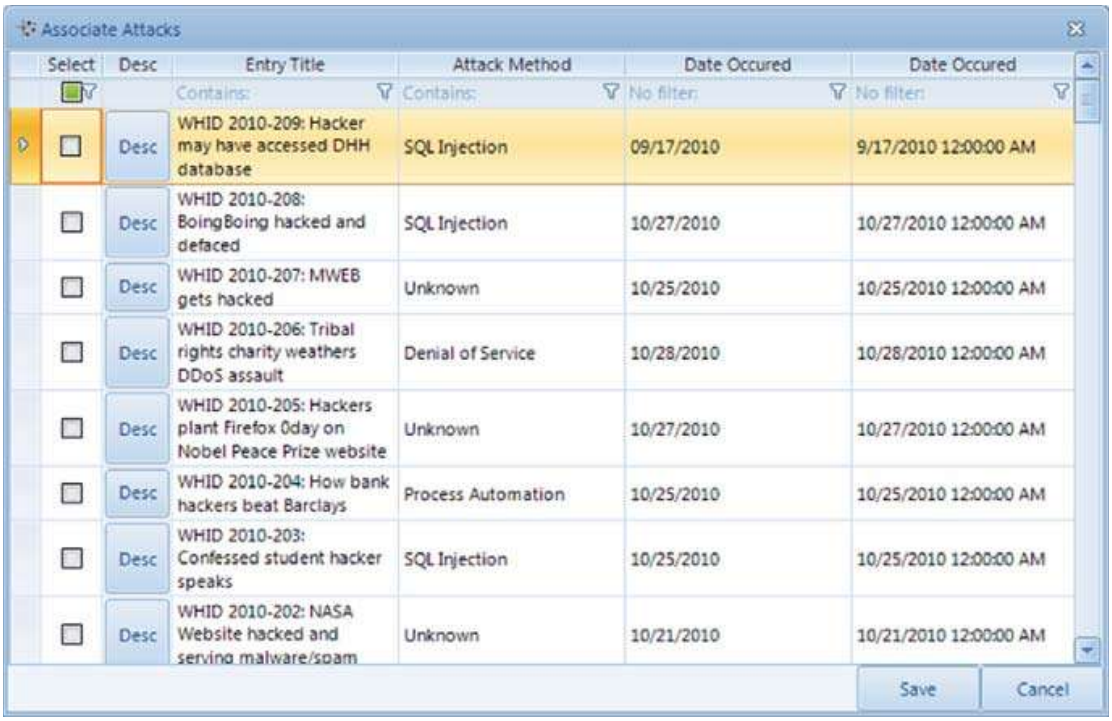


Figure 8.15 Web Incident Hacking Database Attack Library

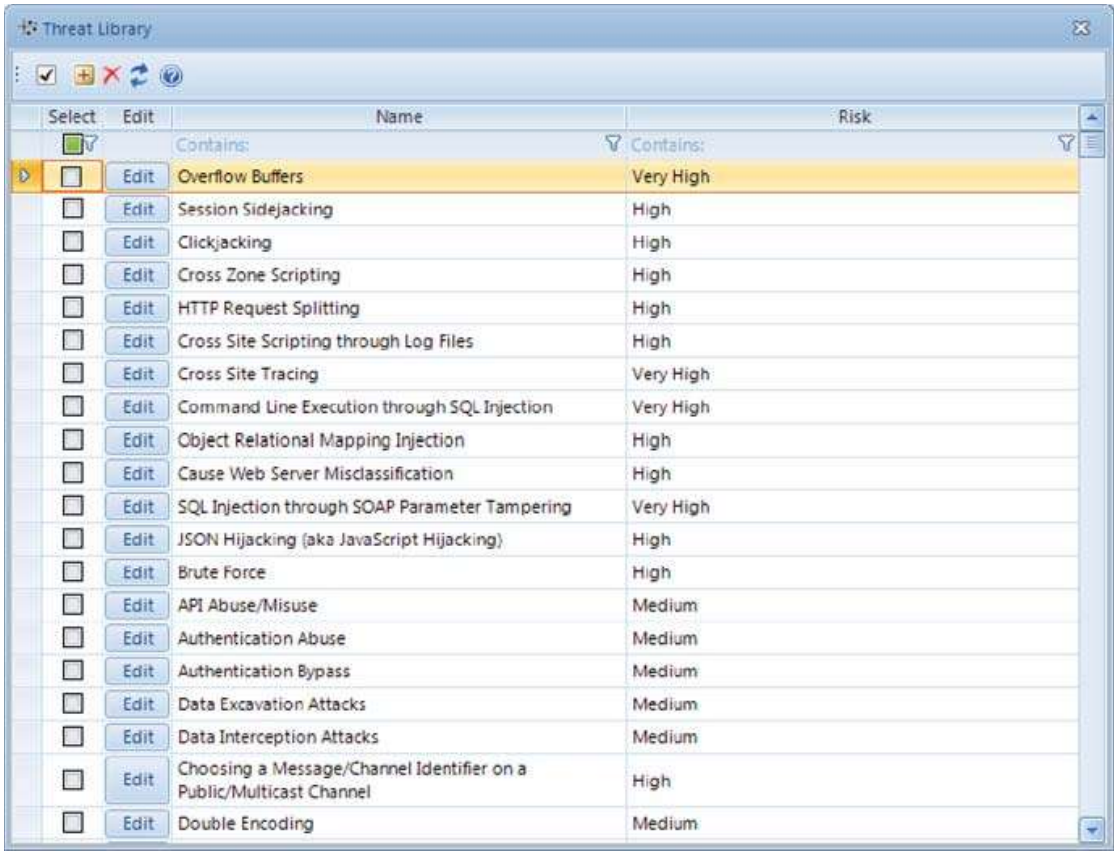


Figure 8.16 ThreatModeler™ Tool Threat Library

This leads to the next activity that *TA 4.4 – Update the threat libraries* with the new aggregated threat information from the different sources of threat intelligence. Threat libraries can be incorporated in threat modeling tools and enumerated against the various security controls of the application to drive security requirements, conduct tests, and identify vulnerabilities. An example of generic threat library that is incorporated in the ThreatModeler™ tool is shown in Figure 8.16.

Threat libraries can be updated with the inclusion of new cyber threats as these are shared from different sources of threat intelligence. As new threats emerge, it is important to reevaluate the application exposure to these threats and specifically derive a new set of security requirements for the secure design of the application architecture whose cyber-threats need to be mitigated and to revisit the architecture design reviews to identify potential design flaws and vulnerabilities that these new threat might seek to exploit and security test the application against new attack vectors that seek to exploit these vulnerabilities and specifically by following the same tactics, techniques, and procedures used by the threat agents.

Since security requirements, security issues, and security tests are associated with architectural components such as web servers, application servers, and databases, the best use of these updated threat libraries is by associating new emerging threats with the data and the components that might be exposed by these threats. Besides the update of the threat library to map to the existing components such as the company

assets (e.g. data and functions), it is also important to update the different parameters that characterize each cyber-threat such as the types of threat agents/actors, the threat campaigns announced by the threat agents, the threat agent capabilities and the threat agent tools, techniques and strategies, the security incidents observed including the security controls and the security measures impacted (e.g. authentication, authorization, encryption, input validation, session management), and the control gaps such as design flaws and vulnerabilities that are known to be exploited by these threats.

The mitigation of the risk of new threats might require new countermeasures; hence, when these countermeasures are identified, they need to be incorporated in an updated risk control framework to evaluate new application as well as changes to these applications.

As an example, the update of the threat library to include banking Trojan type of threats drives the update of the risk control framework to include the following new and updated security controls:

- Updated fraud detection and monitoring controls with updated rules
- Updated anomaly user behavior transaction detection rules controls
- New suspicious user activity detection rules to be monitored
- Dual controls for initiate and approved/authorize high-risk transactions such money movements/transfers and payments
- Strong multifactor authentication controls such as OOB One-Time Passwords to authenticate users and authorize money movement transactions and payments
- Use of additional layers for authentication that relies on different factors that can be validated on the device where the transaction originates such as biometrics.
- OOB verification and authorization for money movement transactions (e.g. banking customer receives notification of authorize a transfer over the phone and ought to positively confirm the transaction before execution)
- Use of external device such as USB/token to establish a two-way SSL channel with the online banking site
- Maximum transaction limits on money transfers such as max money value amount per transaction, allowable payment window
- Blacklists and white list of money transfer recipients
- Reputation-based controls to identify "true IP addresses" and block transactions from known bad IPs
- Complex fingerprinting of the device to determine the trustworthiness of the device based upon flash one-time cookie, browser settings, O.S settings, IP address, and geolocation of the device
- Malware detection and protection for client browsers/PCs
- Customer subscription alerts via e-mail or SMS for money movements

Once the threat library has been updated including the new controls that can be assessed and designed for the mitigation of the risks of new cyber threats, the next activity of the threat analysis stage consist of "*TA 4.4 – Correlate threat agents to*

assets/targets." Correlation of the threats with the assets such as the various components of the application architecture is also referred to as threat enumeration in threat modeling.

Threat enumeration focuses on correlating threats to security controls to determine if these threats are either mitigated by the presence of countermeasures or are not mitigated because countermeasures are either missing or have design flaws or vulnerabilities that expose the application assets. From the perspective of application threat modeling, the specific mapping of threats to architectural components of the application architecture including data in storage and in transit is important to be able to visualize how these threats map to the application logical and physical architecture. Specifically it is important to correlate these threats with the assets that in the case of online banking applications include the sensitive data of customers such as bank and credit card data, PII, and application's business critical functionality that can be abused for financial gain. The focus of the threat analysis and correlation with the application assets is called out in the application threat modeling process as specific activity.

For what concerns the data, since data is both in storage and transit, it is important to map these threats not just to the data storages such as databases but also to the data in transit across the different components of the application that includes the different servers allocated in the different tiers of the application architecture shown in Figure 8.8. In the threat modeling example, there are shown the threats enumerated from a small list using STRIDE (e.g. spoofing, tampering, repudiation, information disclosure, denial of service and elevation of privileges) affecting some specific components such as user/browsers and authentication credential store that is considered an high-valued target by the attackers (Figure 8.17).

A STRIDE- based security control risk framework can also be used to assert if the application is designed to include security controls that specifically reduce and control the risks of these threats as shown in Table 8.9.

Another view that can be used for the threat to asset modeling assessment is a defensive view that consist of looking at each of the application security controls from the lenses of an Application Security Framework (ASF) and then determine if any weaknesses and vulnerabilities in the design and implementation of these application security controls expose the application to specific threats as shown in Table 8.10.

Both the STRIDE attacker view and the ASF defensive view for the enumeration of threats are essential for an in-depth analysis of the threats affecting the application data and functional assets.

Threat modeling tools can incorporate these threat and control framework and facilitate the analysis of the threats against the assets such as both the data and the functional components of each basic element of the application. An example of functional level decomposition in support to the analysis of threats against data and "widget" of functionality associated to each component.

The correlations of threats to assets such as data and functional assets lead to the final step of the threat analysis that is *TA 4.6 – Assignment of the probabilistic values for identified threats* that is the estimate of the level of risk for each of the threat being analyzed and modeled. The level of risk for each threat used in the analysis

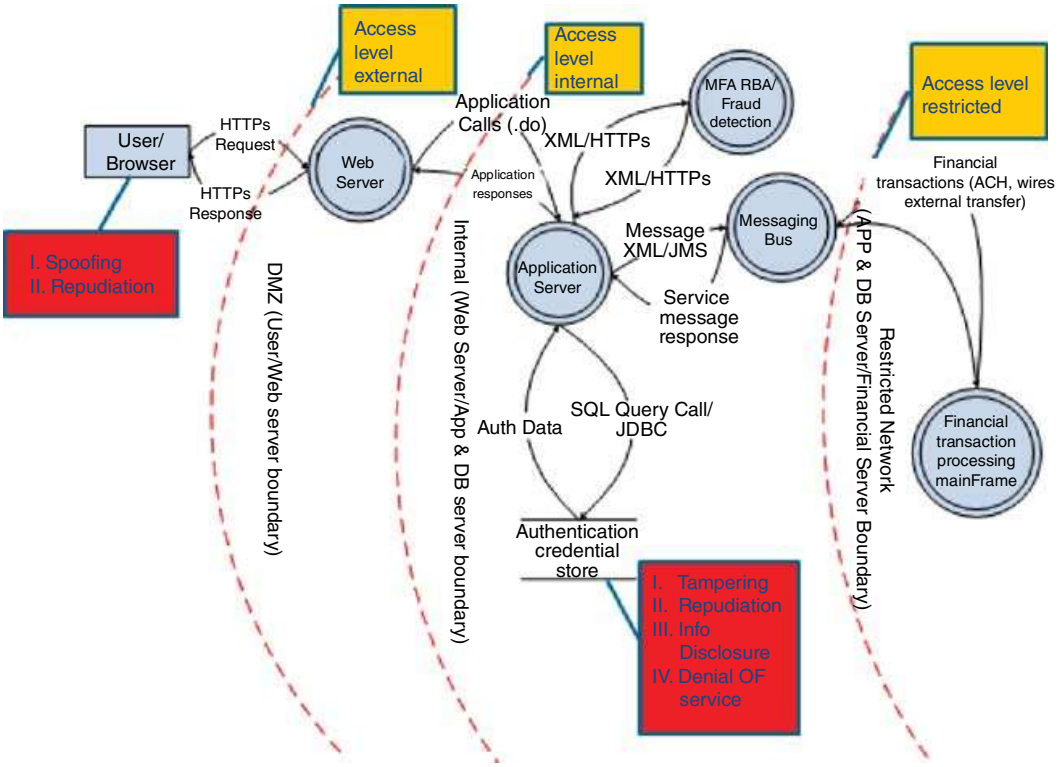


Figure 8.17 Threat Model Using STRIDE per Element

TABLE 8.9 STRIDE Threat List

STRIDE Threat List		
Type	Examples	Security Control
Spoofing	Threat action aimed to illegally access and use another user's credentials, such as username and password	Authentication
Tampering	Threat action aimed to maliciously change/modify persistent data, such as persistent data in a database, and the alteration of data in transit between two computers over an open network, such as the interest	Integrity
Repudiation	Threat action aimed to perform illegal operation in a system that lacks the ability to trace the prohibited operations.	Non-Repudiation
Information disclosure	Threat action to read a file that they were not granted success to, or to read data in transit.	Con fidentiality
Denial of service	Threat action to deny access to valid users such as by making a web server temporarily unavailable or unusable.	Availability
Elevation of privilege	Threat aimed to gain privileged access to resources for gaining unauthorized success to information or to compromise a system.	Authorization

TABLE 8.10 Application Security Frame

Type	Description	Attack Examples
Auditing and logging	Threats caused by failure to maintain detailed and accurate application logs that can allow for traceability and nonrepudiation and provide enough information for administrators to identify security issues and for incident response specialists to trace an attack. An attacker can use logs to obtain critical information about the system as well as tamper them to clear his traces after an attack.	1. Non repudiation 2. Cleaning the logs to remove evidence 3. Inserting of faked logging entries
Authentication	Threats caused by lack of strong protocols to validate the identity of a user to access a system or component outside the trust boundary. An attacker can obtain illegitimate access to the system or its individual components.	1. Impersonation 2. Spoofing attacks 3. Brute force attacks 4. Dictionary attacks 5. Token/Cookie Reply attacks 6. Man in The Middle attacks(MiTM)
Authorization	Threats caused by a lack of mechanisms to enforce access control on protected resources within the system. An attacker can get access to resources that should not have privileges to access to.	1. Escalation of privileges 2. Luring attacks 3. Forceful browsing
Configuration management	Threats caused by insecure deployment and administration. An attacker can get access to system/user data and gain unauthorized access to system functionality	1. Elevation of privileges 2. Unauthorized access to configuration data 3. Unauthorized access to administration interfaces
Data protection in storage and transit	Threats caused by the implementation of encryption and lack of adequate protection for secrets and other data in storage or transit. An attacker can compromise user/system confidentiality and data integrity by bypassing the cryptographic mechanisms used by the system	1. Access to sensitive data in storage 2. Access to secrets (e.g. encryption keys) 3. Eavesdropping of data during transmission 4. Data tampering 5. Brute force attacks to crack encryption

can be evaluated based upon risk factors for the threats such as the probability for the threat to be realized in an attack and conduct an exploit and cause an incident such as compromise of sensitive data and fraud.

This estimate of the risk for each threat at this stage is critical for deciding the course of action to mitigate the risk of the threat. For example, if the threat analysis is conducted to review the impact of new application functionality as well as changes to data that might increase the risk, the analysis of threats at this stage can drive risk management decision at design level such as designing a threat resilient security control that protects the data assets and functionality or otherwise the design of the application must change to remove the threat entirely (e.g. do not store credit card numbers to remove the threat of disclosure). Once a risk severity is assigned to each threat, it can be prioritized to discuss with the business and risk management how the risk of threats should be managed (Figure 8.18).

For the calculation of the risk of threats, a threat analyst can calculate the risk of each threat by using risk calculation formulas. These formulas are based on the assignment of the level of risk, HIGH, MEDIUM, and LOW, to the different factors of risks such as likelihood and impact. The factors of probability of a threat occurring depend on the degree of mitigation of the threat (e.g. the presence or not of a vulnerability) and associated factors for the determination of the likelihood for an attacker to exploit the vulnerability in an attack.

For a given attack, the possible exploit of a vulnerability depends on several factors such as being able to discover the vulnerability, conduct the exploit, and reproduce the exploit in other attacks. From technical impact perspective, the impact can also be further analyzed by assigning values to the damage potential of the threat and the number of affected components.

This analysis of the risk of a threat initially ought to make assumption on the exploitability of a vulnerability that is the realization of a threat in an attack and the presence of vulnerabilities that at this stage can only be roughly estimated since vulnerabilities and attacks are evaluated only during STAGE V Weaknesses and Vulnerability Analysis and STAGE VI Attack Modeling that are essential for the final quantification of risk is STAGE VII Risk Analysis and Management.

At the end of the threat analysis stage, the threat analyst will assign a level of risk to the threat and later revise it higher based upon the analysis of vulnerabilities that map to that threat and the model of an attack that could exploit that vulnerability as shown in Figure 8.19.

Stage V – Weaknesses and Vulnerability Analysis (WVA)

Goals of This Stage The analysis of weaknesses and vulnerabilities and correlation of these with the threats previously analyzed. Any weaknesses and vulnerabilities in the application architecture are also identified during this stage and prioritized for validations/tests along with any weaknesses and vulnerabilities of the application/IT system in scope that were previously analyzed with other assessments.

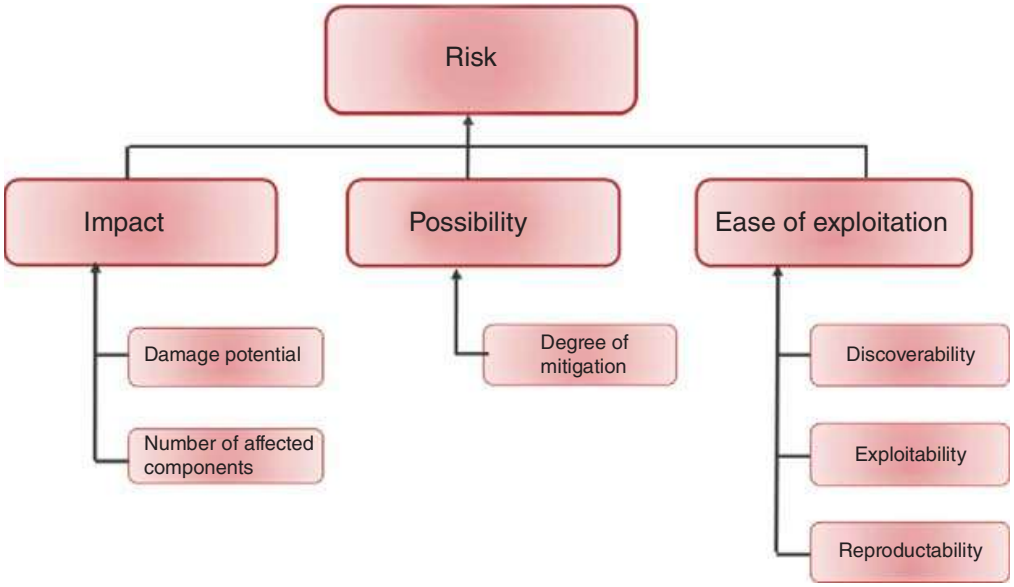


Figure 8.18 Threat Risk Factors

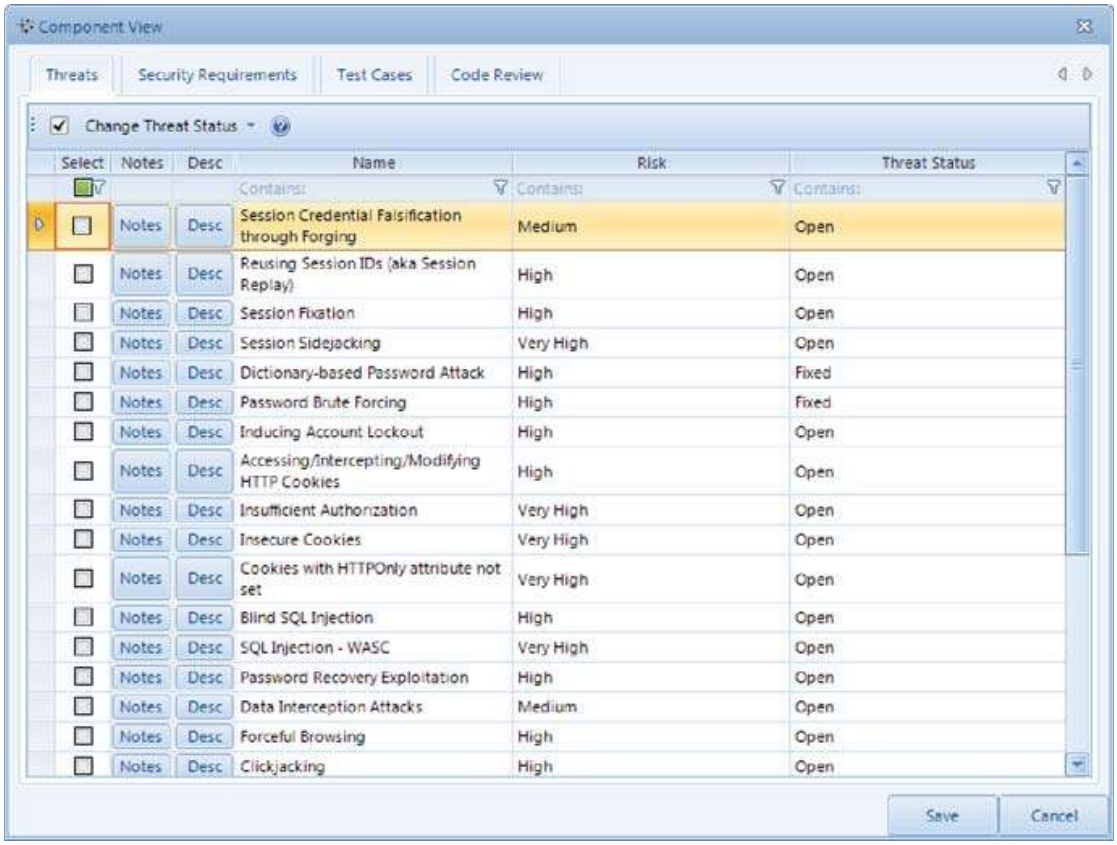


Figure 8.19 Threat Dashboard with Threat Risk and Status

Guidance for Conducting This Stage During this stage, we will reassess any existing vulnerabilities and weaknesses of the several components of the technology stack including software components, servers, and services supporting the application in scope along with any design flaws identified by conducting an architectural risk analysis of the application components. These weaknesses and vulnerabilities will be correlated with information of the threats targeting them as threat target and prioritized for security testing based upon the information of "ThreatTargets," "Observables," and "Indication of Compromise" analyzed from threat reports received from internal and external threat sources. The information about the stage of the attack such as the attack kill-chain stage included in the observable can also be used to prioritize the security tests along with the severity of the vulnerability that was previously tested. The information about the threat agent's TTPs Tools, techniques, and processes that has been observed can also be used for creating specific test cases for these vulnerabilities that are more realistically representative of the threat scenarios.

During this stage, vulnerability reports from different security assessments that were previously executed on the web application such as application dynamic security tests and static application security tests are considered for the analysis. The information about weaknesses and vulnerabilities found in the application constitute the knowledge base of vulnerabilities that is considered for this assessment as well as any design flaws that could be identified during the security review of the application and software architecture.

Previously identified application vulnerabilities can be stored in a vulnerability management repository/knowledge base so can be categorized by application assets. Such vulnerability management repository/knowledge base should be made available for querying based upon the specific assets in scope for the risk analysis identified during stage II and decomposed during stage III. These application and system vulnerability assessments are typically performed in compliance with standards and processes such as security architecture reviews, secure code reviews, manual application security tests, and automatic dynamic and static code analysis scans. The assumption of this stage is that application vulnerabilities can be mapped to impact to each of the web application assets that include source code, architectural components, and data potentially targeted by the threats previously analyzed.

Any gaps in the security controls and weaknesses in the design of these security controls (e.g. design flaws) identified during the application architectural analysis in this stage are also correlated to threats during this stage. At the end of this stage, we would have a mapping of threats to vulnerabilities and control weaknesses (design flaws) of the application, software, and systems in scope to ascertain whether security control weaknesses and application and system vulnerabilities exist and might expose the application assets (data and functions) to attack from threat agents and what their probability is to be targeted by threat agents.

Based upon the mapping of threats to vulnerabilities, it is possible to conduct a more accurate scoring of vulnerability severity. Among the factors that can be used for scoring of risk for vulnerabilities, a threat analyst can use the ease of discovery of the vulnerability, the exposure of the vulnerability to specific threats and the fact that events to exploit the vulnerability are logged and audited. The factors that can

be assessed at this stage also include technical impact factors such as impact on data confidentiality, integrity, and availability in case these vulnerabilities are exploited by a specific threat. The calculation of the risk severity of vulnerabilities in presence of threats can use standard vulnerability severity risk scoring methods such as CVSS. Finally the vulnerabilities and threat that have the highest probability and technical impact that can be assessed during this stage can be put in scope for additional security tests to validate the likelihood of exploit and determine the technical impact.

Activities This stage consists of the following activities:

WVA 5.1 – Review/correlate existing vulnerabilities: This activity consists of the review of existing application vulnerabilities affecting security controls that protect assets such as data and functions that are targeted by the threat agents that were previously analyzed. Typically, each organization should have a vulnerability management repository that is a repository that stores vulnerability data from different security tests previously executed on the application/system such as architectural risk analysis, source code analysis, and penetration testing. During this activity, the threat analysis will be querying vulnerabilities previously identified for each web application and select vulnerabilities in security controls that expose specific data and the functional assets such as authentication data, confidential data, and various type of functionality such as user authentication, change of authentication credentials, read access to confidential data, change of confidential data, and others.

WVA 5.2 – Identify weak design patterns in the architecture: After the design of the application has been decomposed in basic architecture elements including the type of data that these components process as data in transit through other components and as data stored, it is possible to analyze the scope for the security control and to assert that security requirements are followed for the secure design of security controls, application architecture, and functionality. Security control gaps to protect the data assets can be identified by following an architectural review process that enumerates the security control to review and provide guidelines to review how these controls should be designed by following security architecture principles and documented security requirements, guidelines, and standards. Security control frameworks can also be used to assess the security strength of the control and commensurate it against the inherent risk of the data and functional asset that the control is required to protect by providing either a detective or protective function. If a design flaw is identified, this should be reported as finding and an initial risk level should be assigned at this point. This risk of such design flaws also depends on the probability of a specific threat agent to exploit that design flaw. Such risk can be initially assigned as this stage by grouping the design flaws using standard weaknesses enumeration methods (CWE) and by using standard risk scoring methods for calculating the risk of the vulnerabilities such as First CVSSvs2. In calculating the risk for design flaws with these methods, it is important to consider the threat agents that might potentially exploit these weaknesses and vulnerabilities and look

whether threat intelligence provides information of these threat agents seeking to target these vulnerabilities as a factor of probability of a threat. This activity builds on the previous stages of stage III application decomposition and specifically enumeration of the assets, creation of DFDs, and stage IV threat analysis activities such as mapping of threats to the assets and the probabilistic value assigned to these threats during stage IV.

Note that the risk that is assigned to the design flaw at this stage needs to be revised later on during the attack modeling stage after attack simulation and tests are performed to determine the likelihood and impact of threat agents and attack vectors used by these threat agents in the exploitation of these design flaws.

WVA 5.3 – Map threats to vulnerabilities: This activity will consider each threat of the threat library that is already mapped to a security control identified in stage IV to determine if these controls have vulnerabilities or design flaws that might increase the exposure to each threat. This activity consists of mapping threats to vulnerabilities and architectural design flaws (e.g. control gaps/ weak design of security controls). Design flaws that are previously identified during the application decomposition analysis and assertion should also be mapped to threats.

WVA 5.4 – Provide Context risk Analysis based upon Threat-Vulnerability: With this activity, we will review weaknesses and vulnerabilities in the context of threats. The goal of this activity is to determine the risk severity of weaknesses and vulnerabilities by considering the risk posed by exposure to threats in the context of the overall threat scenarios. Standard factor for calculation of risks includes probability and impact; furthermore, probability factors can be calculated based upon the assignment of values for ease of discovery, ease to conduct the exploit, awareness of the exploit, detection and prevention of exploit of the vulnerability by detective controls. Impact factors are technical impacts on the assets when the vulnerability is exploited such as impact of confidentiality, integrity, and availability of the asset. The risk calculation will use standard grouping for vulnerabilities such as CWE and standard risk scoring methods such as CVSS.

WVA 5.4 – Conduct targeted vulnerability testing: The goal of this activity is to create a suite of security tests for specific weaknesses and vulnerabilities targeted by threat agents. These security tests can be executed using different types of security tests including manual security tests of vulnerabilities, black-box and white-box tests, static and dynamic source code scanning, and manual code reviews. These security tests are no longer blind of threat agents but consider the type of threat agents and attack tools used by these threat agents as these were to exploit the vulnerability in a real attack against the application. The purpose of these tests is to assert the exposure and the risk as technical impact on the assets. For example, this can be a case of testing a known SQL injection vulnerability with specific attack vectors used by the threat agents.

Note: The goal of these tests is not to identify if these vulnerabilities can be exploited (e.g. exploitability tests) but to determine a more accurate value of risk severity of the vulnerability based upon known exposure to threat agents. Seldom by using the same attack vectors used by the threat agents that can be discovered by the analysis of threat targets, additional weaknesses, and vulnerabilities can be identified. This is the case for example when design flaws are identified during the review of the architecture design and the validation of these design flaws by testing also identifies additional instances of similar type of vulnerabilities caused by implementation/coding as well as insecure configuration.

Inputs Used for Conducting This Stage

1. Documented application security risk profile for the assignment of risk to vulnerabilities from stage I.
2. Technical documentation in scope from stage II.
3. Application architecture documents detailing users, components, functions, data assets analyzed in stage III.
4. Application assets (data and functions) targeted by threat agents analyzed in stage IV.
5. Vulnerability reports from previous vulnerability assessments (ethical hacking, source code analysis, secure code review).
6. Documentation from the previous analysis of threats in stage IV including:
   a. Application security controls weaknesses (CWEs) and application, software, and system vulnerabilities (CVEs) targeted by threat agents/actors as Threat-Targets that are part of the threat scenario previously analyzed.
   b. Information from threat intelligence (e.g. threat reports) about the threat TTPs that can be used to prioritize the security tests of the vulnerabilities targeted.
   c. Information from threat intelligence (e.g. threat reports) about the stage of the kill-chain of the attack in which these threats have been observed as threat event as well as IOC indication of compromise.

Artifacts Produced at the End of This Stage At the end of this stage, we will produce the following artifacts:

(a) List of application vulnerabilities and weaknesses affecting the architectural components of the application mapped to threats
(b) Risk severity of the vulnerabilities calculated using standard risk scoring methods (using CVSS) based upon the information of the "threat illumination" of the vulnerability by a threat as (exposure to threats as opportunity and target)
(c) List of identified design flaws from architecture risk analysis
(d) Prioritized security test cases for retesting the presence of vulnerabilities that are targeted by various threats actors

Tools The execution of this stage can be facilitated by the use of vulnerability management repositories such as SVGI Vulnerability Repository, Source: Sourceforge.net Standard vulnerability risk scoring methods (CVSSvs2) to score risks of vulnerabilities. Formal methods to map threats to vulnerabilities and countermeasures such as OWASP threat trees are used for this stage. Vulnerability reports from OWASP (e.g. OWASP Top 10) and WASC as well as MITER CWEs as well as vendors sources of OSINT for threats and vulnerabilities exploited by these threats are used herein. Security test libraries for specific functional components embedded in threat modeling tools such as myAppSecurity ThreatModeler™ are also shown herein to create risk-based test cases for web application vulnerabilities.

Web Application Use Case Example The goals of this stage of application threat modeling exercise are to query existing weaknesses and vulnerabilities, identify any design flaws in the architecture design, and determine the risk, map threats to vulnerabilities and design flaws identified, and prioritize security tests for validation of design flaws and weaknesses and vulnerabilities that were previously identified and now can be reassessed and retested based upon the information of threats targeting them and tools, techniques, and procedures used.

During this stage, we will focus on both weaknesses and vulnerabilities previously tested, as well as the vulnerabilities that are identified during the review of the architecture and need to be validated and tested with security tests. We will first query any known weaknesses and vulnerabilities for the online banking application that was in scope and identify the type application security component that was affected and the assets impacted (data and functionality). We will then proceed to map known threats to these vulnerabilities and the security controls affected including new design flaws identified in this stage and calculate the risk severity of these vulnerabilities based upon exposure to the threat agents. Finally we will prioritize the security tests to test these design flaws and vulnerabilities.

The first activity of this stage consists of the *WVA 5.1 – Review/correlate existing vulnerabilities*. The scope of this activity is the analysis of existing weaknesses and vulnerabilities that were previously identified in performed security assessments on the online banking application such as secure architecture design reviews, source code analysis, and ethical hacking penetration tests of the application. After these weaknesses and vulnerabilities are reported to the business and discussed with application development teams for remediation, they are typically stored in the vulnerability risk management repository so can be queried for each application to track their resolution within the compliance remediation time frames. These vulnerabilities can also be queried for later analysis during this stage of the threat modeling exercise, specifically to correlate them to specific threats and assign a risk score for the severity by considering the probability of threat agents seeking to exploit them. The severity of the vulnerabilities can be calculated using standard risk scoring methods such as CVSS that use factors of probability and impact to determine the impact of threat agents on the assets such as their confidentiality, integrity, and availability.

The previously conducted threat analysis and specifically the information gathered from threat intelligence provide data for the calculation of the factors of probability

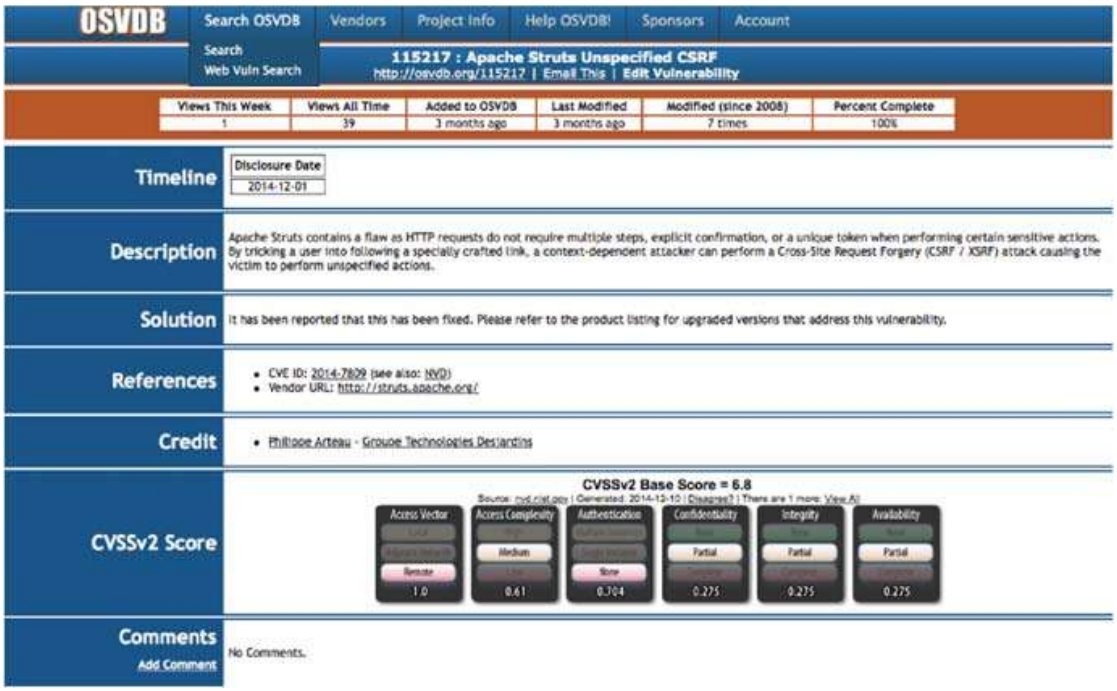


Figure 8.20 OSVDB Open Source Vulnerability Database source http://www.osvdb.org[3]

and impact. For example, the information that application vulnerabilities are currently being exploited by specific types of attacking tools or malware that is used to target online banking sites (e.g. banking malware Trojans) increases the probability of a potential exploit and the risk for the target. The exposure of the vulnerability to specific threat agents and the fact that the exploitation of these trigger events that are monitored and logged for potential Indication Of Compromise (IOCs) decreases the probability of a potential impact.

The analysis of vulnerabilities based upon the information of threat agents and the countermeasures in place is also critical for the derivation of security tests to validate the effectiveness of the existing countermeasures in the mitigation of the risk of threat agents seeking to exploit these known vulnerabilities.

Let us start with the first activity, that is, query existing vulnerabilities in the application. The goal of this activity is to query existing application vulnerabilities by the inventory ID of the inventory of the web application and IT system owned by the organization. Typically, each organization that has rolled out vulnerability assessment processes in compliance with information security standards should have a vulnerability management repository that stores vulnerability data from different security tests previously executed on the application such as architectural risk analysis, source code analysis, and penetration testing.

An example of query for vulnerable application software using the NVD (National Vulnerability Database) as a source using the Open-Source application SVGI's vulnerability repository is shown in Figure 8.20. Through this tool, it is possible to

[3]Sourceforge.net.

search for software vulnerabilities based upon the application software used by the application and determine the type of vulnerability grouped according to MITRE CWE and the risk scoring based upon CVSS.

Similar types of repositories can also be used to aggregate vulnerabilities from internal application vulnerability assessment and organized by a repository of IT assets that also includes applications besides IT systems. After we have queried existing vulnerabilities and weaknesses, the next step of this stage is to perform the activity "*WVA 5.2 – Identify weak design patterns in the application architecture.*" The goal of this activity is to analyze the architectural components of the application architecture in scope and identify design flaws that might expose application assets (data and functional) at risk and need to be included in prioritized security tests for validations. Architectural design flaws consist of either gaps or weaknesses in security controls that might expose the data and functional assets to potential impacts such as the loss of confidentiality, integrity, and availability of confidential data and business critical functionality.

This activity can be executed at this stage since we have defined the security functional requirements for the application during stage I and defined the application technical scope in stage II including the technology stack of the various components, servers, and services and network components that support the application functionality residing in the hosts and servers located at different tiers of the application architecture (e.g. presentation tier, business tier, and database tier). In stage III, we also have decomposed the application in data and functional assets and we have analyzed trust boundaries and security controls deployed within these trust boundaries to protect sensitive data assets and business critical functions. In stage IV, we have conducted a threat analysis and we have updated the threat library with the information about threats learned from threat intelligence and mapped these threats to the assets. In stage V, we are not ready to identify any design flaws in the architecture that might be exploited by these threats with knowledge of their targeting specific weaknesses and vulnerabilities.

Since the objective of this activity is to analyze the architecture of the application to identify design flaws in security controls, we can start by reviewing any security control that is required to protect highly sensitive data (e.g. security keys, passwords) in storage and in transit. The previously documented DFDs DFDs may be useful to conduct this analysis and any potential design flaw that might expose DFD components (e.g. user client, data assets, processes located in servers in the different trust boundaries) and exposed to a threat agent and threat vector used by the threat agent represents a potential exposure and a security impact for the data assets that these security controls are designed to protect.

The assertion of the design of security controls for protecting the data in storage and in transit as well as the access to the various components of the application architecture can be done by following the flow of data as depicted in the DFD as well as by asserting the security requirement for each functional component. In the case of the online banking application, it can also be done by looking at the data assets in transit and in storage as shown in DFDs and by walking-through each architectural component of the DFD that is traversed by the data in transit. By starting the

data flow from the external user interaction with the online banking application, for example, we can look at the data end to end from the user browser to the validation in the authentication credential store. The requirements for security controls such as authentication, session management, input validation, audit and logging, and encryption can be asserted at the level of the component that either process authentication data while in transit or in storage can be asserted by walking through the data flow.

Examples of design flaws that might be identified during this stage include design gaps in security controls that use weak architectural design patterns to protect sensitive data and critical business functionality.

The secure design of these security controls can be reviewed by asserting the security requirements for the design of each security component. If the design misses a requirement for the design of a security control such as access controls for accessing the confidential data, this will potentially expose data and functionality to potential threats. By following the data flow for other type of data such as, for example, data used in financial transactions, it is also possible to identify if there are design flaws in enforcing mutual authentication between the application server and web service calls exposed by the messaging bus as well as enforcement of secondary authentication and session management before allowing the data to cross the trust boundary with the financial processing mainframe.

When this activity is conducted during the SDLC such as in the case of development of a new online banking application or to assess the impact of design changes introduced in an existing online banking application, the assessment of the protection of data that is either stored or processed by each component can be done by asserting the inclusion of specific security requirements for the design of security controls to protect the data in storage, in process, in transit, and on display to users. The lack of design of a security control to meet these requirements is a finding that can be fixed during the design phase of the SDLC by requiring the development teams to include it in the architecture design documents. If these design flaws are identified during the SDLC these can be remediated based upon the analysis of the potential technical impact and likelihood of exploit. The assessment of the risk of the design flaw in the implementation of a security control will be done by analyzing the likelihood of threats and the technical impact in case such threats will exploit these design flaws to impact either the application data or the functional assets.

The architectural analysis of the security components can be done following the decomposition of the architecture of the application by analyzing the design of the security controls and by enumerating a set of requirements to assert that the confidentiality, integrity, and availability of each data component are protected by a secure design of a security control. In the case when security control is either missing in design or cannot be asserted by the documentation, available specific tests should be documented to validate that no security control gaps or vulnerabilities are present.

Architecture analysis reviews can have either positive or negative assertions. A positive assertion consists of identifying the security control as documented and designed by following the security requirement. A negative or failed assertion represents the instance where a security control cannot be asserted in the design because is missing or it is designed without following security requirements and

security standards and therefore introduces a design flaw. An example could be, for example, encrypting the authentication data using a nonstandard encryption algorithm or not using standard secure protected storage for keys.

An example of security architecture design guidelines that can be used to review the design of application security controls is shown in Table 8.11.

The assessment of security controls in the application architecture can be done by embedding the security requirements previously documented and security control frameworks such as STRIDE and ASF in threat modeling tools. By using the ThreatModeler™ tool, for example, it is possible to conduct an architectural risk analysis by reviewing the various security controls of the application and conduct an in-depth design review of each security control. An example of this approach using the ThreatModeler™ tool is included in Figure 8.21.

Once the security control to review is selected (e.g. authorization), a more in-depth security risk control review can be performed by asserting the requirement of the control as this requirement has been reviewed and asserted against the application security design guidelines previously shown. The tracking of the review of application security controls at architectural and functional level can be done by enumerating each security design requirement for this control using a threat modeling tool as shown in Figure 8.22.

The functional decomposition of the application can also be used to assert security requirements for protecting both the data and the functionality associated to each functional component. For example, a login component that allows a user to authenticate and access the wires fund transfer functional component would need to assert the implementation of multifactor authentication during login. The functional decomposition can provide a better assertion of security requirements for protecting data for specific business process implemented by the online banking application such as requests for bank statements, opening bank accounts, money transfers, and payments.

These design flaws can be identified by using the secure functional analysis for business critical functions of the online banking application that was conducted in stage III. Using the results of this analysis, it is possible to assert that security controls implemented protect each of the application business critical functions, as these are functional assets and as data assets have a requirement for the protection of confidentiality, integrity, and availability risks. At high level, this type of analysis allows to analyze the risks of each functional asset based upon the information of the threat targeting it and identify weaknesses in security controls (e.g. MFA that might be bypassed by MiTM attacks) and should be remediated by introducing changes in the design of the application.

After we have identified design flaws and tracked for validation with security testing including the other types of security controls, weaknesses and vulnerabilities that were identified in previous security tests the next step of this stage consist of correlating each of these design flaws, security control weaknesses and vulnerabilities with the threats that were previously analyzed. This is the activity *WVA 5.3- Mapping threat to vulnerabilities.* This activity leverage the results of the threat analysis performed during stage IV and the type of vulnerabilities such as CWE and CVE that are known to be exploited by threat agents of the threat scenarios being analyzed.

TABLE 8.11 Secure Architecture Design Guidelines

Security Control	Secure Architecture Design Requirements
Authentication	Authentication strength should be commiserated to the risk of the data/functionality being accessed; Internal and external connections (user and entity) should be authenticated; Non-repudiation of a connection should be enforced through IP filtering or mutual authentication; Credentials and authentication tokens should be protected with encryption in storage and transit; Protocols used for authentication should be resistant to brute force, dictionary, and session replay attacks; Strong password policies should be enforced; Logging errors should not reveal password hints and valid usernamesAccount lockouts should not result in a denial of service for users
Authorization	Authorization mechanisms should work properly, fail securely, and cannot be circumvented Authorization should be checked on every request and cannot be bypassedServer side; Role-Based Access Controls (RBAC) should be used to restrict access to specific operations The principle of least privilege is used for users and service accounts.Privilege separation is correctly configured within the presentation; business and data access layers Permissions such as Access Control Lists (ACLs) are used for enforcing authorized access to resources
Encryption	No credentials and sensitive data are sent in clear text over the wireSensitive data is encrypted in storage and in transitStandard encryption algorithms and minimum key sizes are being usedDigital signatures are used to protect data integrityPrivate keys are cryptographically protected by either secure file storage or hardware encryption modules
Session	Authentication tokens are sent encrypted over SSLUser is forced to reauthenticate when performing critical functionsAll open sessions are expired at logout and at timeoutSession tokens cannot be replayedSession tokens are changed after authenticationHTTP cookies are set with the secure flag
Input validation	All data types, format, length, and range checks are enforcedAll data sent from the client is validated by the server before processingNo security decision is based upon parameters (e.g. URL parameters) that can be manipulatedInput filtering via white list validation is used to filter malicious dataOutput encoding is used
Error Handling/ Information leakage	All exceptions are properly handled in a structured mannerPrivileges are restored to the appropriate level in case of errors and exceptionsNo system error messages (e.g. memory stack) are returned to the userError messages are scrubbed so that no sensitive information is revealed to the attackerThe application fails in a secure mannerResources are released if an error occurs
Logging/ Auditing	No sensitive information is loggedSecurity events (e.g. authentication, password changes) are loggedAccess controls are enforced on log files to prevent unauthorized accessIntegrity controls are enforced on log files to provide nonrepudiationLog files provide for audit trail for sensitive operations and logging of key eventsAuditing and logging is enabled across the tiers on multiple servers



Figure 8.21 Architectural Risk Analysis Component of ThreatModeler™

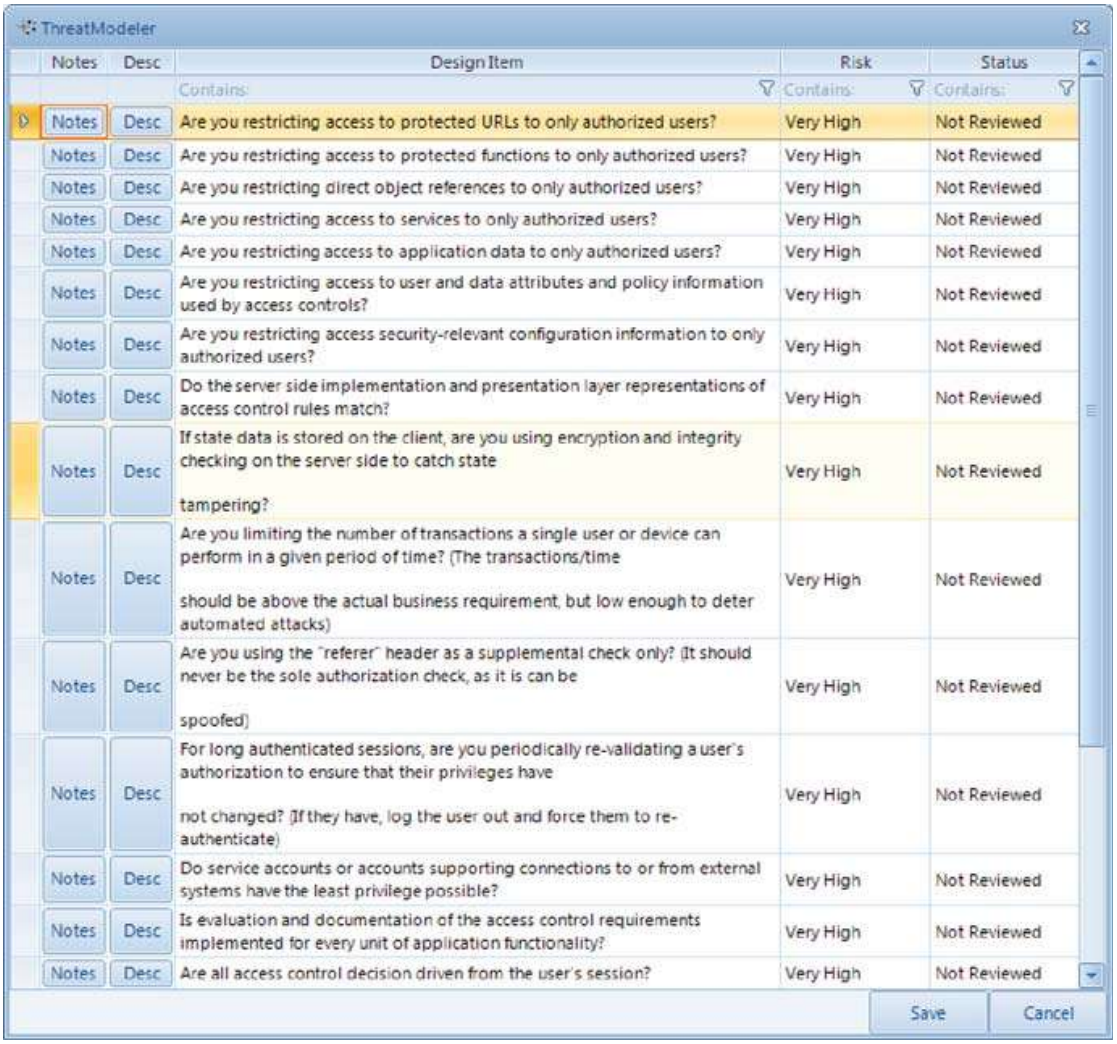


Figure 8.22 Architectural Risk Analysis of Authorization Controls

Specifically the map of threats to vulnerabilities when is done using the STIX threat language consists of mapping the "Threattarget" attribute with CWEs and CVEs that are known from threat intelligence to be exploited by this threat agent.

Without a mapping of threat to vulnerability that comes from a threat intelligence feed using STIX in which the "threattarget" attribute is populated with the types of CWEs that are known to be exploited by the threat, the threat analyst need to recur to formal methods and conduct this mapping to the best of the knowledge of known application vulnerabilities as these are reported in vulnerability assessments and known threat agents and attacks that seek to exploit them.

One of the formal methods that analysts can use for the mapping of threats to vulnerabilities is the threat tree, an example of, which it is shown in Figure 8.23. In the example shown, a threat agent, the attacker whose goal is to read other user's messages will try to exploit different type of vulnerabilities to reach his goal. Examples of vulnerabilities that the threat agent might seek to exploit includes an SQL injection vulnerability, a design flaw in authorization, a cached page, or file stored on the desktop that can be retrieved by accessing the unattended desktop or even a nonvulnerability such as careless employee who would leave his desktop unattended without logging out hence accessing the logged authenticated session of that user to get his data. The walking of a threat tree from the attacker goals to the vulnerabilities that can be exploited to reach such goals is also useful to determine the course of action (the end of the the branch of the threat tree) to mitigate the risk of these vulnerabilities.

The deliverable of this activity consists of a list of vulnerabilities at the application and system level and their level of exposure to the threats. Since these vulnerabilities affect the different IT assets of the application such as the several architectural components of the web application such as web and application servers and databases as well as the servers and application software on which these applications run and the network where these servers are hosted, it is also important to correlate these vulnerabilities for each IT application assets.

Since this stage builds on top of the previous stages and specifically the assessment of the technical scope for the IT assets in stage II, the decomposition of the application in these IT assets at architectural and technology stack level in stage III including any design flaws identified during the architectural analysis in that stage as well as the threat analysis profile with threat list for web application assets (data and functions) from stage IV, it is possible at this stage also to visually correlate these vulnerabilities with the different components of the application architecture as well as the threats that seek to exploit these vulnerabilities as shown in Figure 8.24.

In the DFD of Figure 8.24, the different components of the online banking application are shown including the flow of the data, the different process/components of the application architecture and the different trust zones/boundaries for accessing the data in and out from these components. Threats such as data compromise, data theft, online fraud, abuse of functionality, abuse of privileges, and denial of services are shown as external threat agent threats.

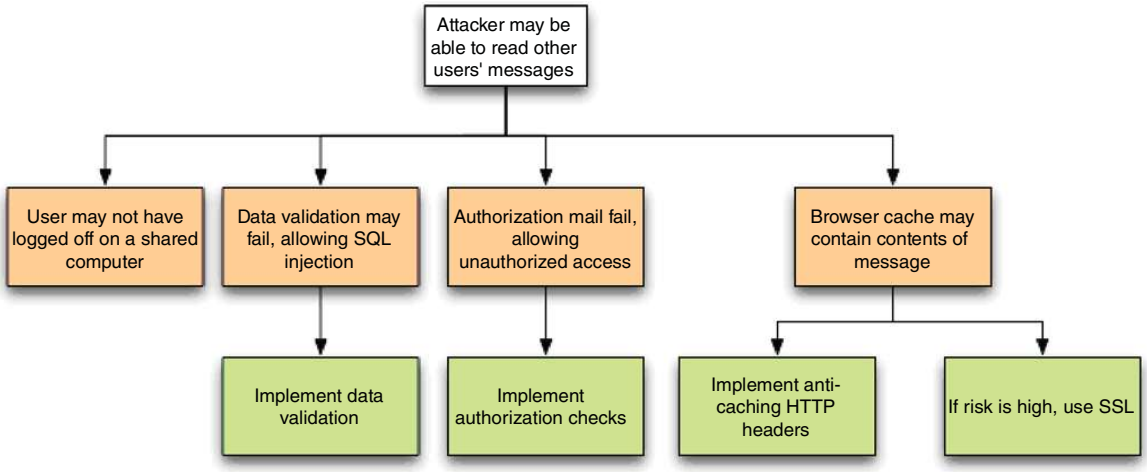


Figure 8.23 Threat Tree (Source OWASP)

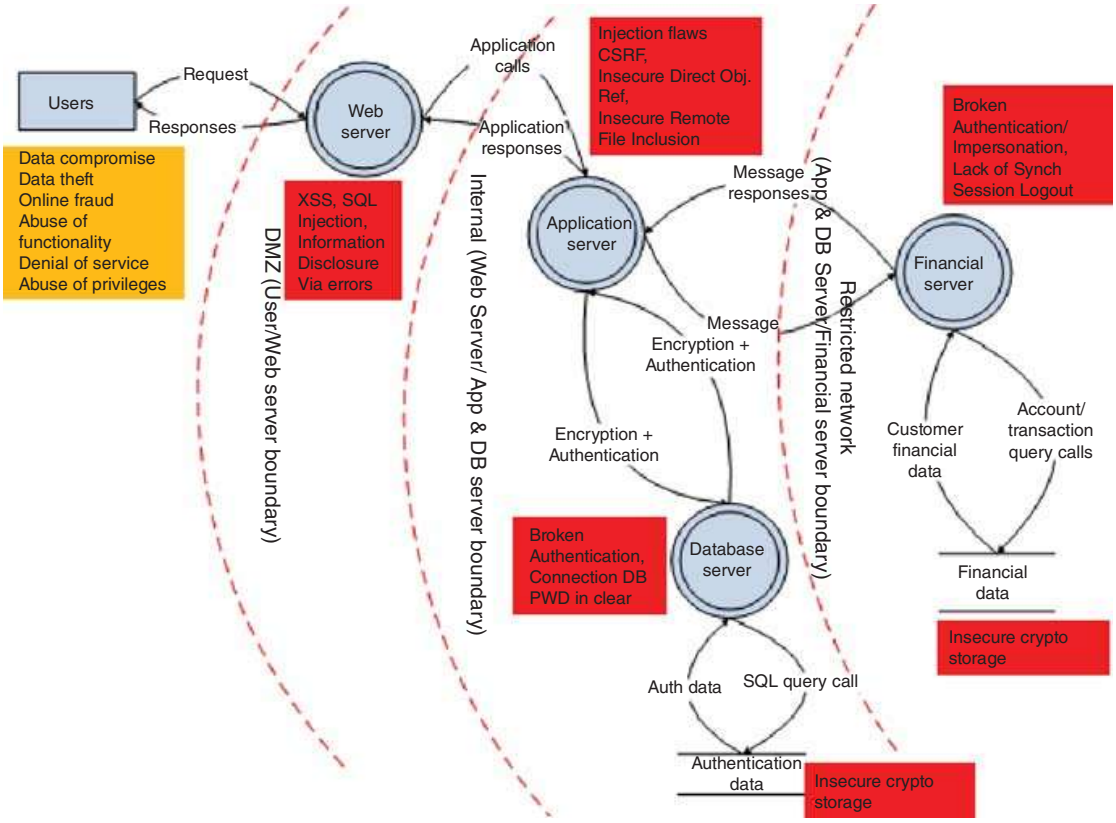


Figure 8.24 Mapping of Threats with Vulnerabilities of Different Application IT Assets

The vulnerabilities that these external threat agents seek to exploit to realize these threats are marked for each component. A threat agent can exploit one of more vulnerability at different layers of the application architecture for the threat agent to be capable of reaching his goal. For example, data compromise and theft might occur because of an exploit of SQL injection vulnerability at the web server and insecure direct object reference at the application server to retrieve financial records for another user or either weak authentication (e.g. passwords in clear or guessable) or insecure session management (e.g. authentication session for accessing the financial server is not in synch and not timing out) at the financial server allowing the external threat agent to login to the database where financial data are stored.

This correlation of threat agents with vulnerability exploits that can be pursued at the different layers of the application architecture is important to determine which vulnerabilities should be prioritized for remediation based upon the risk posed by a threat agent exploiting them. This mapping also allows application security teams to look at the protection of the application from the perspective of trust boundary layers that can breach and decide where and how to apply defense in depth for protecting from these threats. Once the application weaknesses and vulnerabilities are correlated with the application assets and the threat agents, the threat analysis has finally gathered the overall picture of the application threats and vulnerabilities and is ready for the next step, that is, to calculate and assign a risk severity to these vulnerabilities.

This is the objective of the activity *WVA 5.4 "Provide Context risk Analysis based upon Threat-Vulnerability."*

The scoring of the vulnerabilities need to consider both the risk severity of the vulnerabilities that was previously estimated as blinded to threats, that is, without factoring the instance of a threat seeking to exploit the vulnerability in the risk calculation and the risk posed by exposure to known threats.

To assign the risk severity to vulnerabilities, standard accepted risk scoring methods can be used such as CVSS. These methods allow a threat analyst to calculate the score of a vulnerability based upon the assign to different factors such as probability and impact. For calculate probability, the factors that can be considered are the ease of discovery, ease to conduct the exploit, awareness of the exploit, and security controls that might either prevent or detect the exploit of the vulnerability. For the calculation of technical impact caused by an exploit of vulnerabilities, the factors that can be used for the risk calculation are the impact on the confidentiality, integrity, and availability of the data.

The calculation of the severity of vulnerabilities is essential for prioritize these vulnerabilities for threat remediation: vulnerabilities whose severity is ranked HIGH can be prioritized before MEDIUM risk vulnerabilities. The remediation of vulnerabilities can also be prioritized based upon other criteria such as the types or group of vulnerabilities that are found most prevalent and commonly found in web applications and included in the OWASP (Open Web Application Security Consortium) Top Ten 2013, the WASC (Web Application Security Consortium) vs.2, the SANS CWE/SANS Top 25, and MITRE CWE (Common Weaknesses Exposures). These categorizations of vulnerabilities follow different taxonomies but at high level can all be mapped and prioritized accordingly as it is shown in the Table 8.12.

It is important to notice that while OWASP Top Ten is prioritized with OWASP A01 as the most critical weakness and OWASP A10 as the least critical weakness and the same for SANS Top 25, which is a yearly list of the most severe weaknesses, other classifications such as WASC and CWE are not prioritized based upon the same criteria based upon progressive IDs. WASC Threat Classification is a weakness and attack classification maintained by the Web Application 2Security Consortium where IDs are assigned chronologically upon definition of the weakness or attack: WASC TCs lower IDs are not necessarily more severe than WASC TCs with a higher ID, they were just defined earlier. CWE (Common Weakness Enumeration) are also assigned on a sequential basis as they are defined.

It is important to notice that there is a difference between a prioritization by severity based upon the risk score assigned to the vulnerability (e.g. CVSS score), based upon the most prevalent type or group of vulnerabilities (e.g. OWASP) and based upon the most common vulnerabilities found in exploits such as security incidents and instances of compromise.

In essence, the fact that a vulnerability group that is most common (e.g. OWASP A01 Injection) does not mean is the one most exploited in attacks observed from threat intelligence and security incident reports. For example, Imperva's 2013 Web Application Attack Report (WAAR) identifies how many attacks a typical application

TABLE 8.12 Mapping of OWASP-WASC and CWE Source CriticalWatch: OWASP to WASC to CWE Mapping, Correlating Different Industry Taxonomy

OWASP Top 10, 2013	WASC v2	CWE/SANS Top 25 2011	CWE-ID
A01 - Injection	External Control of Assumed-Immutable Web Parameter		CWE-472
	File Injection		CWE-98
			CWE-73
	PHP Injection		CWE-94
	WASC-05 Remote File Inclusion		CWE-98
			CWE-426
			CWE-73
	WASC-19 SQL Injection	1	CWE-89
			CWE-564
	WASC-20 Improper Input Handling	27	CWE-20
	WASC-23 XML Injection		CWE-91
	WASC-25 HTTP Response splitting		CWE-113
	WASC-28 Null byte injection		CWE-158
	WASC-29 Ldap Injection		CWE-90
	WASC-30 Mail command Injection		CWE-88
	WASC-31 OS Command Injection	2	CWE-78
	WASC-36 SSI Injection		CWE-97
	WASC-39 Xpath Injection		CWE-643
	WASC-46 XQuery Injection		CWE-652
A02 - Broken Authentication and Session Management	WASC-01 Insufficient Authentication	5	CWE-306
			CWE-287
	WASC-11 Brute Force	21	CWE-307
	WASC-12 Contents Spoofing		CWE-345
	WASC-18 Credential/Session Prediction	7	CWE-798
		31	CWE-330
	WASC-37 Session Fixation		CWE-384
	WASC-47 Insufficient Session Expiration		CWE-613
	Reliance on Untrusted Inputs in a Security Decision	10	CWE-807
	Download of Code Without Integrity Check	14	CWE-494
A03 - Cross Site Scripting (XSS)	WASC-08 Cross-site scripting	4	CWE-79
	WASC-24 HTTP Request Splitting		CWE-93
A04 - Insecure Direct Object	Unrestricted Upload of File with	9	CWE-434

Reference	Dangerous Type		
	WASC-01 Insufficient Authentication		CWE-287
	WASC-02 Insufficient Authorization	6	CWE-862
		15	CWE-863
	WASC-33 Path Traversal	13	CWE-22
	Inclusion of Functionality from Untrusted Control Sphere	16	CWE-829
	Improper Access Control		CWE-284
	Authentication Bypass Through User-controlled Key		CWE-639
A05 - Security Misconfiguration	Information Exposure Through an Error Message	39	CWE-209
	Sensitive data under web root		CWE-219
	WASC-13 Information Leakage		CWE-200
		28	CWE-754
	WASC-14 Server Misconfiguration		CWE-16
	WASC-15 Application Misconfiguration		CWE-16
	WASC-16 Directory Indexing		CWE-548
	WASC-17 Improper Filesystem Permissions	11	CWE-250
		17	CWE-732
			CWE-280
			CWE-538
			CWE-552
A06 – Sensitive Data Exposure	WASC-50 Insufficient Data Protection	8	CWE-311
		19	CWE-327
		25	CWE-759
			CWE-326
	WASC-04 Insufficient Transport Layer Protection		CWE-311
A07 – Missing Function Level Access Control	WASC-02 Insufficient Authorization		CWE-285
	WASC-21 Insufficient Anti-automation		CWE-799
			CWE-084
	WASC-34 Predictable Resource Location		CWE-425
A08 - Cross Site Request Forgery (CSRF)	WASC-09 Cross-site Request Forgery	12	CWE-352
A09 – Unsing components With Known Vulnerabilities	[NO WASC]	16	CWE-829
A10 - Unvalidated Redirects and Forwards	WASC-38 URI Redirector Abuse	22	CWE-601

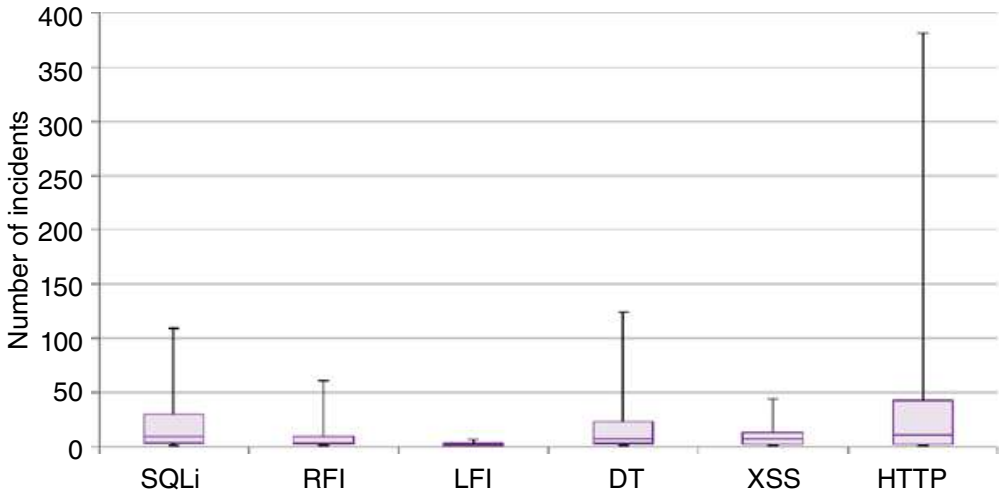


Figure 8.25 Number of Attack Observed in 6 Months by Imperva 2013 WAAR

can expect to suffer annually. This is based on observing and analyzing Internet traffic to 70 web applications during a period of six months. The number of attack incidents observed is shown in Figure 8.25.

According to this report, "*the most prevalent types of attacks are SQL injection, and directory traversal HTTP protocol violations, which often indicate automated threats, evasion techniques, and denial of service attacks.*" By comparing with

the most prevalent vulnerabilities found in application, we can drive the following conclusions:

1. The top 1 attack HTTP protocol violations (automation attacks) only maps to the top seven vulnerability missing function level access controls and WASC 21 missing antiautomation.
2. The top 2 SQL injection attack maps to the top 1 vulnerability OWASP A01-Injection and WASC-19 SQL Injection.
3. The top 3 directory transversal attack maps to the top 4 common vulnerability OWASP Top 4 Insecure Direct Object Reference, WASC-33 Path Transversal.
4. The top 4 Cross-site scripting attack (XSS) maps to OWASP top 3 (XSS).
5. The top 5 Local File Inclusion attack maps to the top 1 common vulnerability OWASP top 1 injection WASC vs2 File Injection.

If an organization ought to prioritize vulnerabilities based upon attacks observed using this metrics, the focus would be on fixing instances of lack of antiautomation countermeasures, input validation vulnerabilities, insecure direct object reference, and cross-site scripting.

According to the same WAAR report, different business verticals are also differently affected in terms of attacks sought, for example, online retain web applications experience about twice as many of SQL injection attack than other verticals including financial and fewer remote file inclusion attacks than the general application population. It is therefore important to prioritize vulnerabilities based upon observed security incidents and indication of compromises as well as common vulnerabilities observed for applications that belong to the same industry sector.

For the application in scope of the analysis, that is, online banking application, the vertical sector of reference is the financial sector. It is critical that the risk severity of the vulnerabilities identified as being most common in the sector of relevance is also compared with the threat intelligence shared by organizations in the same sector of relevant (e.g. for the financial sector the FS-ISAC) on which attacks are more prevalent and how to detect and block these attacks.

Known vulnerabilities analyzed at this stage and design flaws identified during stage V known to be exploited by the top attacks (e.g. input validation flaws, missing detection and countermeasures against automation attacks/bots) are the ones that should be prioritized. The effort for vulnerability remediation needs to be driven from the perspective of fixing them but also detecting the vulnerabilities by considering also the most effective measures that can detect and block attacks (e.g. black lists of attack sources) that seek to exploit them with countermeasures. In order to prioritize the monitoring effort on attacks, it is important to be able to acquire threat intelligence on malicious sources and apply this intelligence in real time. Since this threat intelligence can only be seen by analyzing data gathered from a large set of potential victims, it has to be gathered from information assurance centers such as FS ISAC.

At the end of this activity, it is important to produce a report of vulnerabilities for the online banking application that is correlated to the threats affecting each security component of the application. The risk severity of these vulnerabilities will be

updated by taking into consideration the likelihood of threat agent seeking to exploit them. This report also includes updated risk for control gaps/design flaws exposing data assets/functions to threats previously analyzed in stage IV (Architectural risk analysis activity)

Once the risk severity of the vulnerabilities has been analyzed based upon threat analysis information and the severity risk information has been updated in the knowledge base of exploitable weaknesses for the online banking application to prioritize the vulnerability remediation effort, the next activity of this stage consists of "*WVA 5.4 – Conduct targeted vulnerability testing.*" The goal of this activity is to conduct targeted vulnerability tests. These tests will also include a set of new test cases that include the knowledge of threats targeting these vulnerabilities from the threat to vulnerability mapping analysis done previously. The scope of these targeted vulnerability tests is to revalidate known vulnerabilities previously identified as these were identified by previous application security assessments such as security design reviews, source code reviews, and application static and dynamic tests. These security tests will also validate any design flaws and vulnerabilities that are part of the knowledge based and are considered at high risk of being exploited by threat agents.

The value added by conducing these targeted vulnerability tests comes from testing known and new vulnerabilities as a real threat agent will try to identify them for later exploit them in an attack and using the same tools and attack techniques known from the threat intelligence and threat analysis. The purpose of these tests is also to determine the exposure and the risk as technical impact on the application assets including confidential and sensitive data as well as business critical functions that can be targeted by specific threats. For example, this can be a case of testing a known SQL injection vulnerability that was previously assessed and remediated but can know be retested with the knowledge of specific threat vectors (e.g. encoded threat vectors seek to bypass known filtering in place) and by using the same attack tools (e.g. Havij SQL injection tool) used by known threat agents.

When creating these test cases, it is important to be able to correlate the security test cases that apply to each functional component of the application and correlate these with the threat being analyzed in stage V and the security requirements documented during stage I as well as vulnerabilities identified in previous assessments that belong to that component under review. An example of deriving security test cases for each functional component for the login functional component of the online banking application (refer to the Hacme Bank ThreatModeler™ template and project) is shown in Figure 8.26.

These security test cases can be executed during this stage of the assessment as well as during the validation phase of the SDLC to reassess the risk posed by these vulnerabilities in the presence of security measures such as protective application security controls (e.g. authentication, authorization, encryption, data input validation, session management) as well as countermeasures that can detect specific threat triggered events (e.g. application and system logs).

By looking at each functional component of the application and by analyzing security functional requirements, threats targeting weaknesses and vulnerabilities including the ones identified as design flaws during the architectural risk analysis

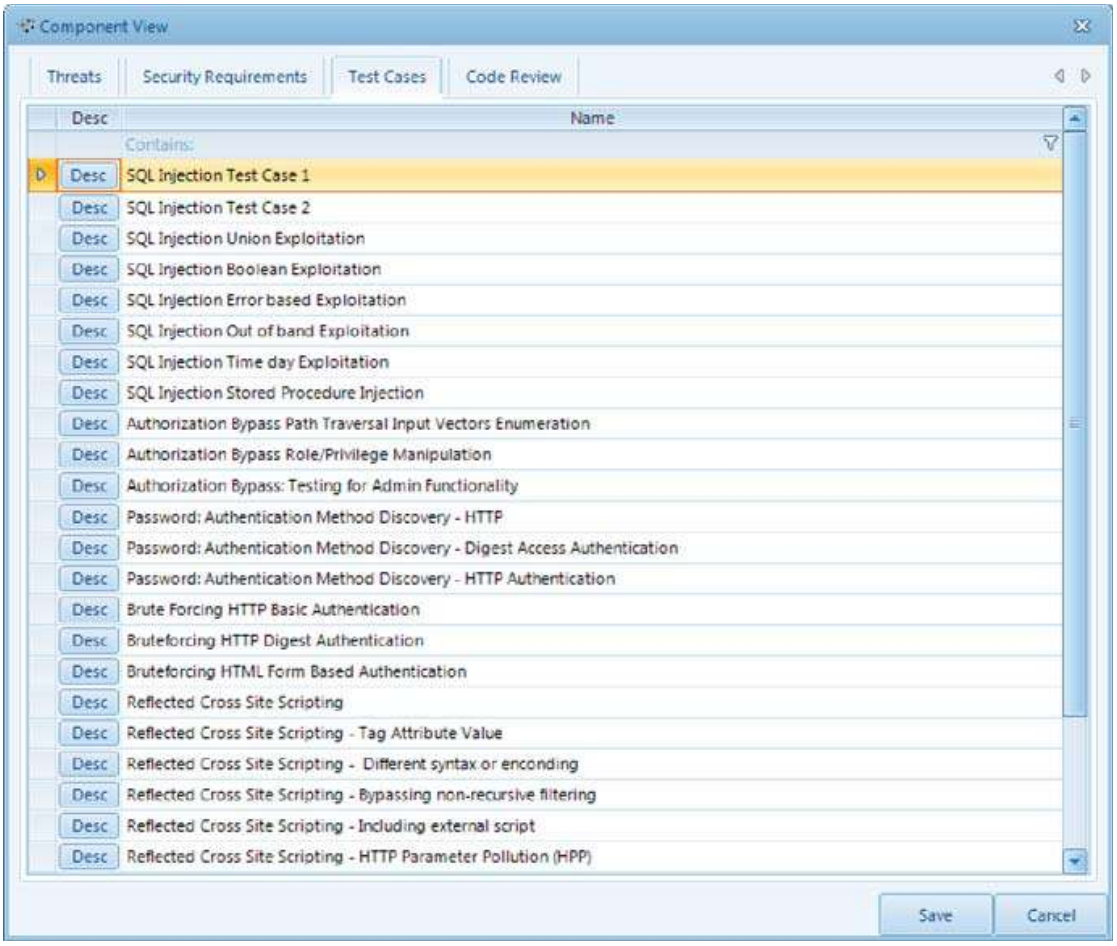


Figure 8.26 Test Cases to Validate Vulnerabilities at Component Functional Level ThreatModeler™

that affecting these security controls associated with functional components and the security tests that can validate these security controls weaknesses and vulnerabilities, we have an actionable view to assess the risk of each functional component, and we can track each threat to mitigate the impact by applying fixes of these weaknesses and vulnerabilities as well as by applying additional countermeasures when the remediation of these vulnerabilities is not enough to mitigate the impact. Since, to determine the extent of the impact, it is important to look at the specific of the attacks and how these vulnerabilities are exploited prior to conducting these attacks, we would need to conduct an attack modeling exercise. This attack modeling will be performed in the next stage of PASTA, stage VI Attack Modeling and Simulation.

Stage VI – Attack Modeling and Simulation (AMS)

Goal of This Stage The goal of this stage is to analyze and model the attacks against the application. The analysis of the attacks relies on the analysis of the chain of events such as kill-chain reported by threat intelligence or as root cause analysis of a security incident that was analyzed "post mortem." Based on the analysis of the attacks and the

previous identification of the threat agent (TTPs) Tools, Techniques, and Processes, it is possible to analyze the course of actions of the various attack scenarios and associate with the TTPs used by the threat agents/actors of these attack scenarios. Once the attack scenarios have been analyzed and the various attack vectors are identified and updated in the attack libraries, the next step consists of deriving a set of cases to test existing countermeasures and conduct attack-driven security tests and simulations. The objective of these test cases is to simulate realistic attack scenarios and determine if exploits from the threat agents are possible and identify countermeasures to prevent and detect the attack vectors used.

Guidance for Conducting This Stage The scope of the Attack Modeling and Simulation (AMS) stage is to create an attack model to simulate attacks against the application and to determine whether these attacks could result in possible impacts on the application assets such as sensitive data and business critical functions. By analyzing the various attack scenarios in the attack model, it would be possible to determine the effectiveness of application security controls in mitigating the impact of these attacks. The modeling of these attacks starts by considering the threat scenario that was previously analyzed in stage IV including the various types of threat agents/actors identified in that threat scenarios. These threat agents are also known to use specific tools, techniques, and procedures (TTPs) to conduct their attacks. In this stage, we focus on these TTPs as well as on the various attack vectors used by these threat agents. Some of the TTPs and attack vectors can be attributed to specific threat agents, their capabilities and motivations as well as the tools used such as cyber-crime tools to conduct these attacks.

Attack modeling considers both opportunistic exploit of weaknesses and vulnerabilities in web applications and targeted attacks against products/applications. This attack modeling relies on the knowledge of the threat agent kill-chain and on the analysis of the overall threat scenarios. The attack vectors that are identified by the analysis of the attack scenarios will enumerate all types of possible attacks and the specific types of security controls (e.g. authentication) that are attacked and will document how these can be attacked by using an attack library.

Some of these attacks can also opportunistically target known vulnerabilities that are accessible through user and data interfaces including vulnerabilities that should be considered inherent of the technology stack such as software framework and service vulnerability used by the application. For example, mobile applications have inherent vulnerabilities that might be exploited by specific threat agents using different attack vectors. These attacks might follow a specific chain of events such as will seek to compromise the mobile device first and then bypass authentication controls such as multifactor authentication with (MiTMo) Man-in-the-Mobile Attacks.

One of the techniques that can be used for modeling attacks includes attack trees. Attack trees help to create a model of how the attacker's main goals can be realized by following a specific branch of the attack tree. The model of attacks with attack trees help to identify the most probable path that can be followed by an attacker to reach his goals. Another method that can be used to model attacks is to analyze the

business use cases and identify the possible abuse cases that can be followed by the attacker to bypass security controls.

After the various attack scenarios have been analyzed, the next step consists of analyzing how threat agents can conduct attacks to cause an impact on the application.

Since an attack describes how a threat can be realized, the probability of a threat to be realized in an attack depends on several factors and among them the costs for an attacker to conduct the attack based upon the attacking tools at his disposal and the opportunities to exploit known vulnerabilities. To assess the probability of a threat to be realized in an attack to cause an impact, it is important to assign the values to the costs of the attacks and the benefit for the attacker as gain and choose to simulate the attacks that maximize the gain and minimize the costs for the attackers.

An attack scenario might occur because of the exploit of specific vulnerabilities that increase the probability of a threat to conduct successful attacks against the application assets. The analysis of the probability of possible attacks among the various options of attack available to an attacker can be done by analyzing attacks using attack trees. By analyzing the various possible paths for conducting attacks with an attack tree, it is possible to determine the attacker's course of actions and attack events that have higher probability to succeed in achieving the attacker's goal. By modeling attacks using attack tree, we can model each step of the attack as the opportunity that an attacker has to move to the next step of the attack till it reaches the desired goal for the attacker. The realization of an attack can be analyzed by assigning conditions to each node of the attack tree. At each node of the attack tree, an attacker has the options of "OR" (one attack or another can be chosen) or "AND" (both attack paths can be chosen). For example, a threat agent seeking to compromise the data in a database for the attack scenario of a banking Trojan needs to own the client "AND" attack the application "OR" attack the application directly. Given that the probability of attacking the client is lower than attacking the application directly, it will choose that attack path. The probability of success is given by the consideration of all single probabilities of each node whose conditions need to be fulfilled in order for the attacker to reach his goal. Once the most probable attack paths have been analyzed including the application components, processes, and security controls that a threat agent need to transverse on its way to the target, it is possible to create specific test cases for simulating the attacks.

Attack trees can have different scope such as network layer scope, architectural level scope, and functional level scope. The analysis of attacks using attack trees at the network later can be useful to analyze (CNE) Computer Network exploits as well as (CAN) Computer network attacks. At the application layer that is the focus of risk-centric application threat modeling, attack trees can be used to analyze exploits of vulnerabilities affecting architectural components including the various services, API and libraries, and system-level processes that support the application functionality.

The analysis of sequence of events that attackers follow when conducting attacks against the application assets, the analysis of the attack surface to identity the application data entry points that are sought to conduct the attacks as well as the analysis of the most probable attack scenario based upon the analysis of the attack trees helps in the derivation of test cases to simulate the attack scenarios. Once each of the attacks

is simulated with tests, it is possible to determine for each threat agent the probability to achieve his desired impact such as exfiltration of data, stealing money, and fraud. The analysis of impacts include both technical impacts such as loss of data confidentiality, integrity, accountability, and availability as well as business impact as monetized loss associated with a data asset. The determination of the probability of threats to be realized in attacks and the likelihood of the attack to produce either a technical or business impact allows determining the risk of each attack. The identification of the various events of an attack leading to an exploit of vulnerabilities to cause an impact is critical to determine the detective controls that can be deployed to detect these events as indication of an attack and to decide the appropriate response to mitigate the impact.

Once the attack has been analyzed and simulated, it is possible to update the attack library with the mapping of threats, controls, and vulnerabilities that these attacks seek to exploit. This attack library can be inclusive of all known attack vectors. These attack vectors can be later used for the creation of specific tests for testing the resilience of security controls and countermeasures to these types of attack vectors.

Inputs for Conducting This Stage Since attacks describe that threat agents might exploit security controls and weaknesses to cause an impact, the type of information that is required for modeling of attacks is the knowledge of the threats and of the vulnerabilities. The type of documentation that can be useful for modeling attacks includes:

1. Threat scenarios analyzed in stage IV.
2. Threat agents analyzed in stage IV.
3. Threat actors TTPs analyzed in stage IV.
4. The assets targeted by threats analyzed in stage IV.
5. Existing vulnerabilities and weaknesses targeted by threats analyzed and mapped in stage V.
6. Results of testing of vulnerabilities and design flaws executed during stage V.

Artifacts Produced at the End of This Stage At the end of this stage, we will produce the following artifacts:

(a) *The model of the attack scenarios* including the course of action of the attacks using kill-chain model that is followed by the different threat agents/actors.
(b) *Attack trees* with the determination of the most probable attack paths leading to exploits and identification of various vulnerabilities and security controls that might be bypassed.
(c) *Use and abuse cases* covering bypass of security controls and abuse of functionality for fraud and data compromise.
(d) *The attack surface* that is targeted by threat agents previously identified in the attack modeling scenarios (analyzed for threats and data entry points previously identified stage IV Threat Analysis).

(e) *Updated attack library* with new attacks that can be enumerated for simulating/testing the various attack scenarios.

(f) *Documented test cases* that can be used to test the resilience of the application in the presence of specific conditions of exposure of vulnerabilities as well as by considering the real attack scenarios, automated and not, single or group based, capabilities, motivations, cyber-attack tools and techniques used in real attack case scenarios.

(g) *Attack testing/simulation report* that includes the results of the test cases for testing the various attack scenarios for vulnerabilities and design flaws. The report includes the assessment of risks of each security issue (design flaws, vulnerabilities) that is identified such as probability and technical impact can be determined based upon the results of the test/simulation.

Tools The execution of this stage can be facilitated by the use of WASC WHID attack libraries and threat-driven test cases to simulate attacks included in myAppSecurity ThreatModeler™. MS STRIDE methodology is used here to derive and prioritize threat-driven test cases. Formal methods for capturing attack scenario used here is Lockheed™ kill-chain and use of kill-chain in MITRE STIX for threat-vulnerability-attack correlation. Analysis of attacks and probability includes the use of formal methods such as Amoroso attack trees. Standard use and abuse cases and MITRE CAPEC attack patterns are also used for this stage. Attack analysis and intelligence are also provided by sources of OSINT threat intelligence and vendor reports.

Activities This stage consists of the following activities:

AMS 6.1 – Analyze the attack scenarios: An attack scenario can be created for each threat agent based upon the information captured from various sources such as threat intelligence, logged security events incidents and the analysis post mortem of security incidents. Threat agents can be classified based upon their motives and capabilities. Examples include hacktivists, script kiddies, cyber-criminals, fraudsters, and government/state and corporate-sponsored spies. A threat agent might rely on different type of manual attack techniques and automated attacking tools. The attacks against the targets can follow a chain of events such as the attack course of action. The analysis of the sequence of events of a cyber-attack helps to model the attack scenario. The model of the attack scenario also includes the attacking vectors used and the specific vulnerabilities that are exploited by the threat agents to realize their goals such as data compromise, data theft, online fraud, abuse of functionality, denial of service, and abuse of privileges as example. The outcome of this activity is to create attack stories that describe how the various threat agents might attack the application.

AMS 6.2 – Update the attack library/vectors and the control framework: After the attack scenario has been analyzed and modeled, it would be possible that

some of the attack vectors attributed to the threat agent's attack techniques and tools used are not part of the threat library and therefore need to be updated including the type of countermeasures that have been found effective in either detecting or preventing the attacks. The new countermeasures that should be considered in the threat library might include new emerging threat attack vectors analyzed in the attack modeling exercise as well as updated vulnerabilities and weaknesses and gaps in preventive and detective security controls. (*Note:* It is important to revisit the control assessment risk framework to add new preventive and detective controls as options to mitigate the attacks.)

AMS 6.3 – Identify the attack surface and enumerate the attack vectors: Against the data entry points of the application. Use the up-to-date threat library to identify the type of attack vectors that can be used, the type of vulnerabilities of the architecture components that can be exploited and the security controls that should be in place to detect and prevent the attacks.

AMS 6.4 – Assess the probability and impact of each attack scenario: Determine the probability of each attack of the threat scenario using attack trees (probability of exploit producing the desired technical and business impact that maximize gain for the attacker). Identify the various security controls that can be bypassed and functionality that can be abused leading to the exploit and impact. Each attack scenario can be associated with the probability of the attack and the impact so can be prioritized for the analysis and mitigation.

AMS 6.5 – Derive a set of cases to test existing countermeasures: These test cases are both threat driven using threat classification (e.g. STRIDE) and threats, vulnerabilities identified in each architectural components. Attack driven test cases can be documented based upon the previous analysis of the attack chain of events and the attack vectors used by the attackers. Additional tests can be created using use and abuse cases. Each test case can be prioritized based upon the risk that was previously calculated for each of the attack scenarios using attack trees.

AMS 6.6 – Conduct attack driven security tests and simulations: The scope of these tests is to identify vulnerabilities and design flaws that are exploited by the attack and determine the risks as factor of probability and the impact. The risk ought to factor the technical impact (loss of confidentiality, integrity, accountability, and availability) of these exploits using the results of the simulated attack exercise. After each test case is executed, the risk values that were previously assigned to each threat should be revised higher or lower depending on the results of the tests.

Web Application Use Case Example Before we start modeling the attacks targeting application in scope, it is important to highlight the difference between threats and attacks. There is a difference between threats and attacks and it is important not to confuse the two: threats describe negative events or conditions with potential for damage, whereas attacks describe how threats are realized to produce exploits and instances of compromise of data and functionality.

Examples of threats affecting online banking applications include breaches of confidential data of customers such as bank accounts and credit card data, fraudulent transactions such as unauthorized money transfers and denial of service to online accounts and transactions. Examples of attacks describing how these threats can be realized include exploit of SQL injection vulnerability to gain unauthorized access to confidential data, exploit of social engineering to install key logger malware on the bank customer's PC, compromises of user's login credentials including multifactor authentication credentials with MitM attacks and session hijacking and account takeover for transferring money to fraudulent accounts.

The description of how a threat is realized that is the definition of an attack also includes the notion of vulnerability since vulnerabilities describe opportunities for a threat agent to conduct an attack, for example, by exploiting a vulnerability to install malware on the client or to break into the web application (e.g. exploit of SQL injection vulnerabilities in the web application). Since attacks are best used to describe how threats can be realized through vulnerability exploits, we need first to analyze the threats and the vulnerabilities. This is the reason why in PASTA stage VI (AMS) Attack Modeling and Simulation comes after Vulnerability and Weakness Analysis (WVA) (stage VI) and Threat Analysis (TA) (stage V).

For the objectives of this stage, that is, the modeling of attacks to simulate the attack scenario with security test cases, we need first to analyze the attack scenario. This is accomplished by executing the stage activity "*AMS 6.1 – Analyzing the attack scenarios.*" To perform this activity, the threat analyst/modeler needs to consider the type of threat agents of the threat scenarios previously analyzed and describe how an attack takes place as sequence of steps that the threat agent/actor might follow. The description of how threats of specific threat scenario can be realized to produce the desired impact such as compromise of confidential data and fraudulent transactions is the objective of the attack modeling. As minimum requirement, the description of the attack scenario needs to be comprehensive and detailed and to include the description of the attacking tools (e.g. cyber-crime toolkits), attacking techniques, and attack vectors used by the attackers against the target in this case the online banking application. The analysis of threat targets done in stage IV is also very critical for the modeling of attacks since it provides mapping of threat targets to application vulnerabilities that could be exploited during the attack. Another important element for modeling attacks is the indication of compromise and the security events that can be monitored as indication of security incidents. These security events might be triggered and detected as indication of compromise of confidential data and fraudulent transactions.

The analysis of attacks starts first and foremost with the analysis of the threat agent's characteristics specifically the type of threat agents that might target the online banking application. This analysis was done during stage IV. At high level, the threat agents attacking the online banking application might include internal threat agents such as malicious users and disgruntled employees as well as external threat agents that can be classified based upon their motivations and capabilities as hacktivists, script kiddies, cyber-criminals, fraudsters, and government/state and corporate-sponsored spies.

Each of these threat agents that were previously analyzed to target the online banking application might have different motives and capabilities and uses different types of attack techniques and tools. Some of these attacks might require the usage of sophisticated attacking tools also known as crimeware that is specifically designed by cyber-crime organizations to attack specific targets (e.g. malware banking Trojans used to attack online banking applications and botnet-based DDoS tools). The attacks used by threat agents to attack online banking sites are not all sophisticated necessarily rather might rely on exploit of common vulnerabilities and the attack vectors be included in freely available tools such as application security vulnerability scanners.

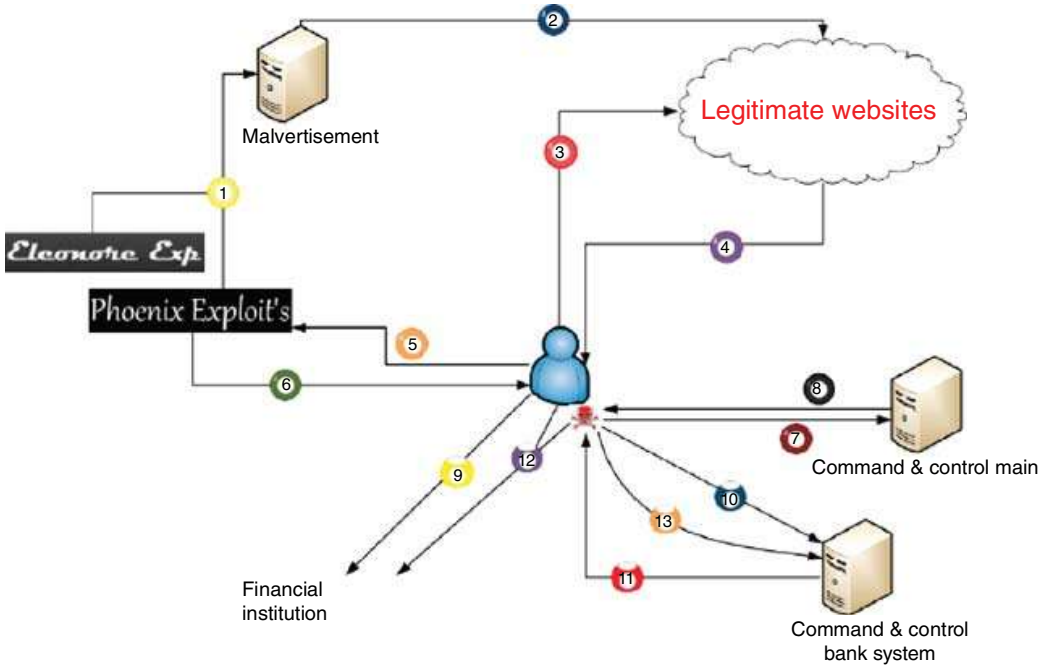
The various agents targeting the online bank application for an exploit might rely on an arsenal of different type of attack techniques, tools, and attack strategies in order to achieve their goals. For the sake of the use case shown herein, we focus on the known attack scenarios of fraudsters targeting an online banking application for online fraud, identity theft, stealing of confidential data of customers, and hacktivists targeting online banking applications with distributed denial of service.

When tasked to conduct the modeling of attacks against the online banking application, a threat analyst could rely on previous knowledge gathered on the attacks that take place after a security incident. Typically what follows after a security incident has been contained is the analysis of the incident to identify the type of vulnerabilities that were exploited as well as the preventive and detective controls that were bypassed during the course of the attack. An analysis of a security incident postmortem might also identify the sequence of events that is followed by an attacker firstly for conducting the exploit to own the target and secondly to use the compromised target to steal confidential data.

The information of the sequence of events that is followed by a threat agent to conduct and exploit of a weaknesses or vulnerability as well as to compromise the target to either steal confidential data or alter the integrity of financial transactions to commit fraud is a critical piece of information that can used for the attack analysis and simulation. This critical type of information is known with the term of “kill-chain” and is usually associated with a threat agent and can be included in the threat intelligence reports that threat intelligence sources might shared with interested parties (e.g. as threat intelligence information shared among financial institutions that are part of information assurance sharing information centers).

In the case of the online banking application, for example, and for the attack scenarios in scope that includes client compromise with banking malware and distributed denial-of-service attacks, several attack vectors can be analyzed based upon the information gathered by threat intelligence. Specifically this threat intelligence might include information about the specific attack vectors used to conduct the exploits such as to compromise the client’s PC/browser through drive by downloads and MitB and compromise of the “secure” online channel established between the client PC/browser and the online banking application using MitM and compromise of the authenticated session to perform unauthorized financial transactions using session hijacking.

For the analysis of the attack scenario, it is necessary to capture the sequence of events that lead to exploits and compromises of data and functionality. By analyzing



1. Uploads malicious ads to legitimate and fraud ad servers
2. Malicious Ads published on legitimate websites
3. User accesses infected website
4. Website content contains redirection to exploit kit
5. User is redirected to exploit kit
6. User's PC exploited and payload downloaded successfully
7. Trojan reports in to C&C server
8. C&C server sends instructions to trojan
9. User accesses FI web site
10. Trojan reports on user activity to C&C server
11. C&C server sends commands to manipulate transaction
12. Bank transaction is altered to unauthorized payee
13. Trojan reports back success/fail to C&C server

Figure 8.27 Sequence of Events Followed in Banking Trojan Attacks

the sequence of events of the attack, it is also possible to identify the type of security controls that are either compromised or bypassed and the vulnerabilities and weaknesses that are exploited by the attacker. An example of this sequence of events used in a banking malware attack against the user's online banking financial transactions is shown in Figure 8.27.

This figure shows the sequence of events that is followed by a fraudster to steal money from a victim's bank account starting with a compromise of the victim's PC with banking malware/Trojan.

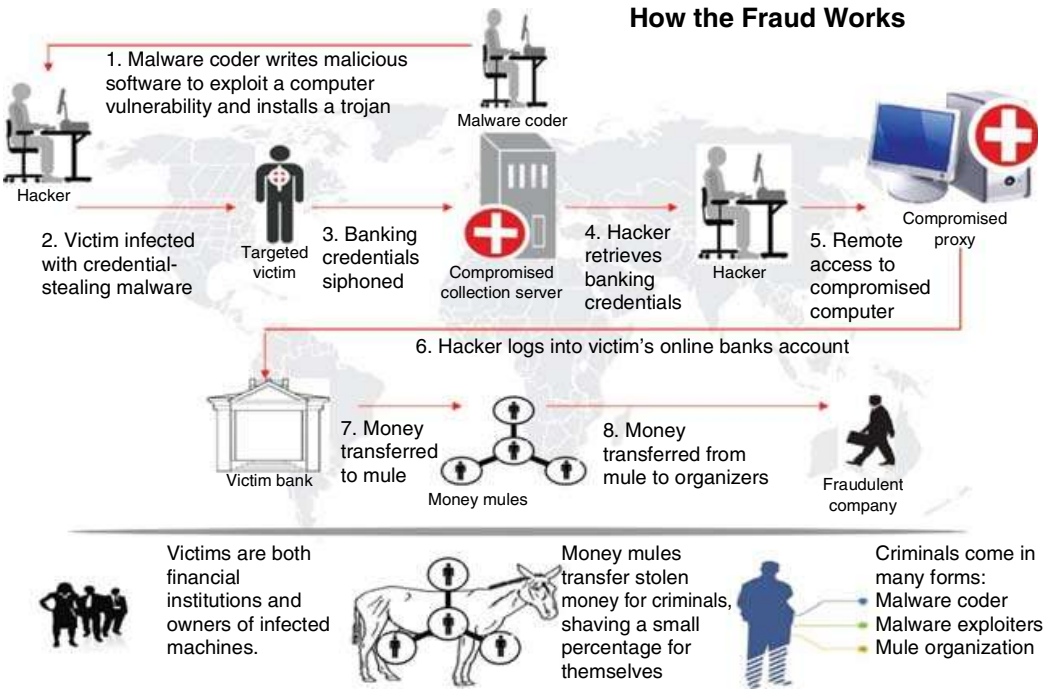


Figure 8.28 Anatomy of Account Takeover and Fraudulent Wire Transfer[4]

Figure 8.28 shows the type of threat actors that are involved in the malware banking attack and the various techniques and strategies that use for conducting fraud such as stealing money from the bank account of the victim.

To model the attacks, we need to start from the analysis of the attack. Once we have modeled the attack scenario, we can analyze more in detail the attacking vectors used and the specific vulnerabilities that are exploited in the attack. At the beginning of the attack modeling exercise, attack scenarios can be modeled at high level by creating attack stories that describe how the various steps taken by the threat agents to achieve their goals.

Once the attack scenario is captured and we have identified the type of threat agents/actors and the attacking tools, techniques, and processes used, we can start to analyze more in detail the attack tools to extract the information about the attack vectors. An example of attacking tools that should be analyzed includes the various types of toolkits used by the attacker to create Trojan/banking malware and analyze the various types of vulnerability exploits and the various types of attack vectors used. An example of this type of analysis is shown in Figure 8.29.

By the analysis of the attack vectors used in banking malware/Trojans, it can be concluded that different types of infection techniques can be used by attackers in order to compromise the target client PC: this include social/engineering phishing, drive by download, malicious and exploit of common vulnerabilities in web applications as well as the browser. Once the malware is installed in the client/browser, an

[4]http://krebsonsecurity.com/tag/operation-trident-breach/.

Trojan	Infection Method					Attack Capabilities										Timing		Type	
	Phishing	Drive-by Download	Malicious Web Link	Malicious Ad	Virus Infection	HTTP Injection	Browser Redirect	From Grabbing	Credential Theft	Keystroke Logging	By Pass MFA	Screen Capture/Video	Certificate Theft	Install Backdoor	Instant Message	Real-Time	Out of Band	Automated	Manual
MB - MitB MM - MitM B - Both O - Other						MB	MM	B	B	B	B	O	O	O	O				
ZeuS	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*	*
Spy Eye	*	*	*	*	*	*	*	*	*	*	*	*		*		*	*	*	*
InfoStealer	*	*	*	*	*	*		*	*	*	*	*	*	*			*		*
SilentBanker	*	*	*	*	*	*	*	*		*	*	*	*	*		*	*		*
URLZone	*	*	*	*	*	*		*		*	*	*		*		*	*	*	*
Clampi/Bugat/ Gozi	*	*	*	*	*	*				*							*		*
Haxdoor	*	*	*	*	*	*		*		*				*			*		*
Limbo	*	*	*	*	*	*		*		*	*			*			*		*

Figure 8.29 Attack Vectors Used in Banking Trojan Malware, Source OWASP Anti-Malware Knowledge Base

attack agent can be activated under control of the command and control server to initiate several types of attacks against the online banking application using attack vectors such as MitM and MitB attacks. These attack vectors use redirects to redirect the victim to malicious sites, allow the fraudster to steal user's authentication credentials including multifactor authentication, take over the victim's bank account and compromise the integrity of the transaction including changing the recipient of the money transfer, changing the amount of money to be transferred, and install backdoors on compromised servers with the ability to take commands and steal confidential data.

The reverse engineering of the attacking tools and techniques used by the attackers helps to capture the attack vectors and use them for security testing. The analysis of the attack scenario is very critical and needs an in depth analysis, specifically in the cases of banking malware and DDoS attacks, it is critical to understand the sequence of the attack from the initial step of the reconnaissance of the target to the final step of realize the threat that is to achieve the attacker goals such as stealing sensitive data

of customers, stealing money from bank accounts, and denial of service to access the online banking site including ability to authenticated and conduct business transactions such as checking account balance, paying bills, and other financial transactions that are available online.

In order to capture the attack scenario and specifically to dissect an attack by qualifying the events and the various activities conducted during each event, it is important to follow a model for quantifying and qualifying the various cyber-threat activities. One possible method to capture the qualifying events of an attack so that the defender can act a proper response is the "kill-chain" method. The kill-chain method was originally developed by the military as US (DoD) Department of Defense technique to locate, track, target, engage, and eliminate threat agents.

From the perspective of cyber-attack detection and eradication of the threat agents, a kill-chain model can be adopted to cyber-attacks for analyzing the actions taken by cyber-threat agents so that these can be used to capture the attacker's logical progression into steps that lead to the attacker goals such as data compromise and financial gain. The phases of the attack kill-chain consist of (1) Reconnaissance (2) Weaponization, (3) Delivery, (4) Exploitation, (5) Installation, (6) Command and Control, and (7) Action and Objectives. This technique has been adopted to cyber security and in particular intrusion security and incident response. This section will adopt kill-chain to analyze the various steps of the attack workflow that motivated threat actors to attack an online banking application.

The model and simulation of the attacks can also follow the sequence of events that were previously analyzed and shared by the threat intelligence sources. An example of information that is critical for the modeling of an attack is the sequence of events before and after an exploit that is the kill-chain phase of the attack. A threat observable event can be correlated to a kill-chain to determine if this observable is correlated to a kill-chain event such as a pre- or a postevent of an exploit of vulnerability. These events can help the threat analyst in the simulation of the attack by simulating the sequence of events in a security test. The security test can be conducted as a simulation exercise of the attack that is as much as possible realistic of the attack vectors, attacking tools and application vulnerabilities that are exploited.

Kill-Chain Attack Analysis of Banking Malware Attack Scenarios An example of capturing the attack scenario for the various phases of the kill-chain of malware banking/Trojan attacks is included herein:

1. *Kill-chain step I of the malware attack: Reconnaissance:* This step of the attack consists of the research, identification, and selection of targets for malware based attacks. During this phase, the attackers scan and identify sites that embed advertisements and are vulnerable to malicious advertisement uploads (e.g. iFRAME vulnerabilities, SQL Injection). Research of client vulnerabilities (e.g. browser plug-ins, flash vulnerabilities) that can be exploited for dropping malware (e.g. XSS, XFS-click-jacking, spear-phishing social engineering and drive-by-download attacks) as well as vulnerabilities of web applications that can be exploited to attack online banking sites. Derive a set of requirements for

the development of the malware as attacking tool crimeware that can exploit vulnerabilities for both droppings and exploitation/installation;

2. *Kill-chain step II of the malware attack: Weaponization:* During this step, the threat agents engineer the malware to satisfy the attack requirements. This includes also acquiring crimeware tool kit by cybercriminals that can be customized to conduct the attack. The attackers engineer online banking malware to include both disclosed (e.g. CVEs) learned from reconnaissance and undisclosed (e.g. "zero-day") vulnerabilities. Vulnerabilities are aggregated into crimeware kits using custom exploit code intended for a specific online banking URLs. Custom exploit code might include attack vectors after learning about the internal network of the bank as well as after reverse engineering the source code of the web application and understanding of the online banking site functionality. Final assembly/packaging of the malware might use open source, toolkits, and use of architected botnets of java toolkits, php scripts, or command line batches. Less skilled attackers can acquire these banking Trojan kits from cybercriminals and configure them to attack online banking sites of specific FIs (Financial Institutions); this phase of the attack includes the transmission and delivery of the malware to the target environment.
3. *Kill-chain step III of the malware attack:* Delivery of the malware (e.g. dropping of malware on target PC/server) might occur via different infection methods such as
   a. drive-by-download browsing a malware infected site to install malware on the client host;
   b. remote command execution exploiting SQL injection or other vulnerabilities to upload malware on the targeted internal server host;
   c. phishing e-mail impersonating the bank/FI with malicious link pop-ups and with malicious attachments (e.g. adobe pdf);
   d. search a site that point search to malicious URL pointed by infected Search Optimization Engine (SOE);
   e. malware installed by disgruntled employee allowing outbound web traffic to C&C from that server;
   f. unauthorized access to internal network of the bank by third-party contractor;
   g. malware installed on BYOD allowing access to internal network resources;
   h. source code with embedded backdoors by rogue S/W developer/contractor;
   i. third-party provisioned software/hardware with backdoor capabilities;
   j. malicious link delivering malware posted by threat actor on social media (e.g. Facebook) and shared with a social media group.
4. *Kill-chain step IV of the malware attack: Exploitation:* This phase of the attack consists of the triggering and activating the malware to exploit vulnerabilities on the client and the server. Once the malware is delivered to the target using one of the delivery methods (e.g. drive by download, opening attachment in e-mail), it will start to execute and scan for vulnerabilities that can be used for the installation and later for the execution of the attack under the control of

the fraudster/cybercriminal. Client vulnerabilities that are exploited for installation via drive by download includes several O.S and browser vulnerabilities listed with known CVEs and zero-days. These vulnerabilities might or might not require admin access to the target PC. Web application vulnerabilities that can be exploited to attack online banking website include common vulnerabilities that facilitate the installation of the malware on the client (e.g. XSS) as well as vulnerabilities that can used to install malware on web server (e.g. iFrames) and to gain access to databases (e.g. SQL injection) and later operation of the malware under command and control of the fraudster that facilitate bank account takeover (e.g. session hijacking)

5. *Kill-chain step V of the malware attack: Installation:* The goals of this phase of the attack are the installation of the malware on the client including possible remote backdoor on online banking application server for persistent access to the application. Once the CVE vulnerability is identified by the dropped malware, the online banking Trojan will install various malware packages (e.g. key loggers) on the client PC. In the cases when the exploitation/compromise also includes access to the internal servers, for example, through the exploit of a server vulnerability (e.g. command injection, SQL injection, remote file inclusion) by direct access to server through the internal network, the malware will install on the server. Example of malware installed on the server includes backdoors by exploiting vulnerabilities on the server and exfiltration data from these servers through remote access to that system as well as packet sniffers to capture data from the network.
6. *Kill-chain step VI of the malware attack: Command and Control:* During this phase of the attack, the attacker will establish a channel to enable remote control of the malware installed on client and server hosts. The banking Trojan will send messages out to the Internet to establish a communication channel to the attacker. Once the channel is open, the threat agent will have access to their target environment and receive command from the bot Command and Control (C&C) so that it can fully operate to control online banking sessions initiated by the client as well as to remote access both client and server hosts for exfiltration data and manipulate financial transactions (e.g. money transfers/wires).
7. *Kill-chain step VII of the malware attack: Action and Objectives:* This step of the attack includes the execution of the malicious/fraudulent actions such as exfiltration of online banking credentials and movement of money from the customer account to the fraudsters. This step of the kill-chain is when the attacker will attempt to accomplish their intended goal by operating the malware by following a sequence of events:
    a. Once the customer log-on the online banking site, the banking malware will report on the customer activity to the C&C server.
    b. The C&C server will send commands to the banking Trojan to steal authentication data, multifactor authentication credentials (e.g. challenge questions), and PII from the victim. The attack vectors include the following:
        i. Logging keystrokes
        ii. Form grabbing for transaction authentication numbers (TAN)

iii. Taking screenshots
iv. Exporting private key certificates
v. Exporting protected storage passwords
vi. Monitoring for file transfer and e-mail passwords (FTP and POP3)
vii. Routing connections through the infected machine

c. The banking credentials will be sent to malware operators in near real time (using Jabber).
d. The fraudster manipulates the online transaction by exploiting full control of the user's session and the web channel through different attacks (e.g. session hijacking/automatic transaction hijacking (ATH), MiTM).
e. The bank transaction is manipulated and is altered to send money to unauthorized payee (e.g. money mule).
f. The banking malware Trojan reports back the success/fail to C&C server.
g. The money stolen from the customer bank account is transferred from mule to fraudsters.

Similarly to the analysis of the online banking malware attacks, we can use the kill-chain for modeling of distributed denial-of-service attacks scenarios:

Kill-chain analysis of DDoS attack scenarios: An example of capturing the attack scenario for the various phases of the kill-chain of DDoS includes the following:

1. Kill-chain step I of the DDoS attack: Reconnaissance. This step of the attack consists of the research, identification, and selection of targets for DDoS attacks. The most likely threat agents targeting online banking sites for DDoS are politically motivated hacktivists for damaging the reputation of the financial institution, money-driven fraudsters seeking to distract the financial institution during account takeover attempts and country-state sponsored attackers seeking to disrupt the financial operations of a state owned or country's government operated online banking site. In general, the likely targets of DDoS attacks are public and private organizations with high visibility. Once the threat agents have decided which private or government operated Financial Institution will be targeted by a DDoS campaign, they will start to search for the most visible websites (Note: one potential method to assess the web site visibility is to use web analytics such as Alexia rankings for company websites). After a list of target websites has been identified, the threat agents will conduct a survey of the security of the target websites. The goal of this survey is to identify website vulnerabilities and weaknesses that can be exploited with specific attack vectors. Since DDoS seek to attack websites directly, the focus is on exploit of application layer and network layer type of vulnerabilities that lead to DDoS exploits. According to threat intelligence reports gathered during stage IV, for example, a DDoS reconnaissance phase of DDoS campaigns previously conducted by hactivist threat actors consists of the following activities:

i. Scanning for web application vulnerabilities (e.g. directory traversal, SQL injection, XSS)
ii. Scanning for pre-authenticated web pages of the web application which are time-consuming and resource consuming and might lead to exhaustion of the server resources (e.g. download of large PDF files, use of search engines that are not cached involving high consumption of CPU resources)

After these application vulnerabilities and weaknesses have been identified similarly to how penetration testers find web application vulnerabilities using commercially available pen testing tools (e.g. Havij automated SQL injection tools, Acunetix web vulnerability scanner, Nikto web scanner), the threat actors will conduct a preliminary preattack test to validate their findings using different attack vectors. Some of these application attack vectors such as HTTP GET Flood to attack a vulnerable web page are already part of freely available DDoS attack tools (e.g. High Orbit Ion Cannon (HIOC), BroBot DDoS, DirtJumper, Pandora). The attackers will also inspect the HTTP layer for exploit of vulnerabilities leading to DDoS such as testing with web methods the HTTP web server supports (e.g. POST or GET methods), checking if the web server timeout connections and checking if session creation can be abused to exhaust resources on the web server. After the attacker has conducted a set of preattack tests, it will have identified vulnerabilities and weaknesses in the online banking web application that can be targeted and the specific attack vectors that can be used for exploit them.

2. *Kill-chain step II of the DDoS attack: Weaponization.* After the DDoS attackers have scanned the target web sites for weaknesses and vulnerabilities to exploit and tested the attack vectors against the target, the next step consists of preparing the DDoS attack by configuring and engineering tools and various techniques to attack the web site target. Specifically in this phase, the threat agents will prepare the attack by weaponizing DDoS attack tools with attack vectors that seek to exploit network layer and application layer vulnerabilities for DDoS. Most of DDoS attack tools are botnet-based attack tools and consist of a network of enabled bots that can be coordinated by a command and control center host to flood the targeted online banking web site with malicious traffic. The attack vectors used for weaponize DDoS attack tools seek to exploit both traditional network layer (level 3 and 4 of OSI stack) and application layer (level 7 of OSI stack) vulnerabilities. DDoS attack tools are engineered and customizable to attack specific targets and to conduct a mix of application layer attack on HTTP, HTTPS, and DNS and network layer volumetric attacks on a variety of network protocols (e.g. TCP, UDP, ICMP). Since DDoS attack tools are freely available for download and are written as toolkits in programmable languages (e.g. PHP), they can be easily adapted and customized to include specific attack vectors for DDoS such as UDP, TCP floods, HTTP GET/POST floods, and configured to use proxies to hide the IP-source of the attacks. The weaponization of the DDoS tools can be as simple as configuring them to attack a specific target UR: or IP address and select the various attack parameters

(e.g. port number, protocol) that are part of the exploit. The URLs that are attacked are specifically resource-intensive URLs such as pre-authenticated downloads of large files such as PDF documents, images, and video from the web server. When resource-intensive URLs also include search pages that are not cached and can be abused to exhaust the search engine URLs, these are also weaponized at this stage by injecting iFrames in another compromised website, which pointed to the target online banking site search URL.

3. *Kill-chain step III of the DDos attack: Delivery.* Once the malicious code of the DDoS agent is weaponized and packaged in a Trojan virus or a computer worm, the next step is to deliver it to the target. In this step, the target is the host that the bot master seeks to compromise to become a "zombie." This web server "zombie" will be later controlled by the command and control server and used in a coordinated DDoS attack against the online banking site. The possible ways to deliver DDoS agents on the targets include spam campaigns with malicious code for download and execution as well as use of worms that exploit a particular vulnerability in a system of software and spread through other machines so these will become botnet "zombies" as well.
4. *Kill-chain step IV of the DDoS attack: Exploitation.* In this step of the attack, the vulnerabilities of the hosts where the DDoS Trojan is dropped are exploited to compromise the host and later install the DDoS agent. The malware that is dropped on the host can exploit known CVEs and weaknesses in the secure configuration of the hosts as well as web server vulnerabilities (e.g. iFrame injection, remote file inclusion) that can be used to upload malware with the DDoS agent on the compromised host. The DDoS bot agents can also exploit vulnerabilities in software that is used for managing content on the web server such as Content Management Systems (CMS) software from Joomla and WordPress.
5. *Kill-chain step V of the DDoS attack: Installation.* After the DDoS master has uploaded the malware on the vulnerable hosts, the next step consists of the installation of the DDoS agents and Command and Control Servers so these become part of the DDoS botnet. Once the DDoS agents are installed on the hosts, these are turned into "zombies," which are controlled by the bot master through the Command and Control (C2) servers. The C2 Servers themselves can also be installed on compromised hosts. The installation of DDoS agents includes download of updated malware from the C2 server and installation of backdoors that can be programmed to later receive remote commands and direct the flood of HTTP and UDP traffic to the online banking site targeted. This backdoors of the DDoS agents are written using script programming languages such as PHP and JavaScript and can be controlled remotely by the bot master using APIs. These APIs allow the bot master to configure the DDoS agent and upload updated attack scripts.
6. *Kill-chain step VI of the DDoS attack: Command and Control.* Once the DDoS bot agent is installed on the hosts, they become controlled by the DDoS bot master. To complete the installation of the DDoS bot agent on the host target, the DDoS bot agent will also call home the C2 server to receive the updated attack scripts for the malware to conduct the attacks against the target. In order

to execute DDoS attacks, the DDoS agent need to establish a communication channel with the DDoS Command and Control (C2) servers so that it can receive and execute commands. Depending on the type of C2 architecture being used for the DDoS botnet, the DDoS agent will communicate with the DDoS C2 server through different channels and protocols such as IM, IRC, HTTPs, P2P, and even social media such as Twitter and Facebook. This communication with the C2 server allows the bot master to coordinate and launch the DDoS attack of several DDoS agents against the online banking site target.

7. *Kill-chain step VII of the DDoS attack: Action and Objectives*. This is the step of the attack where the activities originated by the threat agent in control of the DDoS botnet including DDoS agents and DDoS C2 are directed against the online banking site target. Once the DDoS botnet is established, it can be used directly by the bot master that developed it to direct DDoS attacks against the target or it can rented for use by other threat actors. This step of the kill-chain is when the threat agent will attempt to accomplish their intended goal by directing specific attack vectors against the online banking site. These attack vectors are directed to exploit web site vulnerabilities such as SQL injection, Directory Transversal, and XSS as precursor phase to DDoS attacks whose objective is the exfiltration of sensitive customer data (e.g. bank account data credit card data) from the online banking site that is attacked. Following the findings of reconnaissance such as pre-authenticated web pages hosting large files (e.g. a BMP image or a PDF document of 5 MB of size or later), the attackers will send automated simultaneous requests by configuring the HTTP GET Flood attack vector of automated tools to download these files as well as exploit of search engine backend by running repetitive queries.

The DDoS attacks are typically automated and depending on the DDoS attacking tool used and how the DDoS attack bot agent is designed or weaponized, several DDoS attack vectors can be used against the online banking site. Examples of the sequence of attacks using an automated DDoS tool such as BroBot include the following commands and attack vectors:

1. The DDoS bot agents receive commands with the attack instructions in the form of HTTP GET and POST request parameters from DDoS bot C2 server.
2. The DDoS bot agents are instructed to attack the target web server URL with the following attack vectors:
   a. TCP flood at a specific port;
   b. UDP flood;
   c. HTTP GET flood of specific URL/web page (e.g. large file download);
   d. HTTP POST flood of specific URL (e.g. large file upload);
   e. HTTP GET flood with randomized HTTP “User-agents” against a specific URL;
   f. HTTPS GET flood of specific URL.

Other tools used for DDoS attacks include Dirt Jumper that allows the bot master to conduct the following attacks:

1. HTTP GET Flood: causes web server by repeating conventional HTTP requests and as soon as the bot receives the response from the server breaking the underlying TCP connection and sends a new HTTP request
2. HTTP File Downloading Flood: download/request larger (more bandwidth consuming) files (e.g. large pdf files)
3. POST HTTP Flood: send random usernames and passwords (embedded in HTTP-POST packets) to the online banking site
4. TCP SYN attack: sends a TCP SYN request and immediately closes the connection without waiting for the response from the server
5. HTTP Download: In this type of attack, the bot leaves the server in the state of "waiting" after establishing a full TCP connection
6. Socket attack: concentrate on a particular socket on the victim server.
7. HTTP POST of large request (larger than 1 GB) bytes in length with the purpose of congesting the server's upload channel

Once we have captured the attack scenarios for malware account takeover and DDoS attacks, we can map the various phases of the attack kill-chain to various security measures that can be taken prior to the phase of the attack when the attack take place that is the phase of actions and objectives.

Since application threat modeling is first and for most a security engineering activity, it is important to use the information extracted from the attacks such as the attack vectors and the vulnerabilities that are exploited by these attacks to design countermeasures that can detect and protect from these attack vectors. Since these attack vectors change with the emerging threats as well as the type of vulnerabilities that are exploited, the next step consists of documenting the attack vectors and security controls that protect and detect these attack vectors using the analysis of the attack scenario as a reference.

This is the goal of the *AMS activity 6.2 – Update the attack library/vectors and the security controls of the threat-risk specific control/measure framework."* The goal of this activity is to extract the information about the attack events and attack vectors used to conduct the attack scenario.

Once the new attack vectors have been identified, these can be included in an updated attack library/knowledge base. This attack library can be later used for derive security requirements for security controls that need to be designed and implemented as well as to derive security test cases.

Prior to documenting the attack vectors, it is important to analyze from the attack scenario that was previously captured and documented using the kill-chain the various security measures such as detective, protective, proactive, and reactive controls that can be documented in a risk control framework. After these security measures have been documented, the next step consists of extracting information of the various attack vectors that can be used for testing the application resilience to these attack

vectors including exploits of known vulnerabilities and weaknesses associated with these vectors.

As an example of security measures that can be documented using the kill-chain scenarios previously captured for malware attacks is shown in Table 8.13.

By analyzing the attack scenario captured with through the kill-chain of banking malware attacks against the online banking application and the various steps of the malware attack have been captured, it is possible to map to the various steps of the attacks analyzed through the kill-chain to security measures such as detective and preventive controls that can be deployed to monitor, detect, and protect from the attacks.

The standardized method to capture and analyze the attack vectors using the kill-chain are STIX threat schema attributes from threat intelligence such as the "KillChainType" that is attack specific and the "KillChainPhaseType" that is kill -chain attack specific and any "KillChainPhaseReferenceType."

To determine which phase of the kill-chain specific, the malware banking threat observable is and focus on the security measures that can detect, deny/prevent and disrupt the attack before reaches the last phase of the kill-chain, action and objectives that is when the attack is executed against the target. For example, if an observable provided by the threat intelligence is the receipt of HTTP GET of exploit of specific vulnerability and indication of compromise that provide the context that that observable is associated with banking malware Trojan and the kill-chain is phase three that is "Exploitation" the financial institution can undertake actions for the detection of the attack such as:

1. Detect scanning of web application vulnerabilities (CWEs) exploited by that observable (Note is STIX is used the "ExploitTarget" parameter can be used to determine which system or web application vulnerability, weakness, or configuration issue are exploited).
2. Monitor suspicious activities in application log that can be attributed at that event (e.g. as indication of automatic scan of web application vulnerabilities against the target web application).
3. Monitor application layer exploits with Web Application Firewall (e.g. by looking at the specific HTTP GET URL and attack vector used).
4. Look at alerts from host and Network Intrusion Detection Systems (NIDS) (e.g. by looking at the IP of the source of the attack).
5. Error logging and exception handling (e.g. application logs such as web server errors of page not found that can be correlated to the HTTP GET attack).

As preventive measures such as to deny the opportunity for the attacker/threat agent to install the banking malware on the client browser the focus should be on measures such as prioritize the remediation of the vulnerabilities that the attack vector seek to exploit during the malware installation phase such as common vulnerabilities that facilitate the installation of the malware on the client (e.g. XSS) as well as vulnerabilities that can be used to install malware on web server (e.g. iFrames) and

TABLE 8.13 Malware Banking Trojan Kill-Chain and Security Measures

Phase Kill Chain	Detect	Deny	Disrupt	Degrade	Deceive	Destroy
Reconnaissance	Web Analytics Open Source Threat intelligence (OSINT)					
Weaponization	Specific threat Intelligence on this activity (from sting operation, monitoring of threat-agent communications)	Control developer access to online banking business critical source code	Obfuscate source code that might be potentially targeted			
Delivery	Inform bank customers on cyberthreats Enforce visual trust indicators on browser (e.g. EV SSL) Digital sign e-mails from Banks/FIs	Lock access to removable media Control access for third party vendors Filter employee access to social media Filer access to malicious URL/links Filter malicious attachments in e-mails	Coordinated law enforcement-vendor malware/botnet takedowns			

Exploitation	Detect scanning of web application vulnerabilities (CWEs) Monitor suspicious activities in application logs Monitory application layer exploits with WAF Host and Network Intrusion Detection Systems Error logging and exception handling	Fix and patch host, system and web application vulnerabilities (CWEs) at different layers (network, system, technology stack, application)	Data Execution Prevention Enforce minimum privileges (e.g. no admin access)
Installation	Anti-Malware S/W on client Fingerprint known good S/W installations Logging suspicious installation activity	Deny system access to non admin privileges Sandboxing browser and applications Desktop virtualization and containment	Limit access to users systems and applications from known/ suspicious software installations on clients/ servers

(*continued*)

TABLE 8.13 (*Continued*)

Phase Kill Chain	Detect	Deny	Disrupt	Degrade	Deceive	Destroy
Command and control	Adopt APT C2 detection services from third parties Monitor and detect unusual communication channels and patterns (e.g. IRC, P2P, HTTP, IM, Social Media)	Deny access to IM, IRC, Social Media Filter malicious inbound C2 communications Block known HTTP C2 command with WAF	Coordinated law enforcement-vendor C2 server network takedown	Degrade C&C communication to malware agent	DNS redirect	
Actions and objectives	Detect attack malware banking Trojan attack events and banking malware based attack vectors used for exfiltration of online banking credentials and movement of money (refer to kill chain action and objectives analysis) Logging of suspicious activities related to banking malware Fraud detection rules for malware initiated fraud	2-Way Out Of Band (OOB) Authentication OOB Transaction Validation Authorization Maker/Checker Dual Controls for High Risk Transactions (Payments and Money Transfers) Deny user access to business critical functions based on fraud detection rules	Coordinated law enforcement seize of compromised bank accounts, suspend compromise accounts and credit cards, block fraudulent accounts	Degrade Web Response Slow down the attacker with antiautomation	Honeypots Web page decoys	

to gain access to databases (e.g. SQL injection) and later operation of the malware under command and control of the fraudster that facilitate bank account takeover (e.g. session hijacking).

From a proactive risk management approach perspective, that is, to take measures to detect and prevent malware banking attacks before the banking malware is dropped on the client machine, that is, the malware delivery phase, several actions can be taken to improve awareness of online banking customers of specific attacks before taking any action that facilitates malware compromise through browsing of infected web sites (e.g. drive by downloads) and through phishing (e.g. by clicking on attachments and links that are apparently trustworthy but in reality are malicious). For internal employees, banks can take the following measures to protect from malware compromise:

1. Lock access to removable media (e.g. USB devices)
2. Control access for third-party vendors to the internal network
3. Filter employee access to social media (e.g. Facebook, Twitter)
4. Filter access to malicious URL/links
5. Filter malicious attachments in e-mails

These measures should be in place prior to the detection of malware such as banking Trojans trying to compromise bank employees for internal attacks such as specifically to compromise data processes by business critical systems as well as hosts of internal employees who have access to customer and client data such as bank accounts, tax forms with confidential-PII that can be targeted by malware banking attacks.

If the source of infected websites used to deliver malware is known by threat intelligence sources, another action that can be taken by the organization prior to becoming the target of malware banking attacks is to try to take legal action to disrupt the delivery of the malware with drive by download and phishing. This type of activities consists of try to preempt the attacks before these take place by trying to coordinate with law enforcement and security vendor actions against cyber-criminals. Actions that can be taken with security vendors are trying to eradicate malware from compromised web servers that are known to deliver malware as well as to take down services that serve as command and control centers of banking malware and botnets.

Prior to delivery phase, other actions that can be taken are to learn from threat intelligence source if the online banking site is mentioned in underground forums as possible target for the attacks by monitoring threat actor activity in this forums. This might include discussions on how to configure a cybercrime tool kit to attack a specific URL of an online banking site and any discussions in the underground forums on how to weaponize the banking malware and the botnet to attack specific online banking sites.

In the cases when the scripts used by the banking malware are written with attack vectors that attack the online banking application of a specific bank and the specific functionality and web pages, the knowledge of the script is critical to take actions

to test these attack vectors in a testing laboratory and engineer countermeasures to protect the various URLs targeted by these attack vectors. Often the malware attacking tools are engineered to exploit the specific design flaws in the online banking site that could have been very difficult without the knowledge of the source code. One possibility to make more difficult for threat actors to reverse engineer the source code is to obfuscate the source code, that is, to require the source code to be obfuscated.

After the analysis of the attack scenario by using the kill-chain is completed, there are two groups of attack vectors that can be extracted for engineering and testing web applications:

1. Preattack vectors used by the threat actors for the delivery of malware and for the exploitation of vulnerabilities to upload and install the malware on the target system/host/application
2. Attack vectors used to pursuit the actions and objectives of the attacker such as for exfiltration of sensitive data from the web application and for execute fraudulent transactions

Examples of preattack vectors are the ones exploiting vulnerabilities that allow the installation scripts on the client browser such as XSS and remote command execution via SQL injection into web URLs, auto-executed infections, for example, when a user unintentionally browse a website that is infected advertisement/flash video and user executed infections, that is, when user is lured via social engineering to open malicious attachments, deceptive pop-ups, and selecting URLs pointing to malicious URL. According to the 2013 Verizon DBIR (Data Breach Investigation Report) among all the preattack vectors, the majority are direct installed and injected by an attacker. From the perspective of protecting web applications for delivering malware that can attack the application directly to install malware on serves or indirectly by installing scripts on the client, identifying and fixing of vulnerabilities such as SQL Injection, XSS, and XFS need to be prioritized.

These types of web vulnerabilities that can be exploited for installation of malware need be categorized in relation to the specific threat agent and the threat target for the specific phases of the kill-chain that map to kill-chain delivery such as delivery of the payload, exploitation of vulnerability and installation of malware. This mapping of vulnerabilities to threat agents and threat targets was the goal of stage V of PASTA, WVA (Weakness and Vulnerability Analysis) that was intended to be performed prior to this phase.

In the case of malware banking Trojan attacks, since the attack vectors seek to compromise the browser by exploiting vulnerabilities of the browser prior to attack the web application, the mitigation of such weaknesses and vulnerabilities also reduces the opportunity for an attacker to compromise the client prior to attack the web application. The figure herein, for example, shows CVEs that can be exploited by drive-by-download attacks to compromise the client with banking malware (Figure 8.30).

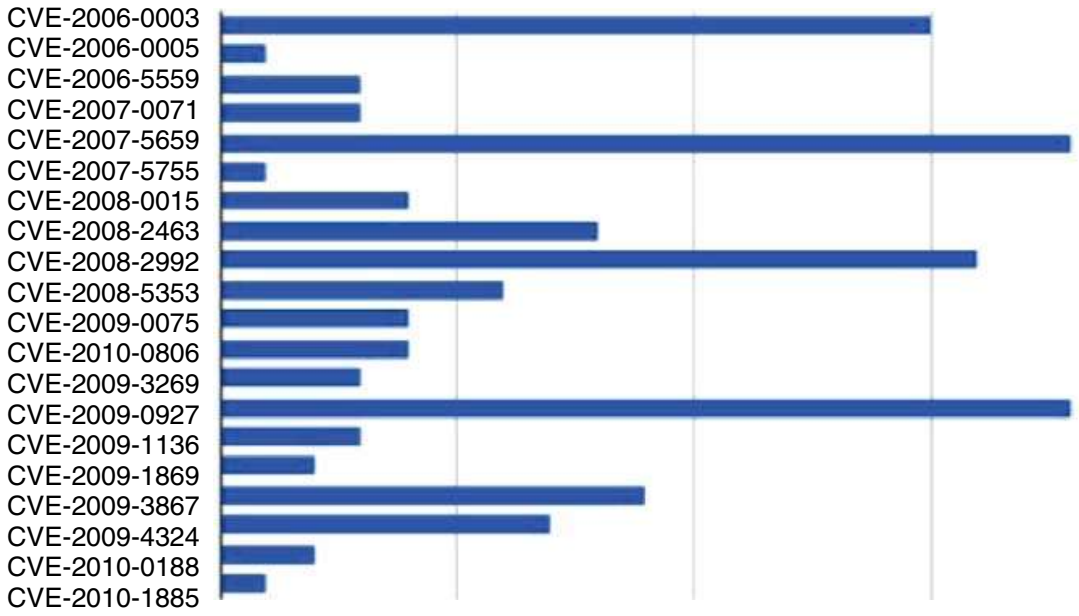


Figure 8.30 CVEs Exploited by Drive-By-Download Attacks

TABLE 8.14 Attack Vectors Used By Banking Malware

	Online Banking Malware Attack Vector(s)
AV1	Capture authentication data from user activity using key logger
AV2	Take screenshots of the user's browser session to defeat virtual keyboard
AV3	Exfiltration confidential data and send to external server
AV4	Redirect to another site URL or IP address
AV5	Update modules/scripts for the malware installed
AV6	Initiate client attacks (XSS, MiTB)
AV7	Initiate web-based attacks (MiTM, Session Hijacking)
AV8	Spoof SMS OTPs sent to fraudster's mobile (MoITM)
AV9	Capture data from volatile memory (e.g. RAM)
AV10	Capture secrets (private keys, certificates) stored on the client device (mobile or desktop)
AV11	Capture sensitive data stored on cashed data/temporary files
AV12	Install backdoor to allow remote access/control
AV13	Listen and execute commands to update malware agent and exfiltration data
AV14	Manipulate money transfer transactions to send money-to-money mule accounts

For the sake of engineering attack resilient web applications, the most realistic assumption is to assume that client PC and browsers are compromised and determine how to detect and prevent malware-based attack vectors to break into the online web application to compromise sensitive data and to conduct fraudulent transactions. A nonexhaustive example of a list of attack vectors used by banking malware is shown in Table 8.14

TABLE 8.15 DDoS Attack Vectors Extracted from the Analysis of DDoS Attacks Against Web Applications

	DDoS Bot Agent(s) Attack Vector(s)
AV1	HTTP GET file downloading flood: causes web server by repeating conventional HTTP requests
AV2	HTTP POST flood: send random usernames and passwords (embedded in HTTP-POST packets)
AV3	HTTP file downloading flood: download/request larger (more bandwidth consuming) files
AV4	Malformed HTTP requests: slowloris attack---
AV5	HTTP download: leaves the server in the state of "waiting" after a full TCP connection
AV6	HTTP POST of large request (larger than 1 GB) bytes
AV7	Slow HTTP POST attack
AV8	TCP SYN flood attack: sends a TCP SYN request and immediately closes the connection
AV9	UDP flood
AV10	ICMP flooding
AV12	Smurf attacks
AV13	DNS attacks (IP spoofing, open recursion, and amplification)

Ideally, each type of cyber threat should have a list of attack vectors that can be queried for and used to derive security requirements and security tests to determine the resilience of the web application to these attacks. As for malware-based attack vectors shown herein, a list of DDoS attack vectors can also be extracted by the analysis of the kill-chain of DDoS and specifically for the Kill-chain step VII of the DDoS attack: Action and Objectives.

In the case of DDoS, these types of attack vectors include both layer 7 and layers 3 and 4 attack vectors that we had previously analyzed and captured during the analysis of DDoS attack scenario using the kill-chain methodology. A nonexhaustive list example of DDOS attack vectors is included in Table 8.15.

One important goal that needs to be achieved during this phase of PASTA is to document attack vectors in attack pattern/libraries that can be actionable for the derivation of security requirements for engineering the online banking application for opportunistic and targeted attacks. This goal can be pursued by adopting a standardized attack library.

An open source standardized attack library that is available today to describe attacks, attack vectors and associated weaknesses (e.g. CWE-IDs) these attack vectors seek to exploit is the MITRE's Common Attack Pattern Enumeration (CAPEC). According to MITRE, "*CAPEC is collection of attack patterns available in the Common Attack Pattern Enumeration and Classification (CAPEC™) initiative, we can help identify opportunities for improving the resilience, integrity, and reliability of our software-based mission capabilities, as well as make our software less susceptible to attack.*"

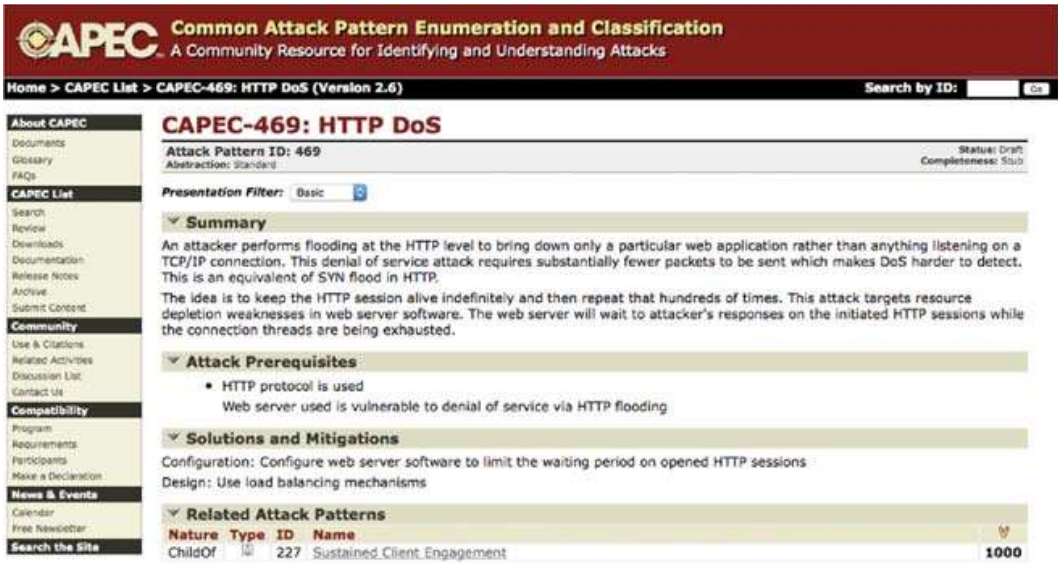


Figure 8.31 CAPEC Attack Pattern for HTTP DoS[5]

The sources of CAPEC are both exploits from proof of concept exploits of vulnerabilities and the research from malware analysis and analysis of attacks observed in security incidents.

In the context of attacks against web applications, CAPEC attack libraries can be used to identify the high-risk weaknesses (e.g. CWEs) that can be exploited by a specific attack and determine a set of security requirements and test cases and tools that can be used to test the resilience of the application against DoS attack vectors.

An example of using standard attack patterns to categorize attacks for HHTP DoS is shown in Figure 8.31.

CAPEC attack patterns also document how to conduct an exploit to test the weaknesses that the attack pattern seeks to exploit. For example, in the case of CAPEC-66:SQL injection, the attack execution flow provides the tester with the attack step techniques to survey the application to determine use controllable input that is susceptible to SQL injection and to exploit the SQL injection vulnerability.

Table 8.16, for example, shows the SQL injection attack steps and the environment that can be replicated in a test to determine whether user-controllable input susceptible to injection.

A positive test of this sequence of event consists of receiving an error message from server indicating that there was a problem with the SQL query. A negative test consists of receiving a normal response from the server. The outcome of this test can be considered positive if at least one user-controllable input susceptible to injection found.

A second series of tests using the CAPEC SQL injection attack sequence consists of a try to conduct the exploit as the attackers do that is to "*add logic to the query to extract information from the database, or to modify or delete information in the database.*" This is shown in Table 8.17.

[5]http://capec.mitre.org/data/definitions/469.html.

TABLE 8.16 CAPEC SQL Injection Attack Sequence 1. Determine User-Controllable Input Susceptible to Injection

ID	Attack Step Technique Description	Environments
1	Use web browser to inject input through text fields or through HTTP GET parameters.	env-Web
2	Use a web application debugging tool such as Tamper Data, TamperIE, WebScarab, and so on to modify HTTP POST parameters, hidden fields, nonfreeform fields, and so on.	env-Web
3	Use network-level packet injection tools such as netcat to inject input	env-Web env-ClientServer env-Peer2Peer env-CommProtocol
4	Use modified client (modified by reverse engineering) to inject input.	env-ClientServer env-Peer2Peer env-CommProtocol

TABLE 8.17 CAPEC SQL Injection Attack Sequence 1. 2. Experiment and try to exploit SQL Injection Vulnerability

ID	Attack Step Technique Description	Environments
1	Use public resources such as “SQL Injection Cheat Sheet” at http://ferruh.mavituna.com/makale/sql-injection-cheatsheet/, and try different approaches for adding logic to SQL queries.	env-Web env-ClientServer env-Peer2Peer env-CommProtocol
2	Add logic to query, and use detailed error messages from the server to debug the query. For example, if adding a single quote to a query causes an error message, try : "’ OR 1=1; --," or something else that would syntactically complete a hypothesized query. Iteratively refine the query.	env-Web env-ClientServer env-Peer2Peer env-CommProtocol
3	Use "Blind SQL Injection" techniques to extract information about the database schema.	env-Web env-ClientServer env-Peer2Peer env-CommProtocol
4	If a denial-of-service attack is the goal, try stacking queries. This does not work on all platforms (most notably, it does not work on Oracle or MySQL). Examples of inputs to try include: "’; DROP TABLE SYSOBJECTS; --" and "’); DROP TABLE SYSOBJECTS; --." These particular queries will likely not work because the SYSOBJECTS table is generally protected.	env-Web env-ClientServer env-Peer2Peer env-CommProtocol

TABLE 8.18 CWEs Exploited in SQL Injection Attacks (CAPEC SQL Injection)

CWE-ID	Weakness Name	Weakness Relationship Type
89	Improper neutralization of special elements used in an SQL command ("SQL Injection")	Targeted
74	Improper neutralization of special elements in output used by a downstream component ("Injection")	Secondary
20	Improper input validation	Secondary
390	Detection of error condition without action	Secondary
697	Insufficient comparison	Secondary
713	OWASP Top Ten 2007 category A2 – injection flaws	Secondary
707	Improper enforcement of message or data structure	

TABLE 8.19 CAPEC-66 Security Requirements For Mitigation of Risk of SQL Injection Attacks

ID	Security Requirement
SR1	Special characters in user-controllable input must be escaped before use by the application
SR2	Only use parameterized stored procedures to query the database
SR3	Input data must be revalidated in the parameterized stored procedures
SR4	Custom error pages must be used to handle exceptions such that they do not reveal any information about the architecture of the application or the database

The outcomes for conducting the exploit test are also described in the CAPEC attack pattern such as success in the case the attacker/tester achieves goal of unauthorized system access, denial of service, and so on or failure in the case the attacker is unable to exploit the SQL injection vulnerability.

The CAPEC attack patterns also describe the application weaknesses that are in scope for the security tests since these are sought to be exploited by the attacker using this attack pattern and attack vectors. This is shown in Table 8.18.

The relevant security requirements and security principles are also documented in CAPEC and these can be used for engineering the web application and make it less susceptible to these attacks. An example is shown in Table 8.19.

CAPEC attack libraries can be integrated in threat modeling tools and assist the threat modeler in creation of security test cases that can be used to identify weaknesses and vulnerabilities that might be part of the legacy online web application that might have been introduced by the architectural and design changes of the application.

The other important requirement is to keep the attack library associated with the assets impacted/targeted and weaknesses and vulnerabilities exposed by the security controls in front of these assets. The association of threat actors, vulnerabilities,

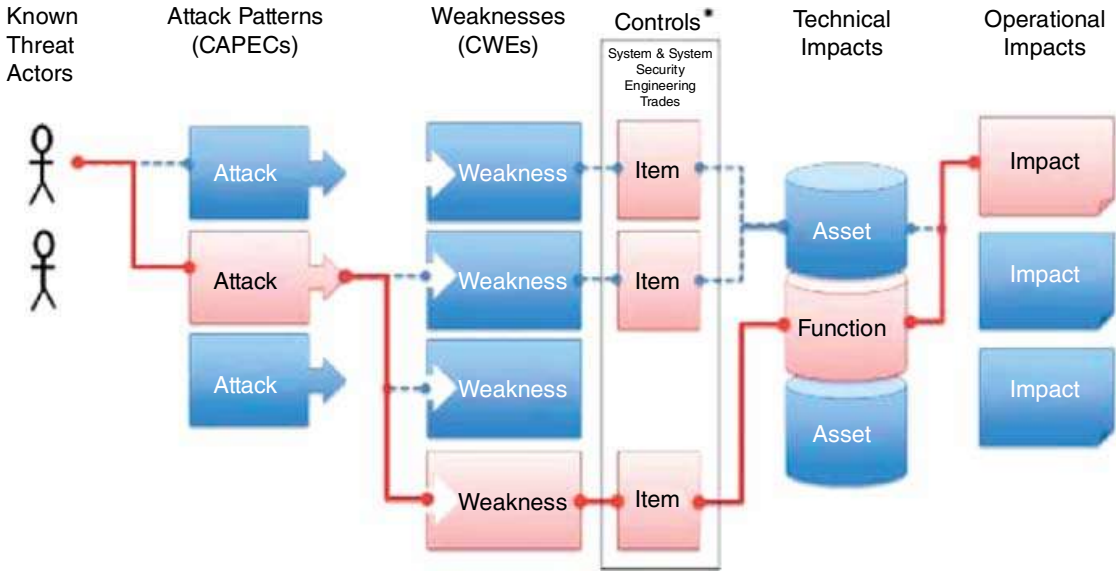


Figure 8.32 Engineering for Attacks Source MITRE[6]

weaknesses, and holistic security controls (e.g. security by design, deployment, and test controls) to cause technical impacts to assets is shown in Figure 8.32.

The standardization of attack patterns is critical for the threat and attack modeling exercise and for the engineering of secure web applications. The use of standard attack library like CAPEC helps security testers to identify vulnerabilities and test the resilience of security controls to specific attacks. These attack libraries should include the attack vectors derived from the analysis of the various type of threats and incidents using external sources such as Information Sharing Assurance Center (ISACs), the Web Hacking incident Data Bases (WHID and well as internal sources such as security logged events from logs/SIEMs, WAFs and honeypots.

Ideally each organization should have a repository of attack libraries that is kept up to date based upon the various sources of threat intelligence, security incidents and research on attack vectors used in attacking tools against the specific target in scope, that is, the online banking application.

Attack libraries can be actionable for the attack-driven engineering of secure web applications when are integrated in a threat modeling platform. An example of WHID attack library that is integrated in a threat modeling tool ThreatModeler™ is shown in Figure 8.33.

In the example shown, the attack vectors map to web hacking security incidents that have been observed against online banking sites and aggregated in the (WHID) Web Hacking Incident Database that is maintained by the (WASC) Web Application Security Consortium organization. These attack vectors are associated with the functional components of the online banking application that were previously analyzed. By enumerating the various attack vectors that target the functional component, the threat modeler can assert that security requirements are documented in architecture

[6]http://cwe.mitre.org/community/swa/attacks.html.

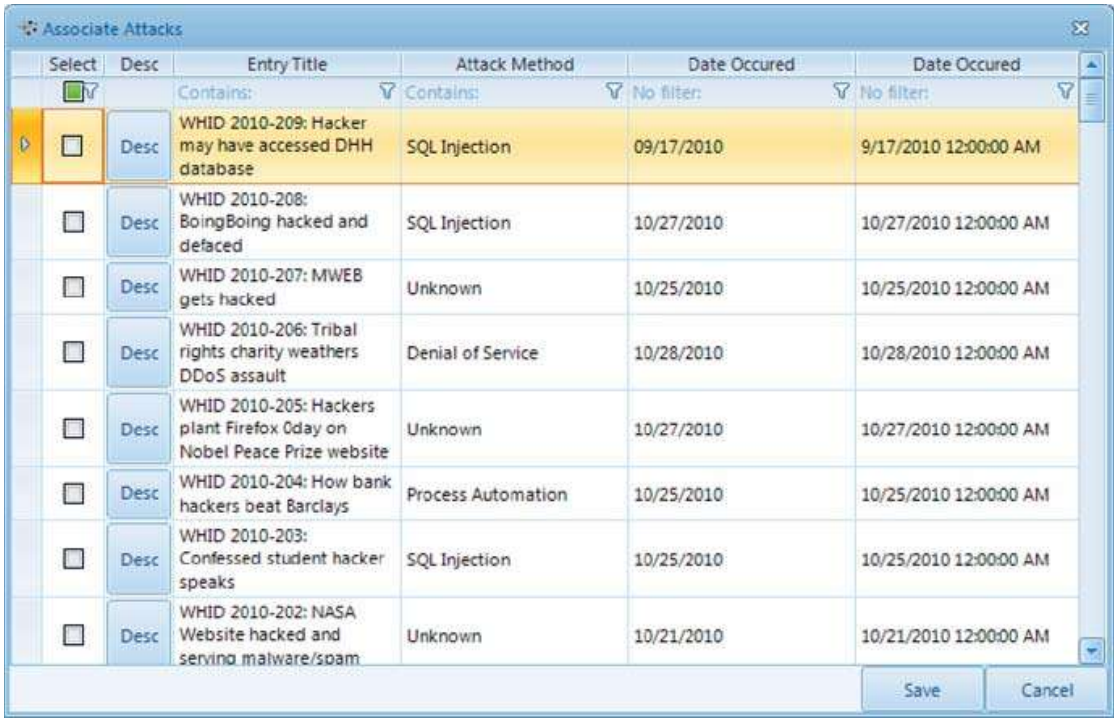


Figure 8.33 WHID Attack Library in ThreatModeler™

design for mitigating the risk of these attack vectors and that security test cases have been created to "close the threats," that is, the various functional components have been tested against these attack vectors.

Once the attack libraries have been updated, the next step to follow consists of *AMS 6.3 – Identify the attack surface and enumerate the attack vectors against the data entry points of the application.*

The main objective of attack surface analysis is to characterize the exposure to the various attack vectors to attack the data that is processed by the online banking application through the various entry and exit points, trust boundaries, vulnerabilities and weaknesses of security controls, and the assets that could be impacted such as data and functions.

To conduct this attack surface analysis, it is important to visualize the previously analyzed architecture diagrams and DFDs including the entry and exit points, the trust boundaries, the assets, and the weaknesses and vulnerabilities and to identify the various security measures and countermeasures in place that act as barriers and challenges to the various attack vectors to conduct exploits. The analysis of the attack surface leverages the information of threat agents (e.g. internal and external) and attack vectors previously analyzed as used by these threat agents for the pursuit of these attacks and exploits.

Ideally a threat modeler should be able to conduct a walk-through of the data from the entry points representing the "attack surface" of the possible attacks and the initial "source" of the attack vector to the final destination or "sink" of the attack vector that produces the desired impact (e.g. compromise of the data asset).

For the sake of this use case, we define attack surface the collection of entry point into the web application that includes both internal and external user interfaces, as well as data interfaces such as the connections among the various tiers of the application architecture. Once the various data entry points are listed, we can associate them with the various security controls in place and enforced at the various trust boundaries such as user authentication and authorization required for access, input data validation, mutual authentication, and data encryption enforced to protect the data and the sensitivity of the data asset that is exposed to the interface including the type of transactions available.

Once the various architectural elements of the attack surface have been captured through the analysis of the architecture diagrams and the DFDs, it is possible to use as measurement of attack risk such as potential exposure and impact of attacks. The attack surface measurement is relative to the various factors that characterize the inherent risk and exposure of the online banking application to potential attacks such as the fact that the application is Internet, intranet, or extranet facing application, has value assets (e.g. confidential data) and functions (e.g. payments, wire transfers) being exposed to the Internet users, integrates with other applications such as through SSO/federation, support different type of devices (e.g. web and mobile), and integrates with business critical backend services (e.g. SaaS, web services)

Once the organization has decided what architectural component best characterizes the attack surface of the web application, it is possible to use as a relative measure of risk being introduced in the design such as addition of new user and data interfaces that might increase the risk of attacks such as new avenues and potential impact from attacks.

An example of capture of attack surface for online banking application based on the same architectural diagrams and DFDs used for previous stages of PASTA (Figures 6.1 and 6.2) is shown herein in Table 8.20.

In the example shown, we have defined the attack surface for the online banking application as baseline and compared with application changes that were introduced to determine the increased risk such as additions of new URLs, web services, and devices. In the example shown, the attack surface has increased as well as the attack risks due to the addition of pre-authenticated URLs to download files and videos (e.g. large PDFs and flash video streaming) and to conduct web searches. These changes represent an increased risk of attacks from threat agent-actors to cause denial of service by running automation scripts that direct a flood of HTTP GET request to download these files and videos for upstream pipe saturation DDoS and for exploitation of search engine vulnerabilities.

The other increased risk due to the increased attack surface is caused by the support of mobile banking through thick (smartphone applications) and thin (web based) mobile clients. This increased risk can be measured as increased exposure to malware attacks specifically targeting mobile devices if bank customers with mobile banking applications installed.

Additional changes that increase the attack surface for the online banking application and the risk of attacks are the deployment of new Web 2.0 features such as widgets components using AJAX since these new components might introduce new

TABLE 8.20 Attack Surface of Online Banking Application

Online Banking Application Attack Surface Changes			
Component	Trust Zone	Application Baseline	Application Features/ Changes (New Release)
User interfaces	Internet	Pre-authenticated URLs-Visitors Post-authenticated URLs-Auth-Users	Pre-authenticated URL-Visitors-File Download Pre-authenticated URL-Visitors-Search API Queries
User devices	Internet	Web Client Browsers	Mobile Thick Client Mobile Thin Client
Data interfaces	DMZ	Web Client-Web Server-HTTPS	Web2.0 Client (Flash, Ajax)-Mobile Web Gateway-HTTPS
	Internal	Web Server-App Server-HTTPS AppServer-DB Credential Store-JDBC-SSL AppServer-Risk Base Authentication-HTTPS-XML AppServer-Enterprise Messaging Bus-SOAP-HTTPS Enterprise Messaging-Financial Processing Mainframe-MQ-SSL AppServer-Fraud Detection Service–HTTPS-XML Appserver-Secure File Transfer Server-SFTP	Mobile-Web Gateway-Mobile Banking Application HTTPS Mobile Banking Application-SOAP-HTTPS Web Server-Search APIs-SOAP-HTTPS Web Server-Secure Token Service-HTTPS-XML
Admin interfaces	Internal	Post-authenticated URL-Admin-Users-Intranet	Post-authenticated URL-Admin-Users-Intranet

and existing vulnerabilities both in the client and on the server and additional avenues for attacking the online banking application from the Internet.

The aggregation of other financial services websites through Single Sign on/Federation also increases the attack surface because the increased number of users can now single sign on and can be potentially the target of malware banking attacks and the increased number of financial transactions are available for attackers that were not available prior to the SSO to these financial sites. For example, when before the services that could be targeted only included services provided by the

bank, the aggregation of other financial services through sign on (e.g. electronic bill payment services, credit card payment services etc) increases the risk of these services of being attacked by one entry point, that is, the user interface to SSO to the online banking site.

In addition, the new data interfaces that are accessible only through the internal network such as web services that expose the backend financial services through the mobile client through the common enterprise messaging bus also represent an internal threat agent attack risk; hence, these need to be secured by ensuring that requests of internal connectivity to these interfaces are repudiated for internal host that cannot be mutually authenticated to these interfaces by enforcing mutual certificate authentication for all web service calls as an example.

Once the attack surface is measured and compared with the baseline, it is possible to make informed decisions such as reducing the attack surface and the risk that is to minimize the risk exposure of the assets to the various threat agents and attack vectors and vulnerabilities that these threat agents could exploit in a targeted or opportunistic attacks against the online banking application. Examples include moving CPU resource-intensive URLs with static content such as videos and files that can be targeted by DDoS attacks to third-party CDN (Content Delivery Networks as well as by caching pre-authenticated web searches that use the Search APIs so these cannot be exploited.

To reduce the risk of additional vulnerabilities that might be introduced by Web 2.0 client components, the organization can introduce additional software assurance services such as application threat modeling, source code reviews/scans and pen tests specifically for threats targeting these components. Attack vectors specifically exploiting Web 2.0 vulnerabilities such as the ones categorized by WASC as Web 2.0 threats need to be incorporated and augment the current application vulnerability assessment/tests.

For the increased exposure caused by the use of mobile devices and online banking applications that are installed on mobile devices to transact with the bank, additional security engineering processes need to be enforced for reviewing and certifying the secure development and provisioning of these online banking applications. Specifically since mobile devices and applications are primary target for malware attacks including for attacking online banking applications as well as online banking credentials, specific security measures and controls need to be engineered and deployed to reduce the exposure of attack vectors targeting both mobile devices, desktops, and online banking applications such as MoITM Mobile-In-the-Middle attacks (e.g. attacks that seek to compromise the mobile device when used for out-of-band one-time password verification and authorization). Specifically mobile-based online banking applications need to be in scope for testing with attack vectors that are specific for attacks targeting weaknesses and vulnerabilities that are inherent of the mobile O.S (e.g. Android, IoS) where the mobile application is installed and the provisioning process of the application on the device (e.g. AppleStore and Android marketplace).

The example of attack surface measurement for the online banking application in scope that is provided herein is meant to capture the risk exposure to the various

attacks and attack vectors at high architectural design level. The approach considered is from the perspective of the attacker's opportunity as this being an external threat agent seeking to attack the online banking application remotely for stealing confidential data such as online banking credentials, credit card data, and bank account data, committing fraudulent transactions such as by stealing money from the victim's bank account through banking malware and distributed denial of service both as diversion attack and for reputational damage.

At lower level, that is, detail functional design including source code, the attack surface might consider specific attacks against application software, processes, and permissions to execute these processes including open ports, named pipes, PRC endpoints, null sessions, installed services, services running defaults and as System, and files and permissions set for these files. Once these parameters are measured as is done in Microsoft's (RASQ) Relative Attack Surface Quotient, they represent an opportunity to measure improvements in security of products. At the source code level, that is, the analysis of attack vectors that can be used to exploit coding issues, the attack surface can also be used to determine the increased risk represented by the introduction of changes in source code and make defensive coding decisions on exposing API calls to internal service calls and access to input data and data files that might represent an avenue for specific attack vectors.

In the case of Microsoft products, at Operative System Level, the attack surface can be analyzed using the attack surface analyzer to determine if system misconfigurations such as weak ACLs, installing services, changing firewall rules and so on that might expose a Window's System installation to specific threat agents and attack vectors.

From the attacker perspective, the online banking application attack surface is represented by all possible attack vectors and avenues at disposal of the attacker that include exploit of weaknesses and vulnerabilities at the various layers of the technology stack including the internal and external networks, the various systems and hosts that the web application is designed to be installed and run. By keeping this in mind, it is important that whenever a new change is introduced to the application being this just a new configuration change, system upgrade changes, connectivity changes, and last but not least new services and features added, it is reviewed for the additional risks that are introduced as exposure of new attack vectors specifically targeting vulnerabilities and weaknesses introduced by changes in design, coding, and configurations.

Once we have documented the attack surface of the online banking application and captured at high level, the exposure to potential attacks for the design, implementation, and configuration changes that are introduced, the next step consists of estimating the probability of these attack vectors to cause an exploit and determine the desired impact. This is the goal of the activity *AMS 6.4 – Assess the probability and impact of the exploit of each attack scenario*. At high level, attack probability, that is, the possibility of an attack to cause an impact can be estimated using attack trees.

Attack trees are formal attack models that threat analysts can use to analyze the probability of the attacker's goals to be realized by considering the various tools, techniques, and opportunity for exploit at disposal to the attacker. For an attacker, a

user interface, data interface application weakness or vulnerability including server or application mis-configuration represent an opportunity and an avenue for an attack. Now the question for an attacker is, given various opportunities for attacking the web application as well as TTPs Tools, Techniques and Processes required to achieve the goal, what is the attack path that is most likely to succeed and cheaper in terms of costs to bear to acquire tools, techniques, and familiarize with attack processes? For example, in the case of a threat agent such as a cyber-criminal whose business is to create malware such as banking malware that can be sold or rented to other cyber-criminals to steal money from their bank account, the main question is which vectors of attack lead to the ultimate goal that is to create a cyber-crime tool that can exploit various types of weaknesses and vulnerabilities to achieve the attacker's goal, that is, stealing money from bank accounts. Attack tree analysis of attacks is what malware writers use to walk through the various possible exploits including social engineering exploits, use of phishing scam tools, exploit of vulnerabilities in client browsers, web application vulnerabilities, and security control weaknesses such as weak user authentication and multifactor authentication that is currently used by banks.

The main objective of the malware writer is to engineer a cyber-crime tool kit that is both sophisticated and flexible enough to bypass common security controls deployed by most banks and adapted for dissemination using different delivery methods and to conduct different type of attacks such as MitM and MitB attacks. By writing such cyber-crime tool, cyber-criminal will also write procedural instructions to both operate the tools and to commit the fraud. For example, to escape most of money laundry controls in place at banks to prevent moving money to bank accounts black-listed as fraudulent by (AML) Anti Money Laundry controls fraudsters figure out that the proceeds of fraudulent wire transfer can be sent first to "money mules" in essence person that can transfer money acquired illegally through a courier service or electronically on behalf of the fraudster for a fee. As part of the cyber-crime scheme that involves using banking malware money mules are hired through work at home schemes.

From the defender perspective, conducting the same type of analysis of the attacker using an attack tree to analyze how the attacker goal can be realized and the various possible avenues to conduct the attacks, this possible avenues can be analyzed by assigning conditions to each node of the attack tree as either "OR" or "AND" conditions. For example, a threat agent seeking to compromise the online banking application using a banking Trojan can attack the client "AND" attack the application "OR" attack the application directly. Given that the probability of attacking the client is lower than attacking the application directly, it will choose that attack client path.

The probability of success is given by the consideration of all single probabilities of each node whose conditions need to be fulfilled in order for the attacker to reach his goal. The attack tree model of banking malware attacks is shown in Figure 8.34.

The attack path of banking malware attack threat agent and the coordinate action of the fraudster agent are shown herein. The path of the malware attack that is followed by the banking malware is shown in the left of the attack tree while the manual steps of the attack that are followed by the fraudster are shown on the right of the attack tree.

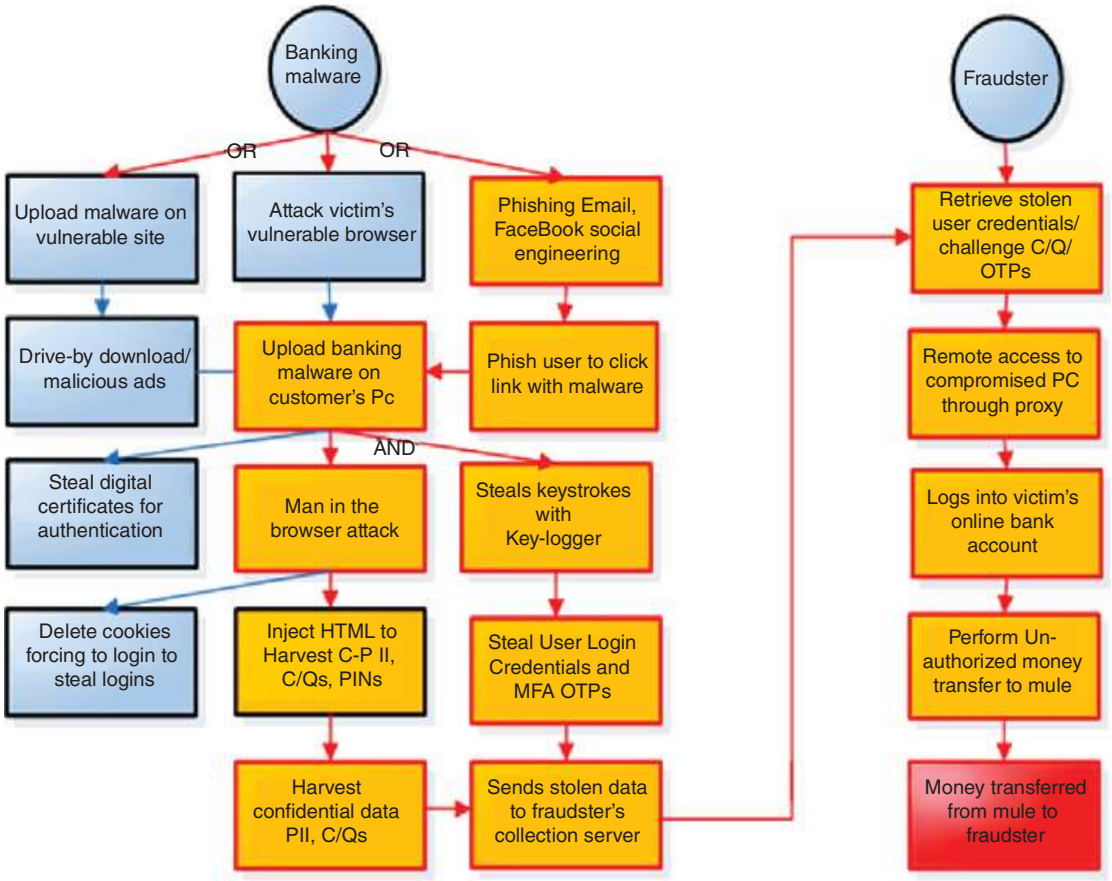


Figure 8.34 Banking Malware Attack Tree

At root of the attack tree the malware attack can use any of the attack delivery methods considered in the branches of the first node such as (1) upload banking malware on vulnerable website with direct attack (e.g. IFrame injection), (2) attack the client directly by remote exploit of browser vulnerabilities, or (3) perform the social engineering attack to phish the use to click on links with malware.

The assumption is that the social engineering attack is the attack vector of choice for uploading malware on customer's PC. From threat intelligence, it is known that banking malware spreads mostly either by drive by download or by phishing. The assumption that attackers seek to compromise the targeted host by luring customers with phishing e-mails is the assumption that the weakest link in every attack is always the human element and therefore social engineering and phishing represent a low-cost, high-gain opportunity for attacker to spread malware. The attack tree shown in Figure 8.34 shows in each leaf the various possible steps that a fraudster could follow in the pursuit of his final goals that are harvest confidential data and credentials from the victim and commit account take wire fraud. Once the malware is installed and the client host machine is uploaded, installed, and configured on targeted client host, it will execute the various attacks such that it is designed to harvest confidential data from the bank customers using MiTM, stealing credentials by logging keystrokes. After the malware has sent the stolen online banking site credentials and customer sensitive data to the fraudster, it will wait to execute

commands from the fraudster's controlled command and control server. The path that is followed by the fraudster includes a series of steps such as remote connecting to the compromised host through a proxy to anonymize his actions, taking over the customer account by hijacking the session, manipulate the transaction to move money to a money mule account and least but not last have the money mule sending the stolen money to the fraudster's account (typically in another country).Once the path of the most probable attack for the banking malware threat in scope has been analyzed, the next logical step of the threat analysis is to derive test cases for simulating the attacks and understand how the exploit occurs and specifically to identify which vulnerabilities and weaknesses are exploited and security controls are bypassed to realize the threat.

This leads to the next activity of the attack modeling that is *AMS 6.5 – Derive a set of cases to test existing measures*. These test cases can be derived with the knowledge derived from the analysis of the attack scenarios including the kill-chain sequence of event that lead to the attack operation and execution, the update of the attack library with the attack vectors learned from the attack scenarios, the analysis of the attack surface to determine the possible avenues of attack vectors from the perspective of the attackable data entry points, and the analysis of the attack trees to determine the most probable attack paths that are pursued by the attacker to reach his goals.

For the derivation of the security test cases to simulate the attacks, two criteria can be used:

1. Test cases for testing the several weaknesses (CWE) and vulnerabilities (CVEs) that attack vectors seek to exploit during the various phases of the attack kill-chain (e.g. using ThreatTarget STIX mapping to CVEs and CWEs) and to conduct the attack (e.g. using CAPEC-CWE mapping).
2. Test cases for testing the possible abuses of security controls including the bypassing of multifactor authentication and traditional client-based security measures (e.g. browser security, antiviruses, and antispyware).

For the first group of test cases that are attack driven, the derivation of the security test cases builds upon the analysis of the kill-chain derived security requirements. These security requirements are different from the security requirements that were written in stage I in response to information security policies and compliance as well as for secure the online banking application functionality including online banking transactions and for protection of the confidentiality, integrity, and availability of sensitive customer information. These security requirements are validated with functional driven test to validate that security controls function as is "accepted" by design, implementation, and configuration.

The security test cases that are documented in this stage of the attack modeling and simulation consist of attack-driven security test cases whose goal is to verify the resilience of the application when attacked with the same attacking tools, techniques, and procedures used by the threat agents being this both automated and human threat agents.

Through the analysis of the attack scenarios of malware banking and DDoS threats by using the kill-chain method, security measures can be identified to detect, deny,

disrupt, deceive, and destroy the malware-based and DDoS-based attacks. Specifically, the effectiveness of these security measures in mitigating the attacks need to be validated in simulated attack exercises. The attack vectors that are extracted from the analysis of the attack scenarios and specifically also from sources of threat intelligence, root cause analysis of security incidents and malware analysis/reverse engineering are instrumental to update information on the security measures that need to be validated in order to detect and protect from these attack vectors. Security measures and controls that are known to be effective in mitigating the risk of these attacks can be uploaded for each attack pattern as shown by using standard attack patterns such as CAPEC or by using threat modeling tools such as ThreatModeler™ where attacks, security requirements for controls and measures, and test cases are mapped to each functional component of the online banking application.

After these security controls and measures are documented in design and implemented in the online banking application, they might still be affected by weaknesses and vulnerabilities that these specific attacks seek to exploit; therefore, a list of specific test cases to test these vulnerabilities and weaknesses should be created. The various weaknesses and vulnerabilities that threat agents seek to exploit were also identified by threat intelligence as threat target-vulnerability mapping and analyzed in stage IV weaknesses and vulnerability analysis to prioritize security tests. These tests seek to identify the presence of vulnerability but not necessarily for security assurance perspective and are compliance driven. The security tests that are derived at this stage are realistic attack driven and are aimed at conducting an exploit of the vulnerability with the realistic attack vectors used by the attackers to produce the same impact. For this reason, it is important to scope vulnerability and weaknesses test for exploit using the same attack scenario and the same attack vectors used by the attackers.

Attack libraries and patterns are key for attack-driven security tests and assist in defining the behavior of the online banking application when is attacked and to prevent or to react to a specific type of likely attack.

From the development perspective, attack libraries help to identify the positive security requirements for designing security controls such as authentication, authorization, input validation, and application functionality, and validate that are resistant and resilient to the specified attacks such as malware banking and DDoS.

Attack libraries also help to identify appropriate negative security requirements (misuse/abuse cases) to specify the behavior of the online banking application when faced with the specified attacks. Negative security requirements can be contained in each attack pattern by fleshing out the details of the various steps for misuse or abuse of functionality that are undertook by the attacker and help to identify weaknesses in security controls and missing security functionality that could enable such an attack. The methodology that can be used to derive negative security requirements following an understanding of the attack scenarios, attack libraries/patterns, and attack tree analysis includes misuse and abuse cases.

Figure 8.35 shows an example of use and abuse cases to derive negative security requirements to strengthen the design of MFA Multifactor Authentication for the online banking application. On the left-hand side of the abuse case diagram are

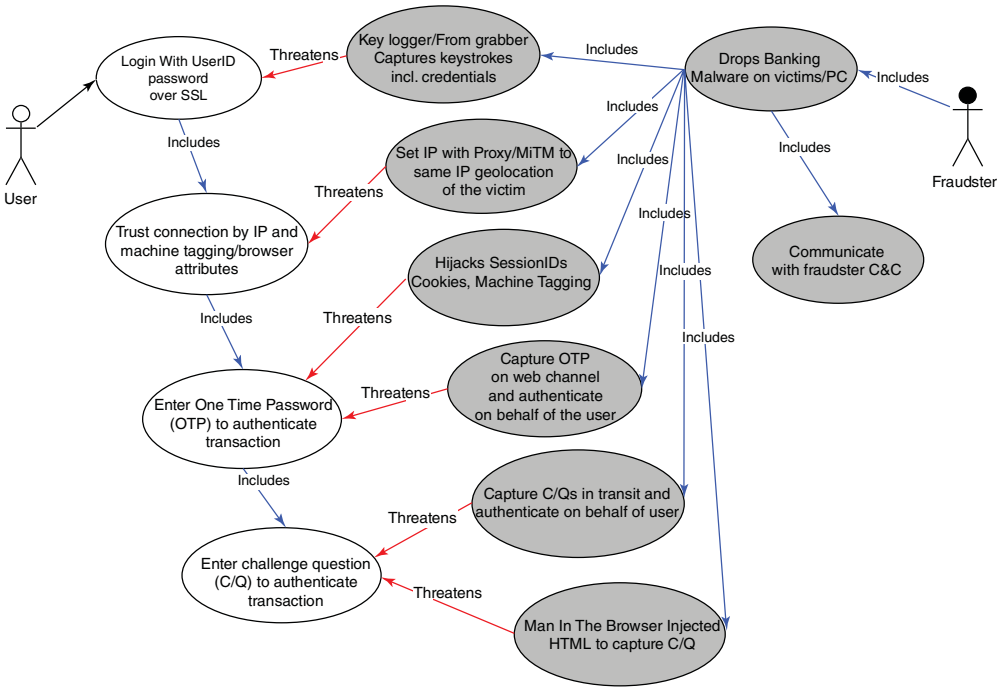


Figure 8.35 Use and Abuse Cases for MFA Controls

depicted the various steps/use cases followed by a bank user when logging to an online web application using standard MFA controls deployed by a bank. On the right-hand side of the diagrams are the steps followed by a fraudster that "threaten" to compromise the MFA functionality for bypassing the log-on authentication steps.

In the example shown in Figure 8.35, the fraudster controls the banking malware dropped on the client host and initiates a set of abuses of MFA that are learned from the malware banking attack scenario previously analyzed. From the abuse case analysis, it is analyzed how the attacker can abuse both single authentication credentials and MFA controls including, risk-based authentication factors such as IP geo-location, machine tagging, and MFA used to authenticate money movement transactions such as One-Time Passwords (OTPs)and Challenge/Question (C/Q) factors, one time passwords.

Based upon this analysis, it is possible to derive missing security requirements and test cases to protect user logging as well as test the threat resilience of multifactor authentication controls to these types of attacks.

Based upon the analysis of the attack tree as an example, the following Security Test (ST) cases can be derived for malware banking-based attacks:

1. *ST1-Malware-delivery-web-based attack test:* Test the security controls of the online banking application using attack vectors that seek to exploit vulnerabilities and exploits used for dropping banking malware on the client such as, XSS, XFS, and iFrame injection exploits.
2. *ST2-Malware-delivery-client-based attack test:* Test the client hosts for exploit of browser vulnerabilities in the executable environment of the client host/PC that are used for drive by download and phishing such as browser plug-ins, adobe and macromedia flash and active X controls.
3. *ST3-Malware-action on objectives attack test:* Test the impact of malware banking attack actions targeting online banking application for vulnerabilities and weaknesses assuming compromise of malware on the client to attack. Attacks vectors to be used for testing the online web application include MitM, MiTB, and session hijacking attacks. These exploit of weaknesses by these attack vectors such as lack of mutual authentication, expired or insecure certificates and cryptographic storage, lack of (OOBA) 2-way Out Of Band Authentication, session fixation and hijacking, and authentication weaknesses (Note the complete list of CVEs and CWEs is extracted from the mapping of threats to vulnerabilities and attacks that are documented in the attack libraries and patterns).

At more detailed level, attack-driven security tests can be derived from the previously defined attack vectors used in malware-based attacks as shown in Table 8.21.

Similarly for DDoS attacks, the following attack-driven test cases can be derived based upon the analysis of the attack vectors used by DDoS agent attack tools. An example is shown in Table 8.22.

Attack-driven tests validate the realization of the attack to accomplish their intended goal of the attack such as stealing confidential information and online banking credentials of a customer, manipulate money transfers, and steal money

TABLE 8.21 Malware-Based-Attack-Driven Security Test Cases

Online Banking Malware Attack Vector(s) and Security Test Cases			
Attack Vector	Attack Description	Security Test Case	Test Case Description
AV1	Capture authentication data from user activity using key logger	ST1	Test effectiveness of antimalware software and sandboxing/ virtualization to detect and prevent malware installation on host PC/workstations
AV2	Take screenshots of the user's browser session to defeat virtual keyboard	ST2	Test capability of frame grabbing malware to capture virtual keyboard keystrokes if effective test alternative methods for trusted UI and OOBV
AV3	Exfiltration confidential data and send to external server	ST3	Test detection of unauthorized HTTP traffic from/to internal host to outside the company network
AV4	Redirect to another site URL or IP address	ST4	Test for invalidated redirect and forwards and filtering of known URLS used for malware delivery at corporate gateways
AV5	Update modules/scripts for the malware installed	ST5	Test detection and filtering of command and control traffic to malware compromised clients
AV6	Initiate Client Attacks (XSS, MiTB)	ST6	Test for client attacks vulnerabilities (CVEs) and weaknesses (CWEs). Test effectiveness of MiTB defenses such as antimalware, client sandboxing virtualization and OOBA
AV7	Initiate Web-based Attack (MiTM, Session Hijacking)	ST7	Test for CVEs and CWEs that facilitate MiTM and Session Hijacking
AV8	Spoof SMS OTPs sent to fraudster's mobile (MoITM)	ST8	Test resilience of SMS OTPs to MoTIM attacks
AV9	Capture data from volatile memory (e.g. RAM)	ST9	Test device memory protection controls and crypto routines
AV10	Capture secrets (private keys, certificates) stored on the client device (mobile or desktop)	ST10	Test secure storage of secrets (keys, authentication data) and validated against weaknesses and vulnerabilities of key management and encryption
AV11	Capture sensitive data stored on cashed data/temporary files	ST11	Test for caching of confidential and authentication data
AV12	Install backdoor to allow remote access/control	ST12	Test for user access restrictions (authN and authZ), unauthorized communication ports and audit and logging enabled for suspicious

(*continued*)

TABLE 8.21 (*Continued*)

Online Banking Malware Attack Vector(s) and Security Test Cases			
Attack Vector	Attack Description	Security Test Case	Test Case Description
AV13	Listen and execute commands to update malware agent and exfiltr$$$	ST13	Test filtering of C2 commands to download malware package updates to internal servers using various communications controls (IM, IRC, HTTPs, P2P)
AV14	Manipulate money transfer transactions to send money to money mule accounts	ST14	Test for impersonation/account take over through session hijacking to tamper money transfer parameters. Test effectiveness of MFA controls deployed (e.g. H/W OTPs, OOB SMS OTPs, C/Qs, RBAs)

from bank accounts and take off the online banking site with denial-of-service attacks.

These specific attack-driven test cases can be added to threat-driven security test cases. The difference between attack-driven and threat-driven test cases is that threat-driven test the realization of a threat into an attack whose successful validation of the test is to ascertain that the threat can be realized.

For example, the following security test cases are generic threat driven using the STRIDE (Spoofing Tampering Repudiation Information Disclosure, Denial of Service and Elevation of Privilege) classification:

1. *ST1-Spoofing Identity:* Test attempts to impersonate a user by sniffing user credentials on the wire or in persistent storage.
2. *ST2-Spoofing Identity:* Test that security tokens (e.g. cookie) issued before authentication cannot be replayed to bypass authentication.
3. *ST1-Tampering data:* Test with a web proxy that is not possible to tamper and replay authentication and sensitive data.
4. *ST2-Tampering data:* Test strength of password hashes against replay attacks (e.g. use of salted hashes).
5. *ST1-Repudiation:* Test mutual authentication and/or digital signatures to enforce trusted connection between client and server and between server to server/point to point connections.
6. *ST2-Repudiation:* Test that all security events are audited and logged.
7. *ST1-Information:* Disclosure: Test that the access of nonpublic data requires authentication.
8. *ST2-Information:* Disclosure: Test that error messages and exceptions do not disclose useful information to an attacker.

TABLE 8.22 DDoS Attack Driven Security Test Cases

DDoS Bot Agent Attack Vector(s) and Security Test Cases			
Attack Vector ID	Attack Description	Security Test ID	Test Description
AV1	HTTP GET File Downloading Flood: causes web server by repeating conventional HTTP requests	ST1	Test Resilience of Web Server to HTTP GET Flooding Attacks Pre-Authentication and determine degradation/denial of service/CPU
AV2	HTTP POST Flood: send random usernames and passwords (embedded in HTTP-POST packets)	ST2	Test Resilience of Online Banking Application Against HTTP POST flooding targeting login web pages determine degradation/denial of service
AV3	HTTP File Downloading Flood: download/ request larger (more bandwidth consuming) files	ST3	Scan online banking application for pre-authentication URLS that allow to download videos and large PDF files. Use HTTP GET flood testing tools to flood request to download videos and large PDF files and determine degradation and denial of service
AV4	Malformed HTTP Requests: Slowloris attacks	ST4	Test Web Server for HTTP GET requests with malformed HTTP headers to determine degradation/denial of service
AV5	HTTP Download: leaves the server in the state of 'waiting' after a full TCP connection	ST5	Test HTTP download attacks used by DDoS attack tools such as Pandora and DireJumper against web server to determine degradation/denial of service
AV6	HTTP POST of large request (larger than 1GB) bytes	ST6	Test effectiveness of filtering of large HTTP request (e.g. limitation of max size of HTTP requests at web server NSAPI filters as well as at Application Server servlet filters)
AV7	Slow HTTP POST Attack	ST7	Test resilience of web servers against sloe HTTP Post attacks
AV8	TCP SYN, UDP, ICMP flooding attacks	ST8	Test data center network devices resilience against traditional L3 and L4 DDoS attacks (e.g. TCP Syn flood, PING flood, UDP Floods etc.)
AV9	DNS Attacks (IP spoofing, open recursion and amplification)	ST9	Test for attacks against DNS server including DNS amplification attacks exploits

9. *ST3-Information*: Disclosure: Test that processes do not cache or log passwords, session information, or PII.
10. *ST1-Denial of Service:* Test that you cannot flood either a process (e.g. HTTP GET and POST requests to the web server) or a service (e.g. web service calls) with so much data so that it stops responding to valid requests.
11. *ST2-Denial of Service:* Test that malformed data (e.g. malformed HTTP headers, XML files etc) cannot crash the process.
12. *ST1-Elevation of Privilege:* Test that user cannot tamper client parameters or forceful browsing to elevate his privileges.
13. *ST2-Elevation of Privilege:* Test that a process cannot be forced to load a command shell, which in turn will execute with elevated privileges.

Threat-driven security tests can be associated with component of the application architecture and security control that is changed/modified to determine the exposure to generic threats.

Threat-driven test cases can also embedded in a threat modeling and tool and retrieve to validate threat mitigation at each component, that is, to assert that the risk of the threat is mitigated by security requirements asserted by review of design and security test cases executed to validate these requirements.

In Figure 8.36, we show an example of security test cases that can be extracted for each functional component of the online banking application and mapped to threats, security requirements, and code snippets that serve as reference to conduct a source code review to assert the implementation of these security requirements in the source code.

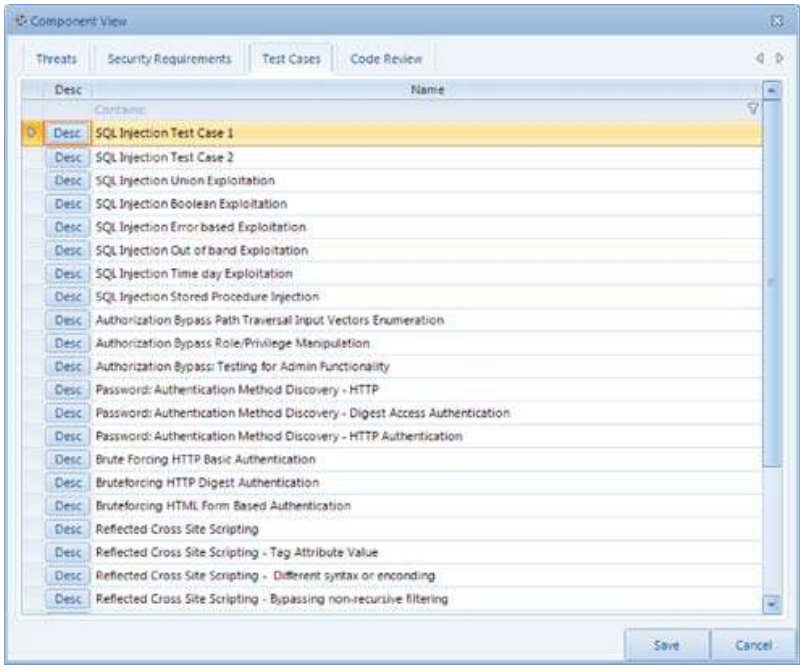


Figure 8.36 Threat-Level Security Test Cases

Once various test cases have been documented, the next and last stage of attack modeling and simulation stage is to conduct an attack simulation exercise for the threats considered that includes, in this case, malware banking and DDoS attacks. This is the goal of the activity *AMS 6.6 – Conduct attack driven security tests and simulations.* The scope of these simulation attack exercises is to identify vulnerabilities and design flaws that are exploited by the attack and determine the risks as factor of probability of attack and the impact of these attacks but also to identify any weaknesses and vulnerabilities that these attack exploit and to learn how to respond in the case of security incidents caused by these attacks.

Prior to execute these simulation attack exercises, it is important to document an attack-driven testing strategy that can be followed by the organization with specific goals that need to be met. An example of goals for an attack-driven security testing strategy include the following:

- Identify the most likely attack course of action against your web application and your customers (difficulty of attack is the least and the impact is the highest);
- Identify the attack surface and the attack vector entry points;
- Identify the access levels required to conduct the attack;
- Test for vulnerabilities and design flaws that these attacks could exploit as used in real case scenarios;
- Test for abuse of functionality/business logic by deriving test cases using use and abuse cases;
- Test the same attack vectors used by the attack tools to try to evade security controls;
- Test defenses at the different layers of the architecture to validate the defense in depth against threats identified in the application threat model.

At the end of the execution of the attack simulation and test exercise, it is important to produce an attack simulation test report for the threat scenarios considered that includes the results of the security test cases and the identification of the exposure of vulnerabilities to these attacks as well as bypass of security controls to cause an impact. The attack simulation report includes the results of the security tests and the security issues identified and the risk of each security issue (design flaws, web application and system vulnerabilities, weaknesses in security controls).The risk severity of each security issue can be estimated based upon the probability of the attack to cause an impact and can be determined based upon the review of the results of the test/simulation.

The results of the security tests cases can be mapped to the various threat agents to determine if the risk of threats is mitigated. Some of these threats might be considered unmitigated and open because of the presence of weaknesses and vulnerabilities in security controls and therefore considered of a certain risk. This risk of these threats can be calculated based upon risk calculation formulas to determine the probability, and the risk severity of the vulnerabilities can also be calculated based upon different scoring methods such as CVSS. These final calculations of the level of risks for each

threat based upon the various factors that can be considered in the analysis such as quantitative and qualitative risk analysis are the objective of stage VII.

Stage VII – Risk Analysis and Management (RAM)

Goal of This Stage The goal of this last stage is to analyze the risk of each threat and attack scenario that was previously simulated, recommend countermeasures that can minimize this risk, and recommend strategies for managing these risks. The initial risk of each threat is calculated based upon qualitative and quantitative risk calculation methods, and the risk after countermeasures and risk mitigations are applied (e.g. residual risk) of each threat is also calculated by considering the effectiveness of each countermeasure and risk mitigation. Finally different risk strategies to manage the technical and business impact of the threat analyzed are also recommended in this stage.

Guidance for Conducting This Stage The objective of the risk analysis and management (RAM) stage is to analyze the various risks of the threats, the attacks that are used to realize these threats, and the vulnerabilities that can be exploited and produce an impact on the application assets that include both data and functionality. The analysis of risk includes the calculation of the risk levels of the analysis of the probability and factors of technical impact such as loss of confidentiality, integrity, availability, accountability as well as business impact such as the various monetary legal impacts to the business that include business continuity loss, fraud monetary loss, noncompliance violations, and data privacy law violation impacts.

For the assessment of technical impact factors and business factors, it is possible to use technical risk and business risk calculation formulas used by each business. Examples of generic risk assessments for technical risks include the standard scoring methods for vulnerabilities such as First CVSS and the categorization of vulnerabilities such as Mitre's CVE. The analysis of technical and business impacts can consider the specific factors for assigning the probability and impact of threats. Once the risks are assigned to each one of the attack scenarios and vulnerabilities identified in the attack simulation analysis, the next step is to recommend the adoption of new countermeasures to reduce the impacts of threats and recommendations for fixing any design flaws and vulnerabilities that were previously identified during the attack simulation tests.

Recommendations for the adoption of countermeasures need to follow a risk mitigation strategy and a risk management process. The risk management process might dictate whether a risk should be mitigated by implementing countermeasures of the risk that can be either transferred to a third party with cyber-insurance or accepted when compensating controls could be identified. Any recommendations of new countermeasure need to consider both the costs and the benefits such as the effectiveness of these countermeasures in reducing the various impacts of threats targeting the application. The cost of countermeasures can be calculated based on their total cost of ownership that includes the cost of acquiring, deploying, and maintaining such countermeasure. The costs of countermeasures need to commensurate with the potential

impacts. The selection of countermeasures should also factor the residual risks after these countermeasures are deployed to determine their effectiveness as a whole. After the countermeasures are identified, the decision of whether to implement these countermeasures need to align with the organization risk mitigation strategies and the objectives for the treatment of risk defined in STAGE I.

Activities This stage consists of the following activities:

RAM 7.1 – Calculate the risk of each threat: The goal of this activity is to analyze the technical impact caused by the attacks that were previously analyzed and factor the probability to determine the risk of each threat to be realized in an attack. Examples of technical impacts include the loss of confidentiality, integrity, availability, and accountability of data and functionality. By associating the probability of each threat occurring based upon the capabilities and motivation of each threat agent, it is possible to assign the risk to each threat attack. Once the technical impacts have been calculated, it is also possible to associate the value of the asset impacted and such as the monetary value of the asset that is at risk to be compromised and quantify the business impact caused the various types of threats and exploits;

RAM 7.2 – Recommend countermeasures and risk mitigations: The goal of this activity is to recommend security measures to detect and protect from the various attack scenarios. Examples of countermeasures are detective and protective security measures and controls that mitigate the impact of attacks against the application assets (data and functionality). Besides identifying countermeasures, the focus of this activity is also to select risk mitigations for the various types of weaknesses and vulnerabilities that attacks might exploit in an attack. Among risk mitigations, design and code changes are also recommended during this stage: these are corrective action plans and design and coding changes whose goal is mitigate the root causes of weaknesses and vulnerabilities such as design flaws or implementation type of vulnerabilities; the types of countermeasures include preventive and detective security controls that are found effective in mitigating the impact of the attacks that were previously simulated. These countermeasures can be deployed in addition to fixing of any design flaws, coding errors, and mis-configuration issues vulnerabilities that were identified to be exploitable with attack simulations and tests previously executed.

RAM 7.3 – Calculate the residual risks: The goal of this activity is to calculate the levels of residual risk both qualitatively and quantitatively after the different countermeasures and risk mitigation options such as design changes for fixing design flaws and code changes to fix coding issues have been deployed in production. The residual risk calculation is the basis to decide if the deployment of countermeasures reduces the risk to a level of LOW risk that is acceptable by the organization. The calculation of the residual risk for a countermeasure depends on the estimate on how much effective is the countermeasure when it is deployed. For example, after the attack analysis and simulation, it is found

that new countermeasures need to be deployed such as two-factor out-of-band authentication solutions that are effective to mitigate MiTM and MiTB type of attacks seeking to compromise single factor and multifactor authentication using traditional hardware and software OTPs. The effectiveness of various OOBA authentication options to mitigate the risk varies depending upon the type of OOBA solution that is selected such as OOBA with biometrics that includes a voice authentication, fingerprints, SMS OTPs, and 2-way 2FA Second Factor Authentication using a completely independent secure authentication channel. The effectiveness of this countermeasure when considered as a whole by applying defense in depth is also enhanced when is used in conjunction with other countermeasures such as antimalware on the client and detective controls on the server to detect anomalies in transactions as indication of compromise. The deployment of these countermeasures in support of the principle of defense in depth is critical to reduce the overall amount of risk for MiTm and MiTB to a residual level of LOW that might be deemed acceptable by the organization based upon the accepted risk mitigation strategy.

By analogy, the deployment of design changes and implementation changes to remediate weaknesses and vulnerabilities can also be considered as countermeasures to mitigate the risk of threat agents and the impact of attack vectors used by these threat agents. Specifically once different type of design options might be implemented to remediate a weaknesses in a security control such as a weakness in a security control that provides data filtering/input validation of malicious commands, the effectiveness of the risk mitigation option can be calculated as a factor to select the remediation that is most effective to mitigate the risk. For example, input filtering for malicious input might include the implementation of an API that filters attack vectors and their encoded versions in different characters to prevent attackers to bypass the filtering. As for new countermeasures, the effectiveness of changes introduced in the application such as deployment of APIs to filter malicious commands from attack vectors increases when these APIs are deployed at different layers of the application architecture such as by deploying NSAPI filters at the web server in the DMZ and servlet filters at the application server located with the internal trusted network.

RAM 7.4 – Recommend strategies to manage risks: the goal of this activity is to recommend the actions that should be taken to treat the risks of each threat and reduce the technical and business impacts to the organization. At this stage of the risk analysis and management, the risk manager has all the information on the table such as the results of the attack simulation exercise, information about the weaknesses, and vulnerabilities that are exploited by these attacks and the selection of different types of countermeasures with different effectiveness and level of residual when deployed as well different options to remediate weaknesses and vulnerabilities previously identified. Each of these countermeasures includes associated costs for design and implementation and effectiveness in reducing the initial risk to a residual level that might be deemed acceptable by the organization.

Based upon this information, it is possible to decide how to treat the risk by recommending different risk mitigation options that align with the organization risk mitigation strategy. This risk mitigation strategy might consist of the following options:

1. Accepting the risk when it is deemed acceptable by the organization based upon the risk level and the presence of compensating controls/measures.
2. Applying the countermeasures/risk mitigations to reduce the risk to acceptable levels.
3. Transferring the risk to a third party such as by signing cyber-insurance.
4. Avoiding the risk such as in the case of deciding not to implement the application feature or storing valued assets that might put the application at risk.

If the risk mitigation strategy followed includes accepting the risk such as by filing a risk exception, it is important that this risk exception is documented and signed off by the business that is ultimately responsible to accept the risk. This risk acceptance certification is also important to provide as visibility to auditors and forensics investigators in the case the organization suffered a security incident whose application impacted was in scope for the application threat modeling exercise to assert whether the root causes of the security incident were also identified as security issues in the application threat modeling exercise whose impact to the business was analyzed and accepted by the business owners including the possible liability of data losses and compromises in the case of security incident exploiting the issues being identified.

If the risk mitigation strategy followed consists of applying countermeasures and risk mitigations, it is important to validate them with security tests after these are deployed and implemented. The results of these tests serve the purpose of certification and assurance that the risks of the threats considered are now mitigated. This assurance also provide visibility to the business owners, risk managers, and auditors who have responsibility in making assertions on whether risks are managed in alignment with business objectives, risk management objectives, and compliance with information security policies and technical standards. At this last stage of the threat modeling exercise, the various security requirements that were identified in stage I can be asserted based upon the results of the threat modeling assessment and the attack-driven security tests cases that were executed in stage IV to validate that these security requirements are validated and tested and the risk of the various threats being considered are now mitigated and closed.

If the risk mitigation strategy consists of transferring the risk to a third party such as in the case of not implementing the countermeasure risk mitigation option but relying on a third party that will assume the costs and the liabilities for implementing it based upon a contract with the business, it is important that the business that accepts to transfer the risk to the third party/vendor also legally bind the liabilities in a legal contract with the vendor. An example of this transfer of risk through transfer of liability is the signing of a contract agreement such as a Service Level Agreement (SLA) with a third-party vendor that takes responsibility in either implementing the countermeasures or taking the risks and the liability in the case of a security incident.

The option of avoiding the risk might be taken by risk management together with the business and the development organization and might include the option of not implementing changes in the application that might increase the attack surface and the risk such as exposing functionality that is considered too risk to be exposed online, not storing secrets such as passwords even if encrypted on mobile devices, not moving business critical services, and customer confidential information on the cloud and other changes that might be considered to bear too much risk for the organization. When making these decisions, it is important to articulate to the decision from business perspective. For example, if a decision ought to be made not to implement a certain application feature/functionality, it is important to commiserate that decision with the potential monetary loss that might be incurred by the business is such application feature/functionality will get compromised by attacks targeting that feature/functionality. If this decision was not made in stage I at the beginning of the assessment based upon the analysis of business requirements and business impacts can be made at this stage after all technical and business risks have been considered.

Input for This Stage This includes one or more of the following:

1. Threat probabilities (stage IV)
2. Severity of the vulnerabilities exploited by each threat (stage V)
3. Exposure of vulnerabilities to each threats (stage V)
4. Attack simulation and tests results (stage VI)
5. Countermeasures and risk mitigations for vulnerabilities and design flaws (stage V)

Artifacts Produced at the End of This Stage At the end of this stage, we will produce the following artifacts:

(a) Analysis of risks each threat including technical and business impacts
(b) List of countermeasures and recommended risk mitigation options for reducing the various impacts of the attacks
(c) Calculation of the residual risks after countermeasures and various options for remediated vulnerabilities are applied
(d) Cost of the countermeasures and risk mitigation options
(e) Recommended risk mitigation strategies for each threat/scenario

Tools The execution of this stage can be facilitated by the use of risk calculators from FIRST such as CVSSvs2, OWASP risk calculation methods, MS™ DREAD for threat-risk severity and industry accepted risk calculation methods for quantitative and qualitative risk calculations. Control risk frameworks that map threats to security controls can be used for identification of countermeasures. Vendor-supplied reports on cost of countermeasures and vendor-unbiased threat intelligence reports on countermeasure effectiveness can also be used. The ThreatModeler™ tool threat management dashboard is also used herein for tracking the risk management of various threats against the online banking application.

Web Application Use Case Example At this stage of the threat modeling exercise, the focus is on the factors that can help to quantify and qualify the risk of each threat so that informed risk decisions for the management of risks can be made. From the information security risk perspective, the main concern is the protection of the confidentiality, integrity, and availability of the data hence the factors that are assessed to quantify and qualify risk are factors of technical impact caused by weaknesses and vulnerabilities in a security control or measures on the confidentiality, integrity, availability, accountability of the data.

A qualitative level of risk can be assigned to the degree of degradation of the security control caused by weaknesses and vulnerabilities that correlate to increased exposure and partial or complete loss of the security attributes of the asset that need to be protected such as Confidentiality, Integrity, and Availability. A step further when this loss is associated to the value of the asset can also be quantified as business impacts that can be either directly or indirectly associated with the business value of the asset lost as well as with the consequences of loss such as additional economic penalties such as fines for noncompliance violations and for data privacy law violations.

Standard methodologies and risk formulas can be used for assigning the various levels for the probability and impact including the standard scoring methods to determine the risk severity of vulnerabilities such as First CVSS and for the categorization of vulnerabilities such as Mitre's CVE. The determination of the business impact caused by the loss of the assets needs to factor the probability of a security incident occurring that might compromise the asset and multiply that probability with the impact that is determined as monetary value expected caused by the security incident. Once technical and business impact are quantified, the next step in the risk management stage is to identify security measures that can reduce the risk including corrective action plans for fixing any design flaws and vulnerabilities that are found exploitable during the attack simulation exercise and validated with attack-driven tests.

The threat risks can be mitigated by reducing the opportunity of attacks to exploit known weaknesses and vulnerabilities in security controls as well as by adopting new countermeasures against these threats. A risk management decision is generally required when the level of risk is higher than that the organization is willing to accept. In this case, the risk manager who ought to make that call whether the risk should be mitigated, transferred, or accepted need to consider both the costs and the benefits of countermeasures. Typically the cost of countermeasures includes the cost of acquisition, deployment, and maintenance and need to commensurate with the potential impacts that the organization might occur if the countermeasure is not implemented. The selection of countermeasures should also factor the effectiveness of the countermeasures and the residual risks after each of these countermeasures are deployed to determine their effectiveness as a whole. After the countermeasures are identified and recommended to executive management and business sponsors, it is important to provide information to support whether the decision of mitigating risks aligns with the organization risk mitigation strategies and the objectives for the treatment of risk that were defined in STAGE I at the beginning of the threat modeling exercise.

The first activity of this stage consists of the activity *RAM 7.1 – Calculate the risk of each threat.* The goal of this activity is to analyze the risk of the threat as technical impact caused on the asset. The risk of a threat can be calculated based upon different factors and methodologies. The risk of threat from probability perspective represents a possibility of that threat to be realized in an attack and cause a compromise. Formulas that can be used to assess the risk of a threat include assigning levels of probability and impact for the threat and multiply with the severity of weaknesses and vulnerabilities using CVSS scoring that the threat seek to exploit and the impact on the assets (data and functions) resulting from the security incident such as data breach where the data is known to be compromised due to the successful attack on the target.

The first step consists of assigning the severity of the vulnerabilities for each security control is affected by the threat based upon the updated information that comes from the analysis of threats, vulnerabilities, and attacks.

The CVSS scoring method can be used for assigning the risk severity using a standard and consistent method that is based on the following factors:

- Base Metrics
  - Access Vector (AV)
  - Access Complexity (AC)
  - Authentication (Au)
  - Confidentiality Impact(C)
  - Integrity Impact (I)
  - Availability Impact (A)
- Temporal Metrics
  - Exploitability (E)
  - Remediation Level (RL)
  - Report Confidence(RC)
- Environmental Metrics
  - Collateral Damage Potential (CDP)
  - Target Distribution (TD)
  - Security Requirements (CR, IR, AR)

To calculate the severity of each vulnerability, the Online Calculator – http://nvd.nist.gov/cvss.cfm?calculator&adv&version=2 can be used and the CVSS vs2 guide can be found herein: CVSSv2 guide – http://www.first.org/cvss/cvss-guide.html.

Once the various risk scores are assigned to the vulnerabilities and weaknesses that map to the threat previously analyzed, the next step consists of assigning the risk likelihood to a threat that is the probability of a threat to occur. For assigning that probability, several methods can be used, but unfortunately none of them is accepted standard. For example, in OWASP risk methodology, threat likelihood factors that are considered are the threat agent factors such as the skills required by the threat

agent to conduct the exploit (e.g. high-level skills correlated to low probability), the motivation/reward (e.g. high reward associated with high probability), opportunity (e.g. full access to resources such as tools to conduct the attack associated with high probability), and size (e.g. the larger the group/population of attacker the higher the probability). The OWASP risk methodology can also be used to calculate vulnerability factors and technical and business impact factors and is documented herein: https://www.owasp.org/index.php/OWASP_Risk_Rating_Methodology and a risk calculator is provided herein: http://paradoslabs.nl/owaspcalc/.

Note: OWASP risk rating methodology is currently under review because it does not provide consistent risk calculations but is reported herein as reference for calculating the threat probability only as one of the methods that can be used. The authors of this book recommend using standard formulas and methods that provide consistent risk ratings for both vulnerability risks and threat probability.

Other nonstandard risk rating methodologies that can be sued to rank the severity of threats include the Microsoft DREAD methodology that is based upon the following factors to calculate the risk of a threat:

- *D*amage potential as the damage if the vulnerability is exploited.
- *R*eproducibility as how easy is it to reproduce the attacks.
- *E*xploitability as how easy is it to launch an attack.
- *A*ffected users as a rough percentage as how many users are affected.
- *D*iscoverability as how easy is it to find the vulnerability.

The risk ranking of a threat can be assigned by using the DREAD Threat Risk Ranking Table http://msdn.microsoft.com/en-us/library/ff648644.aspx#c03618429_011 and by averaging the value with the formula:

$$\text{Risk} = (D + R + E + A + D)/5$$

A more generic formulation for calculating risk is based on the definition of risk as "*The likelihood that threat will exploit a vulnerability to cause an impact to the system*":

$$\text{Risk} = \text{Threat} \times \text{Vulnerability} \times \text{Impact}$$

By putting the emphasis on assets as relate to impact, another empirical calculation for risk can be based on the following definition: "*Risk is the probability of an attack on an asset exposed by vulnerability*":

$$\text{Risk} = \text{Vulnerability} \times \text{Asset} \times \text{Attack}$$

Once the technical impacts have been calculated, it is also possible to associate the monetary value of the asset that is at risk to be compromised and quantify the business impact caused the various types of threats and exploit. If the risk that is calculated is a technical risk, the risk factors considered are the loss of confidentiality,

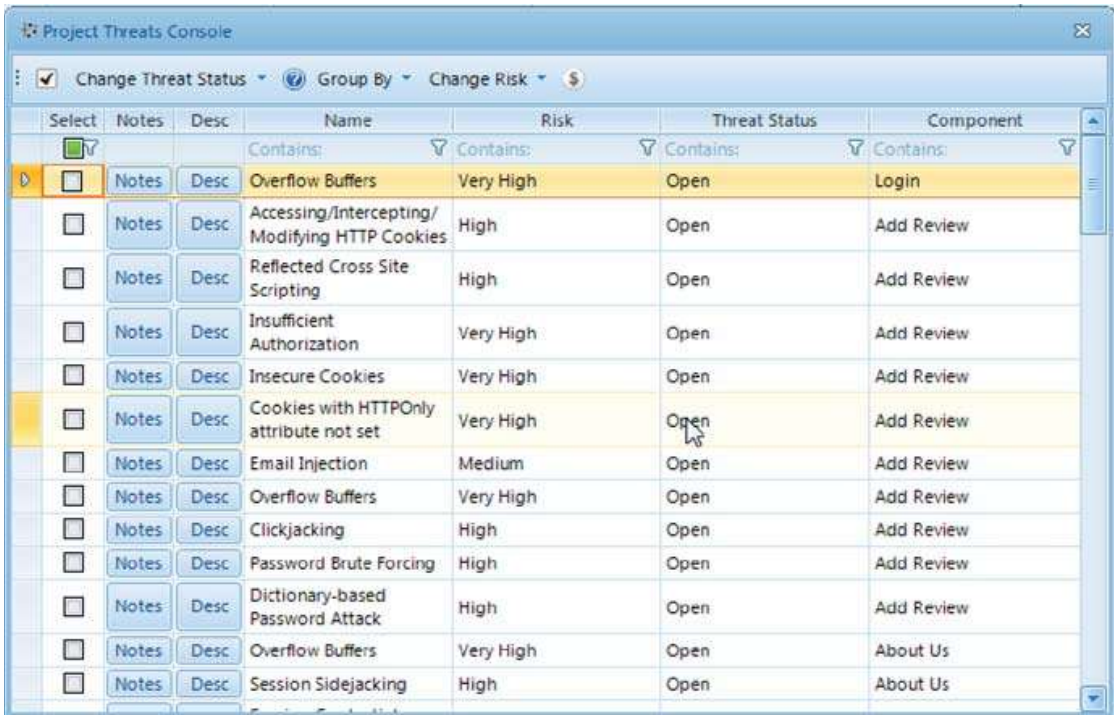


Figure 8.37 Threat and Risk Dashboard

integrity, availability, and accountability of data and functionality. By associating the probability of each threat occurring based upon the capabilities and motivation of each threat agent, it is possible to assign the risk to each threat. The risk for each threat can be calculated, assigned to each threat and tracked for risk mitigation using a threat-risk management dashboard as shown in Figure 8.37.

In the cases when the impact can be associated to the value of an asset, it can be used to determine the business impact. For example, if the maximum value of a functional asset such as money transfer is $ 1 MM per transaction from one account in a single transaction, it is possible to estimate the business impact in the calculation of risk as this transaction can be potentially compromised causing a maximum impact of that amount.

The standard formula that can be used for the quantitative assessment of risk is ALE = ARO × SLO, where (ARO) is the Rate of Occurrence (ARO) or the annual frequency of the security incident causes by that threat and Single Loss Expectancy (SLE) or probability of a loss as a result of a security incident caused by that threat. The SLE can be factored further as by considering the SLE = AV × EF where AV is the Asset Value (AV) and EF is the Exposure Factor (EF).

Factors of probability and impact can also be displayed in the so-called heat map to calculate qualitative level of risk for both technical and business risks. For example, for each specific threat agent, the threat event such as the occurrence of a security incident caused by a malware-based attack or a DDoS can be rated as Very LOW or rare event, LOW or unlikely event, MEDIUM or possible event, HIGH as likely event and Very HIGH as very likely. The level of impacts can be calculated as associated with monetary losses if such security incident ought to occur such as Very LOW for

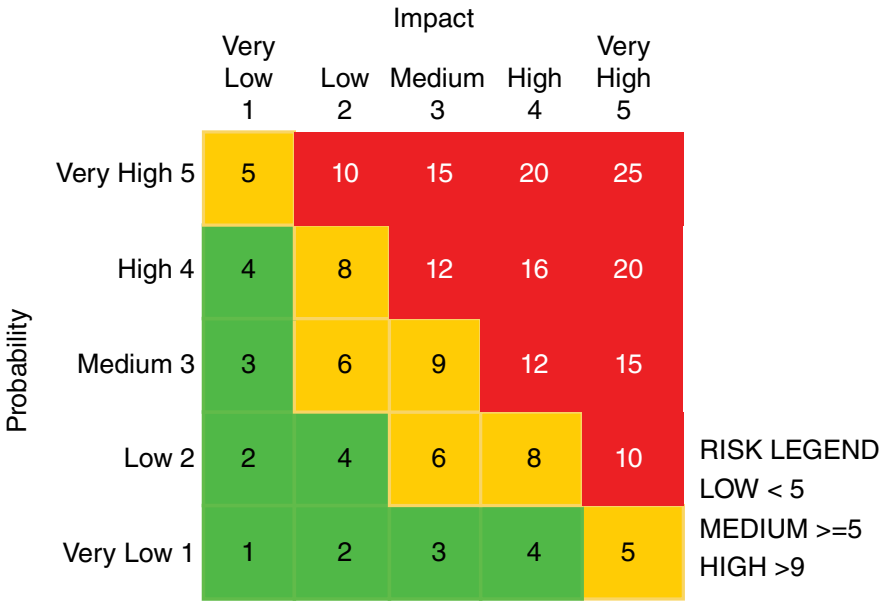


Figure 8.38 Risk Calculation Heat Map

losses up to $100, LOW up to $1 MM, MEDIUM up to $5 MM, HIGH up to $10 MM, and Very HIGH for monetary losses more than $10 MM.

By entering these values in the heat map, it is possible to calculate the overall risk from the business impact perspective. By using the heat map, for example, the monetary impact due to the max amount of $1 MM is considered LOW and the probability of the malware event is considered HIGH, and the overall risk of malware banking attacks is considered MEDIUM. The risk calculation of heat map is shown in Figure 8.38.

In the heat map, the lower band represents the LOW risk, which is the risk that is acceptable based upon the organization risk management policy and appetite for risk. The middle band represents medium level risk and the higher band represents high risk. Both medium and high risks need to be addressed and high-risk remediation has priority over medium risks.

By following one of these risk assessment methodologies, at the end of this activity, the risk manager would have calculated the risk for each threat including technical and business impacts and entered in the risk management dashboard. An example of the risk dashboard showing the status of the threats (open, closed, mitigated, etc.) and the risk associated to the technical risks, which is the risk of the vulnerabilities associated to each threat is shown in Figure 8.39. In risk dashboard, 23 issues are reported to be of very high risk, 38 issues are of high risk, 8 are of medium risk and 5 are of low risk. The risk dashboard help risk managers to prioritize the fixing of issues whose unmitigated threats are considered very high and high risks.

Once each of the threats being assessed has assigned a risk, the next step of the risk assessment and management consists of performing the activity *RAM 7.2 – Identify the countermeasures and risk mitigations*. The goal of this activity is to recommend security measures to detect and protect from the various attack vectors as well as promote changes to the online banking application to remediate vulnerabilities and



Figure 8.39 ThreatModeler™ Threat-Risk Management Dashboard

weaknesses such as design flaws and web application type of vulnerabilities that were previously assessed and validated as exploitable with attack-driven security tests; the term *countermeasures* is referred herein as a broad term that includes preventive and detective security controls as well as fixing of any design flaws and vulnerabilities identified in the threat-driven and attack-driven security tests. In this broad definition of countermeasures for threats, a control gap such as missing authentication for example is also considered a countermeasure.

Ideally at this point of the threat modeling exercise, countermeasures are already documented and mapped to each instance of a threat and attack vector and can be queried through the threat modeling tools for each architectural and functional component of the online banking application. When using a tool such as ThreatModeler™, for example, each functional component has associated security controls that can be implemented to mitigate specific threat targeting the functional component. An example of countermeasures associated with threats to the log-in component is shown in Figure 8.40.

The goal of this activity is to query each of the functional component that the online banking application has been decomposed to the assert if the countermeasures associated with threats have been asserted in design and validated with threat-based and attack-based security tests. For the countermeasures that are not being implemented yet for which the threats are still open and unmitigated hence representing a

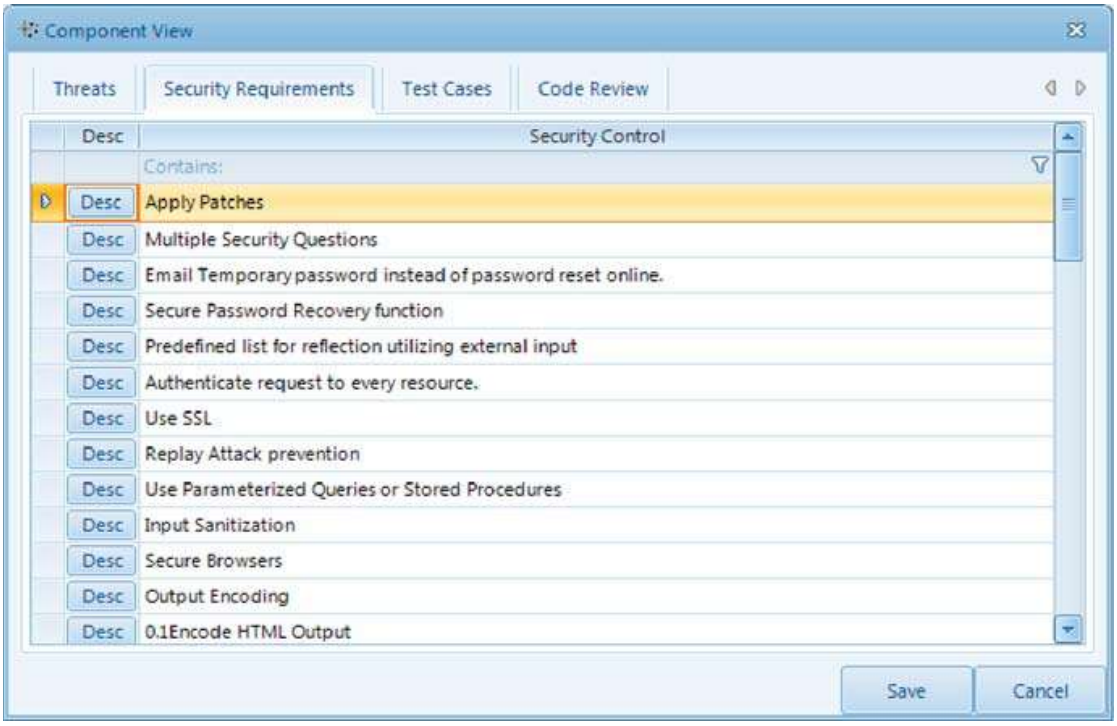


Figure 8.40 ThreatModeler™ Threats to Functional Components and Security Controls That Mitigate These Threats

risk, the goal of this activity is to determine whether these countermeasures should be implemented based upon the risk posed by each threat.

To make this determination, it is important to have an high-level architectural and data flow view of the threats, the attack vectors, the vulnerabilities and vulnerabilities that these threats and attack vectors seek to exploit and the countermeasures that can be deployed within each trust boundary and each data entry and exit point of the architecture to reduce the level of risk. This holistic architectural and data flow view of threats, attacks, vulnerabilities, and countermeasures allows a threat analyst-risk manager to assess the overall risk and the effectiveness of the countermeasures at the different architectural tiers of the online banking application and determine if these can work together to provide risk mitigation as following the various security requirements and security architecture design principles.

An example of this high-level threat and countermeasures holistic view superimposing on DFDs of the online banking application in scope for threat modeling with countermeasures to mitigate the exploit of vulnerabilities by attack vectors that are initiated by various types of threats is shown in Figure 8.41.

This high-level view of threats and countermeasures in place to protect and detect from attacks of various threats is very critical for threat and risk management. Using the military battlefield metaphor, an architectural view of the data flows and view of threats, attacks, vulnerabilities, and countermeasures is as critical for a risk manager to manage the risk of cyber-threats as it is for a military general the view of the battlefield to manage his defensive positions against the enemy. This view of

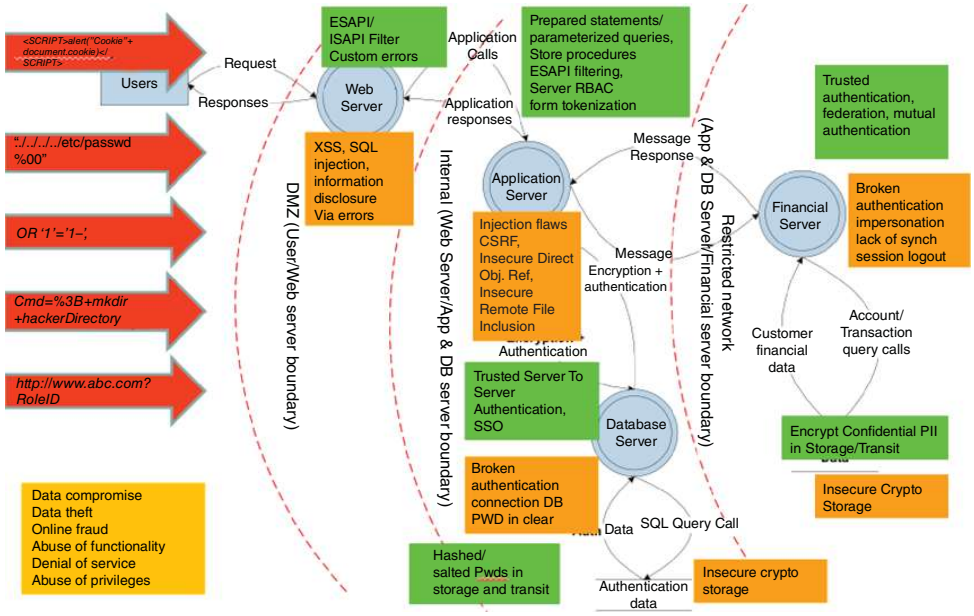


Figure 8.41 High Level View of Threats-Attacks-Vulnerabilities-Countermeasures of Online Banking Application

TABLE 8.23 Security Measures Proposed for Mitigate the Risks of Malware Banking and DDoS Threats

Attack-Vulnerability-Countermeasure Analysis		
Attacks	Phishing, Drive by Download, MiTM, MiTB, MiTM, banking Trojans (Zeus) and C&C botnet control, host and application vulnerability exploits	DDoS reconnaissance, coordinated and targeted DDoS attacks against application and network layer, exploit of web application vulnerabilities
Attack targets	Financial Institutions, card merchants and credit card processors, web and mobile banking sites	Financial institutions, online banking sites, stock exchange web sites, visible credit card web sites
Attack impacts	HIGH: Monetary loss, online fraud, identity theft, wire fraud	HIGH: Reputational Damage via Defacement, Business Disruption, Online Revenue Loss
Attack probability	HIGH: Low cost and risk and high reward for threat agents, low cost of acquisition of cyber crime tools, basic knowledge of hacking tools/techniques	HIGH: Easy recruitment of attackers through social media, availability of free DDoS tools, low cost for renting DDoS botnets, basic knowledge of hacking tools/techniques
Vulnerability exploited	Social engineering, browser/PC vulnerability exploits XSS, SQL Injection, weaknesses in traditional MFA, weakness in virus signature detection, nonrepudiation for connections, indirect	Web Application Vulnerabilities, XSS, SQL Injection, Weaknesses in anti-DDOS defenses at network and application layer, Limited Web Traffic Capability/Architectures
Effective preventive controls	Consumer's education, client antimalware, 2-way Out of Band (OOB) authentication, OOB wire transaction confirmation, maker/checker for transaction initiation/approvals	Anti-DDOS defenses at edge of internet, internet and network (LAN) anti-DDOS layers such as IP filtering, malicious traffic scrubbing and URL blocking at the application layer (e.g. WAF)
Effective detective controls	Fraud detection, suspicious web application activity detection, out of band (e.g. SMS) transactional activity alerts	Network anomaly traffic/bandwidth detection, Real time DDOS Alerts, WAF application layer DDOS attack monitoring

countermeasures represents a defensive baseline that is used as reference to evaluate the security posture of the online banking application to general threats and attacks.

In the case of specific threats such as banking malware and DDoS that are in scope for this threat modeling example, the goal is also to recommend various security measures such as preventive and detective controls that can reduce the attack probability and impact of these threats as shown in Table 8.23.

After the various options of countermeasures have been recommended, the next step in the risk analysis and management stage is to conduct the activity *RAM 7.3 – Calculate the residual risks*. There is a residual risk associated with each threat." We can consider residual risk the risk left after we have applied security controls that reduce the likelihood and impact of the risk posed by a specific threat agent.

The level of reduction of risk provided by the adoption of a countermeasure can be calculated as a factor the risk control mitigation effectiveness such as the reduction of the overall risk factored as likelihood and impact of each threat.

The more effective the countermeasure in mitigating the risk of a threat, the less residual risk caused by the threat. We previously defined in Chapter 5 the amount of "Risk Mitigation (RM)" of the initial risk as a function of the "Control Effectiveness (CE) with the following empirical the formula:

$$\mathrm{RM} = \mathrm{IR} \times \mathrm{CE}$$

The control effectiveness factors both the effectiveness of the control to reduce the likelihood and the impact on the asset value. For example, assuming a countermeasure is 40% effective (it works 40% of the time) and reduces the potential impact by 80%, the overall control effectiveness is 32% (40% × 80%). The amount of Risk Mitigation gained by applying this countermeasure is therefore 32%.

Since there is no one countermeasure alone that mitigates the risk of one threat, it is important to consider the cumulative effectiveness of countermeasures when these are deployed together. This is also very useful for a risk manager to determine if the recommended countermeasures provide an overall reduction of initial risk to an acceptable level.

For example, let us consider that both preventive and detective DDoS countermeasures are applied for mitigating the threat of DDoS and among them some are more or less effective in mitigating the risk. In the case of a DDoS threats, for example, countermeasures might be deployed at different layers of the OSI stack such as OSI layer 7 (application layer) measures such as web application firewall and layer 3 (network layer) countermeasures such as IP whitelists and blacklists, and rate controls. These countermeasures when deployed together can be considered 64% effective that is work 80% of the time and mitigates 80% of the impact. Detective DDoS countermeasures such as monitoring malicious DDoS traffic and issue alerts on suspicious spikes of the network traffic is not considered effective as preventive countermeasures but as effective "defense in-depth" measures since buy the organization that is attacked by DDoS some time to react with DDoS protection countermeasures such as blocking rules for source IPs and scrubbing of malicious traffic.

The Residual Risk (RR) is the amount of Initial Risk (IR) subtracted from the Risk Mitigation (RM) provided by preventive and detective countermeasures when both are deployed to mitigate the risk of DDoS threats. In simple terms, RR can be expressed by the following formula:

$$\mathrm{RR} = \mathrm{IR} \times (1 - \mathrm{CE})$$

Assuming that the initial risk is HIGH (10 risk score) and that an acceptable risk is LOW (less than 5 risk score), the countermeasures deployed for preventing and detecting DDoS attacks need to reduce the level of risk at a level that is acceptable that is LOW risk. Assuming the effectiveness of the countermeasures that are deployed is 64% that will reduce the risk to a level of 3.6 that is considered LOW.

The final activity of the risk analysis and management consist of the activity *RAM 7.4 – Recommend strategy to manage risks.* The goal of this activity is to recommend a risk mitigation strategy to reduce the impact that the exploitation of malware and DDOS threat against the online banking application. In general, a risk mitigation strategy includes four options to mitigate threats:

1. Risk Mitigation – decrease the risk by applying countermeasures, fixing vulnerabilities
2. Risk Acceptance – doing nothing because the risk is either low or is compensated by existing security measures/controls
3. Risk Avoidance – removing the feature that causes the risk
4. Risk Transference – pass risk to an externality/third party using cyber-insurance

A possible risk mitigation strategy could be to fix only MEDIUM- and HIGH-risk vulnerabilities whose cost to design and implement fix is less than the potential business impact derived by the exploitation of the vulnerability. A similar risk mitigation strategy for malware banking attacks, for example, would be to deploy additional countermeasures such as ant-malware software, 2-way out of band authentication for wire transfers, and detective measures such secure event management and log aggregation and behavioral-based fraud detection systems. Another strategy could be to accept the risk when the technical impact such as the loss Confidentiality, Integrity, and Availability implies a small degradation of the service and not a loss of a critical business function. This can be, for example, a viable threat mitigation option for DDoS attacks. In some cases, transfer of the risk to another service provider might also be an option when the cost to implement the countermeasure can be too high and not as effective as being deployed on the premise of the organization's data center such as in the case of adopting cloud services for protection of DDoS attacks.

The decision which strategy is most appropriate depends on different factors that need to be assessed such as the business impact an exploitation of a threat can have, the likelihood of its occurrence, and the costs for transferring (i.e. costs for insurance) or avoiding (i.e. costs or losses due to redesign) it. That is, such decision is based on the risk a threat poses to the online banking application as this is a company asset. Ultimately the overall risk has to take into account the business impact of each threat since this is a critical factor for deciding the risk management strategy.

The risk mitigation strategy is also chosen based upon the effectiveness of the countermeasure considered in reducing the risk to a LOW residual as well as the cost of implementing the proposed countermeasures. In the case of DDoS, for example, assuming the potential economic impact caused by a DDoS attack against a website is $187,506 (year average impact according to 2011 Second Annual Cost of Cyber

Crime Study Benchmark by Ponemon Institute), and a benchmark of control effectiveness in reducing the risk is 64% the max cost that should be allocated annually for rolling our countermeasures for the protection of DDoS is $187,506. Within this boundaries of cost and minimum control effectiveness countermeasures that exceed the effectiveness in mitigating the risk and cheaper to acquire, deploy, and maintain are the one that can be recommended for adoption in the risk mitigation strategy.